

INSPER

Capstone Final Project

Final Project Report - Capstone

**MULTI-CLOUD RESOURCE MONITORING AND ANALYSIS
WITH COST CONTROLS, KPIS, AND CYBERSECURITY**

**ALAN MATHEUS ALVES BARBOSA,
ANTÔNIO AMARAL EGYDIO MARTINS,
ENRICO GEMHA,
ESDRAS GOMES CARVALHO**

São Paulo, November 2024

ALAN MATHEUS ALVES BARBOSA,
ANTÔNIO AMARAL EGYDIO MARTINS,
ENRICO GEMHA,
ESDRAS GOMES CARVALHO

MULTI-CLOUD RESOURCE MONITORING AND ANALYSIS WITH
COST CONTROLS, KPIS, AND CYBERSECURITY

Report presented to the undergraduate programs
in Computer Engineering and Computer Science
as a partial requirement for obtaining the Bachelor's degree.

Advisor:

Prof. Dr. Rodolfo da Silva Avelino

Mentor:

Avner Machado

Capstone Coordinator:

Prof. Dr. Luciano P. Soares

São Paulo, November 2024

**ALAN MATHEUS ALVES BARBOSA,
ANTÔNIO AMARAL EGYDIO MARTINS,
ENRICCO GEMHA,
ESDRAS GOMES CARVALHO**

**MULTI-CLOUD RESOURCE MONITORING AND ANALYSIS WITH
COST CONTROLS, KPIS, AND CYBERSECURITY**

Report presented to the respective undergraduate programs
in Computer Engineering and Computer Science as a partial
requirement for obtaining the Bachelor's degree in the
respective program.

Defense date:

25/11/2024

Examination Board:

Prof. Dr. Marcelo Hashimoto

Inspere

Prof. Maciel Calebe Vidal

Inspere

ABSTRACT

This work aims to improve the infrastructure monitoring system across multiple cloud providers for PinPag, a Brazilian company specialized in payment solutions, which currently uses Amazon Web Services and Azure. The Key Performance Indicators will be enhanced, infrastructure monitoring flows will be restored, Web Application Firewall coverage will be extended to more applications, vulnerability monitoring in code will be implemented with SemGrep, real-time alerts for failure communication will be configured, and data presentation will be simplified in Grafana dashboards. With these improvements, it is expected to enhance infrastructure visibility and control, support internal decision-making, increase application security, and expedite incident response.

Keywords: *multi-cloud* monitoring, cost control, information security, alert system, Security Information and Event Management, Web Application Firewall, Key Performance Indicators.

SUMÁRIO

1	INTRODUCTION	6
1.1	The Business	6
1.2	The Multi-cloud Environment	7
1.2.1	Multi-cloud Challenges	9
1.3	State of the Art	9
2	FIRST PHASE OF THE PROJECT	11
2.1	Introduction and Initial Proposal	11
2.2	Definition of KPIs and Software	11
2.3	Solution Architecture	13
2.3.1	Grafana Integration	13
2.3.2	AWS Information Flow	15
2.3.3	Azure Information Flow	18
2.4	Final Considerations of the First Phase	19
2.4.1	Presentation of Results	19
2.4.2	Objective Achievement	22
3	SECOND PHASE OF THE PROJECT	23
3.1	Analysis of PinPag's Needs and Challenges	23
3.1.1	Stakeholders and their Needs	24
3.1.2	Main Challenges and Points of Attention Identified	24
3.2	Technical, Ethical, and Risk Guidelines	26
3.2.1	Certifications and Standards Followed by the Company	27
3.2.2	Ethical Guidelines Followed by the Project	28
3.3	Proposed Solutions	29
3.3.1	Alert System	30
3.3.2	Web Application Firewall (WAF)	30

3.3.3	Connection Map	31
3.3.4	Application Security	33
3.3.5	Infrastructure Panel	38
3.3.6	Solution Integration	39
3.3.7	Expected Benefits	39
3.3.8	Related Work	40
3.4	Methodology	43
3.4.1	Inspirations	43
3.4.2	Communication	45
4	IMPLEMENTATION OF PROPOSED SOLUTIONS	47
4.1	Reestablishment of Monitoring Infrastructure	47
4.1.1	AWS Infrastructure Correction	47
4.1.2	Azure Correction	54
4.1.3	Wazuh Correction	55
4.2	WAF	59
4.2.1	ModSecurity 3.0	59
4.2.2	WAF Dashboard	61
4.3	Connection Map	67
4.3.1	General Operation and Data Structuring	68
4.3.2	Algorithm Detail	69
4.3.3	Implementation and Tests in Insper Environment	71
4.3.4	Migration and Implementation in PinPag Infrastructure	72
4.4	Application Security	74
4.5	Infrastructure Dashboard	78
4.6	Alert System	79
4.6.1	Metrics Selected for the Alert System	80
4.6.2	Alert System Operation	80
4.6.3	Infrastructure and Cost Alerts Configured in Grafana	81

4.6.4	Initial Tests and Alert Adjustments in Grafana	82
4.6.5	Application Security Alerts	82
4.7	Item Removal Policy	83
4.7.1	Updates Implemented in Code	83
5	COST ESTIMATION	85
5.1	AWS Cost Estimation	85
5.1.1	Infrastructure Overview	85
5.1.2	Amazon S3	86
5.1.3	AWS Lambda	87
5.1.4	AWS Glue	88
5.1.5	Amazon Athena	90
5.1.6	Total Estimated Monthly Cost in AWS	90
5.1.7	Cost Scenarios	91
5.2	Azure Cost Estimation	91
5.2.1	Infrastructure Overview	91
5.2.2	Azure Storage Blob	92
5.2.3	Azure Data Factory	92
5.2.4	Azure Data Explorer	93
5.2.5	Total Estimated Monthly Cost in Azure	94
6	CONCLUSION	95
6.1	Next Steps	97
6.1.1	[section removed]	97
6.1.2	WAF Enhancement with New KPIs, Centralized Integration[removed]	97
6.1.3	Enhancement of Connection Map	98
6.1.4	Expansion of Application Security with SemGrep and Complementary Tools	99
6.1.5	Enhancement of Analysis Capability in Infrastructure Dashboard	100
6.1.6	Refinement and Optimization of Alert System	100

6.1.7	Implementation of Staging Environment and Resilience Tests	100
-------	--	-----

REFERÊNCIAS	102
------------------------------	------------

APÊNDICE A – MODSECURITY INSTALLATION MANUAL	107
---	------------

A.1	Complete Guide for Installing and Verifying ModSecurity 3.0 on Apache in Linux (Considerations for Windows Environment)	107
------------	--	------------

A.1.1	Introduction	107
-------	------------------------	-----

A.1.2	Test Machine Information	107
-------	------------------------------------	-----

A.1.3	Prerequisites	107
-------	-------------------------	-----

A.1.4	Installation Step-by-Step	108
-------	-------------------------------------	-----

A.1.5	Installation Verification	111
-------	-------------------------------------	-----

A.1.6	Final Considerations	111
-------	--------------------------------	-----

1 INTRODUCTION

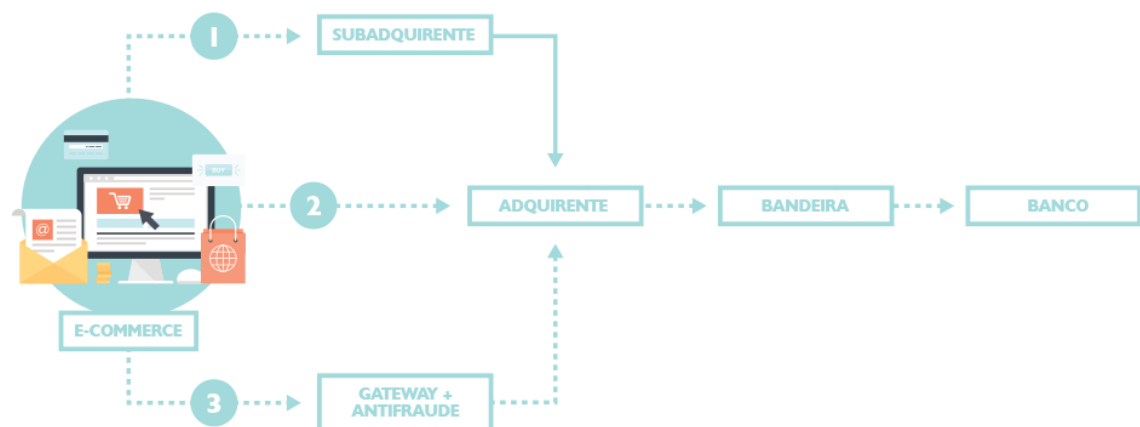
1.1 THE BUSINESS

PinPag is a Brazilian company specialized in payment solutions, offering services such as credit and debit card transaction processing, purchase installment, receivables anticipation, and payment system integration with management *software* (PinPag, 2024a). Its focus is on serving small and medium-sized enterprises. Currently, PinPag serves over 20,000 establishments, processing approximately 10,000 daily transactions (PinPag, 2024b).

Operating in the Brazilian Payment System (SPB), regulated by the Central Bank, and internationally, PinPag must comply with strict regulations related to the collection and processing of financial data, in addition to meeting certifications such as PCI-DSS (*Payment Card Industry Data Security Standard*) and following best practices in information security management, according to ISO 27001. These standards and certifications ensure security in financial transactions and were detailed in Subsection 3.2.1.

The SPB enables the movement of funds between individuals, companies, and financial institutions, ensuring the security and efficiency of transactions, such as Pix, TED, bank slips, checks, and credit and debit card payments (Iugu, 2021).

Figura 1.1 – Flowchart of the payment process in the SPB, for an *e-commerce*



Source: Algar Telecom (2017).

As shown in Figure 1.1, the payment process in the SPB involves the acquirer, responsible for processing credit and debit card payments, connecting the merchant to card brands and financial institutions. In addition, a sub-acquirer, such as PinPag, may

be involved.

The sub-acquirer acts as an intermediary between merchants and acquirers, simplifying the integration of establishments into the payment system, offering transactions through multiple acquirers, with more competitive prices. It also offers services such as receivables anticipation, technical support, card machines, and integrated payment systems. Thus, the process becomes more accessible, especially for small and medium-sized establishments ([PagSeguro, 2022](#)).

PinPag's trajectory reflects the dynamism of the *fintech* market in Brazil. In January 2020, the company was acquired by Linx, one of the largest management *software* providers for retail in Brazil, for R\$ 135 million. This acquisition aimed to strengthen Linx's *fintech* division, Linx Pay ([Mergr, 2020](#)). In August 2020, StoneCo acquired Linx for R\$ 6.8 billion, expanding its presence in the payments sector ([Brazil Journal, 2020](#)). In February 2024, PinPag was reacquired by an anonymous buyer ([Market Screener, 2024](#)).

To ensure operational efficiency and meet regulatory requirements, PinPag uses a *multi-cloud* architecture. This approach isolates the production environment, where its main financial services are hosted, from the rest of the infrastructure, facilitating compliance with international security standards.

1.2 THE MULTI-CLOUD ENVIRONMENT

The global data market is experiencing rapid growth, with a compound annual growth rate of 19.2% projected between 2020 and 2025 ([Statista, 2021](#)). This increase is strongly linked to the growing phenomenon of digitalization worldwide, with the internet reaching 5.4 billion active users ([Statista, 2023](#)). In Brazil, in 2021, 90% of households already had internet access ([IBGE - Agência de Notícias, 2022](#)), which further drives the volume of data generated in the country.

This expansion is not limited to general data collection, but also has an impact on the banking sector, especially in Brazil, where banking transactions conducted via mobile devices grew by 251% ([FEBRABAN, 2024](#)). At the same time, consumers are increasingly demanding regarding service agility. According to Zendesk ([Zendesk, 2023](#)), customers expect fast and effective responses, a scenario even more challenging for companies, which need to deal with the increased use of digital services, generating growing volumes of data to be processed in even shorter time.

In the past, large companies in the financial sector, such as Itaú, faced the challenge of data storage and processing by developing proprietary *data center* solutions

([Valor Econômico, 2020](#)). These were composed of multiple computers and servers, being responsible for storing and processing large volumes of information. In this type of infrastructure, for example, when a customer used the Itaú app to log in, their credentials, such as username and password, were validated by physical servers located at the company's own facilities. Although this model guaranteed total control over customer data, it presented several challenges. These challenges included high energy consumption to keep computers running, the need for periodic *hardware* renewal, costs of robust cooling systems to prevent overheating, and increasing cybersecurity risks ([Volico, 2024](#)). These are factors that require high and constant investment for infrastructure maintenance and protection.

Faced with these difficulties, new approaches emerge, such as cloud computing, which eliminates the need to maintain one's own infrastructure. Instead, companies can use computational service providers, such as, for example, Amazon Web Services (AWS), Azure, or Google Cloud Platform (GCP), which assume responsibility for maintenance, updating, and infrastructure operation. These providers ensure that resources are always available, guaranteeing high availability, which is a percentage metric, usually given by the ratio of execution time without interruptions to the total elapsed time, and is especially critical for companies that depend on uninterrupted services, seeking to remain between 99.9% and 99.99999%. This value indicates how reliable and stable the system is over time, and each additional "nine" indicates an order of magnitude toward 100% availability ([BEYER et al., 2016](#)).

To achieve this, there are several ways to configure cloud computing architecture. One of them is the use of a single cloud, called *single cloud*, where the company centralizes all operations and data in a single provider, benefiting from agility in environment configuration and simplicity in resource management ([Alert Logic Product Management Team, 2022](#)).

On the other hand, the *multi-cloud* environment, in which the company distributes its operations among different cloud providers, offers greater flexibility, resilience, and availability, avoiding *vendor lock-in*, exclusive dependence on a single supplier ([Alert Logic Product Management Team, 2022](#)). The *multi-cloud* model ensures more independent environments, allowing the company to quickly adapt or migrate in case of failure or change in a provider's platform, increasing its security and availability.

Form3, a global payment processor based in the United Kingdom, is an example of adopting *multi-cloud* infrastructure, due to its resilience and flexibility ([Cockroach Labs, 2023](#)). Previously, the company used only AWS for its payment solutions, but with increased regulatory requirements, it chose to migrate its architecture, integrating AWS, GCP, and Azure providers. This approach allowed Form3 to increase security in international transactions.

Returning to the Brazilian scenario, PinPag incorporated *multi-cloud* architecture to increase the flexibility of its operation and meet local regulatory requirements. Amid a revolution in the Brazilian financial system, driven by *Open Finance* and the innovation of proximity Pix, the country stands out globally for its leadership in the sector, creating demands that companies, such as PinPag, need to meet through agile and secure operations in an environment of constant change ([Revista HSM, 2024](#)).

1.2.1 Multi-cloud Challenges

This management presents challenges that need to be considered, among which the following stand out:

- **Fragmented Visibility:** Monitoring multiple clouds can be complex, since each provider offers its own tools and control panels, making it difficult to obtain a unified view of the infrastructure.
- **Data Transfer Costs:** Operating between clouds frequently generates additional costs, such as fees associated with data transfer between regions and providers.
- **Security Management:** Each provider has distinct security policies and tools, making it difficult to maintain a consistent security policy across the entire infrastructure.

In PinPag’s context, these challenges manifest in the need to rigorously control costs distributed among different providers, effectively integrate monitoring systems, and ensure data security and compliance in diverse environments.

1.3 STATE OF THE ART

In the context of cloud computing architectures, the most modern solutions and techniques focus on optimization, security, and cost management in *single cloud* and *multi-cloud* environments. Practices are shaped according to the individual needs of the company, but generally seek to meet demands for high availability, flexibility, and scalability, while ensuring regulatory compliance and data security.

Based on the challenges presented in Subsection 1.2.1, the first stage of the project developed in partnership with PinPag (see Section 2) selected the Grafana tool as an additional solution for monitoring and management, complementing Zabbix and Wazuh, which were already in operation at the company. The functions of these solutions in PinPag’s *multi-cloud* environment are:

1. **Grafana:** Interface for creating customized panels and visualizations, which centralizes monitoring, displaying *Key Performance Indicators* (KPIs) and performance data collected from different cloud providers.
2. **Zabbix:** Infrastructure monitoring tool, used to monitor resource usage such as *Central Processing Unit* (CPU), *Random Access Memory* (RAM), and disk, providing a clear view of server and instance performance in AWS and Azure clouds.
3. **Wazuh:** Security monitoring and threat detection platform. Wazuh was responsible for vulnerability analysis and security alert generation, ensuring compliance with standards such as PCI-DSS and identifying possible security incidents in real time.

These systems are essential for monitoring the proper functioning of resources, services, and infrastructure. They track the use of computational resources, such as CPU, memory, and storage, monitor security, and provide real-time visibility of performance and costs. In *multi-cloud* environments, they are particularly important for integrating data from different providers into a single platform, ensuring efficiency and security in operation.

2 FIRST PHASE OF THE PROJECT

2.1 INTRODUCTION AND INITIAL PROPOSAL

After understanding the current scenario and the challenges related to cloud computing architecture and the Brazilian Payment System, a reassessment of the first phase of the project is necessary. This stage, carried out in partnership between Insper and PinPag, produced a solution for infrastructure monitoring, cost control, and security in *multi-cloud* environments.

2.2 DEFINITION OF KPIS AND SOFTWARE

The project began with the definition, together with PinPag, of the KPIs to be developed. These indicators are essential to ensure control of the company's financial and operational efficiency. These indicators are presented in Table 2.1.

The second step was to conduct state-of-the-art research, in order to identify the most suitable *software* to meet PinPag's needs. The research sought solutions that offered high service availability (with long-term support), flexibility (avoiding subscription or recurring payment dependencies), and low cost, considering PinPag's budget constraints.

To achieve **availability** goals, only open-source software with strong communities was chosen, ensuring continuous support and regular updates. **Flexibility** was achieved by being open-source and self-hosted platforms, allowing PinPag to host these solutions on a machine, on any cloud provider. Although there was no predetermined budget, this led to **low cost**, as PinPag controls and optimizes hosting resources and does not pay for licenses or recurring subscriptions. Thus, the following solutions were chosen:

- **Grafana**: a data visualization platform that allows the creation of interactive panels, in which it is possible to monitor and visualize the KPIs (key performance indicators) defined to track infrastructure and cost performance in real time.
- **Wazuh**: a tool that collects, visualizes, and shares data on security *vulnerabilities*. With it, PinPag can monitor possible threats and ensure compliance with PCI-DSS guidelines.
- **Zabbix**: a network and server monitoring solution that allows tracking the health of the entire infrastructure, alerting to possible problems and ensuring that services remain operational.

- **CloudWatch:** an AWS service used to monitor specific metrics of cloud instances. It complements Zabbix by providing detailed information about resource usage on AWS, such as CPU, memory, and network utilization.

Tabela 2.1 – KPIs used and their benefits

KPI	Definition	Benefits
Cost per Application/-Project	Total resource expenditure for each application or cloud project, grouped by <i>tags</i> (AWS) or resource groups (Azure).	Identifies projects with high consumption and optimizes resource allocation.
Cost per Service	Total expenditure per service (storage, computing, etc.).	Identifies the most expensive services and optimization opportunities.
CPU, Memory, and Network Utilization	Measures CPU, memory, and network usage over time.	Monitors efficiency and adjusts resources according to demand.
Weekly Cost	Daily cost of all resources over a week (AWS daily, Azure cumulative).	Compares weekly expenditures with budget for optimization.
Mean, Median, and Standard Deviation	Statistical analysis of costs throughout the month.	Identifies patterns and <i>outliers</i> in monthly costs.
Weekday/Weekend Usage	Compares resource usage between weekdays and weekends.	Helps adjust resources based on seasonal patterns.
Tagged Resources	Percentage of tagged resources compared to untagged ones.	Facilitates resource categorization and optimization.
Cost Change per Service	Compares current expenditure with the previous month for each service.	Detects cost increases for corrective strategies.
Monthly Cost Comparison	Compares total cloud expenditure between current and previous month.	Identifies cost trends for financial control.
Average Price per Computing Hour	Average cost per hour of computing resources in the month.	Evaluates effective cost of operations and optimizes decisions.
Cost per Region	Expenditure by geographic location of resources (Azure).	Identifies more expensive regions to optimize allocation.

Source: [Frezzatti et al. \(2023\)](#).

After defining the KPIs and choosing the software to be used, the construction of the data collection architecture from AWS and Azure was initiated. Instead of using the

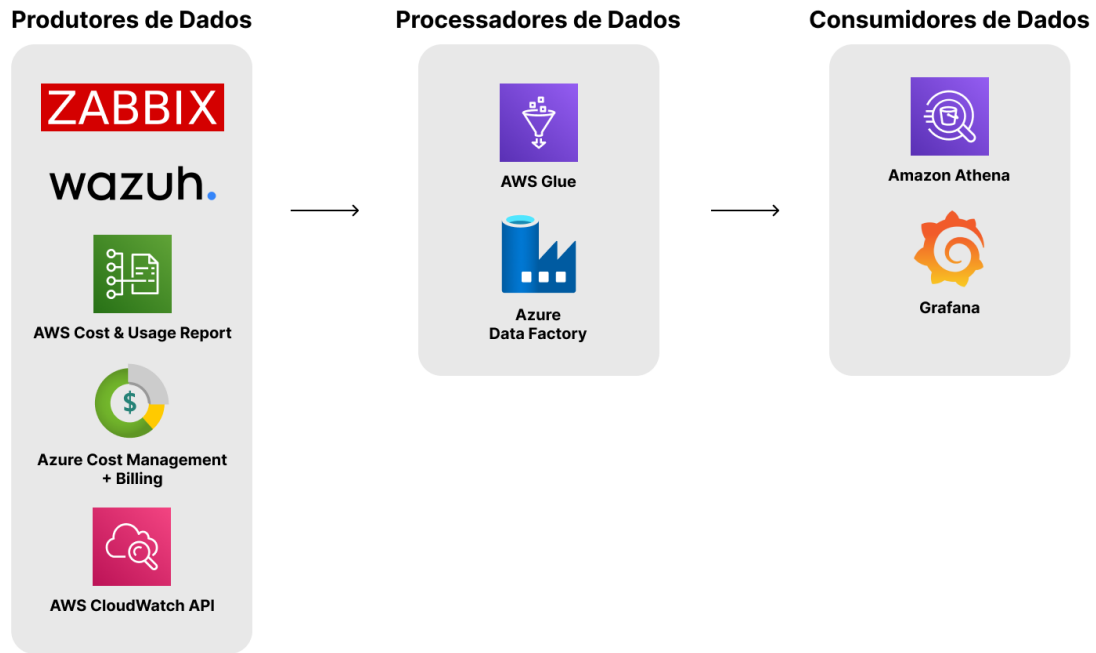
ready-made solutions offered by each cloud platform, the group chose to develop its own flow for data generation, collection, and export. This decision was motivated by budget constraints, allowing greater control over costs and customization of the solution according to the company's needs.

2.3 SOLUTION ARCHITECTURE

2.3.1 Grafana Integration

Grafana was the main tool used for building visualizations, presenting graphs and tables generated from data consumed from other software integrated with the solution. These graphs were fundamental for monitoring and analyzing the defined KPIs, consolidating information from various sources.

Figura 2.1 – Solution Architecture.

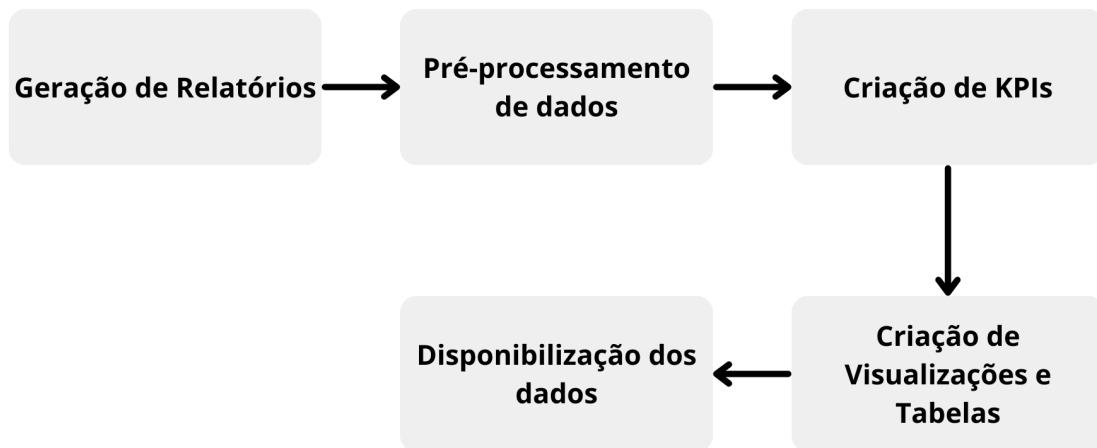


Source: Prepared by the authors.

As shown in Figure 2.1, the solution architecture is divided into three main components: Data Producers, Data Processors, and Data Consumers. Data Producers include monitoring tools such as Zabbix and Wazuh, cost reporting services such as AWS Cost & Usage Report and Azure Cost Management + Billing, and *logging* services such as AWS CloudWatch API. Data Processors are responsible for data transformation and preparation, with AWS Glue and Azure Data Factory. Finally, Data Consumers are Amazon Athena and Grafana, which make processed information available for visualization and analysis.

As shown in Figure 2.2, the data flow follows well-defined stages, both in AWS and Azure, with each phase playing a specific role, allowing precise validation. This clear division of responsibilities is essential for identifying failures or improvements in the process, ensuring that data is processed and visualized effectively and reliably. The implementations of the solution architectures were detailed in subsequent sections.

Figura 2.2 – High-Level View - AWS and Azure Architecture.



Source: Prepared by the authors.

1. **Report Generation:** In AWS and Azure, daily reports are generated containing monetary data from all services used. These reports are normally produced in Parquet format, which is widely used for its efficiency in compression and fast querying, enabling effective processing of large volumes of data. This format is essential for managing the complexity and volume of data generated in cloud environments.
2. **Data Preprocessing:** Not all data contained in generated reports is used directly for creating KPIs. In addition, some fields, such as dates and monetary values, need to be correctly formatted to ensure consistency in analyses. Data preprocessing involves filtering irrelevant information, as well as transforming and standardizing data, preparing them for the next phases of the process.
3. **KPI Creation:** After preprocessing, data is used for creating KPIs, as described in Section 2.2. This stage involves applying specific queries to generate metrics that evaluate the performance of cloud services and resources. Each KPI is calculated based on monetary data and resource utilization, providing a clear and detailed

view of PinPag’s infrastructure performance. These KPIs are stored in an organized manner for later queries.

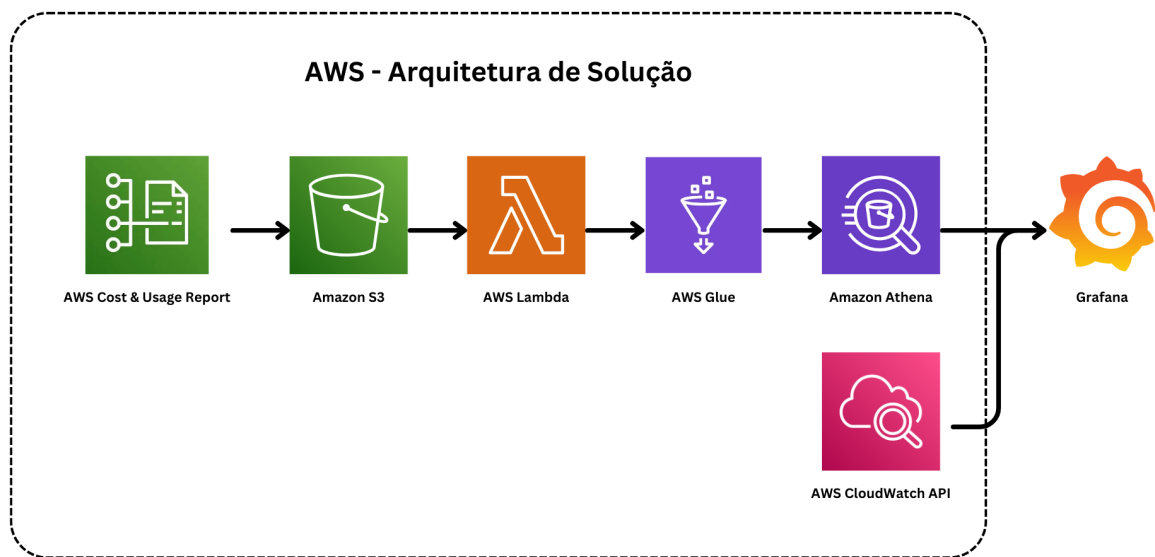
4. **Creation of Visualizations and Tables:** To facilitate consumption of KPIs by Grafana, standardized visualizations and tables are created. These visualizations organize and display the data of each KPI in a structured way, allowing Grafana to integrate these data efficiently. This makes it possible to generate graphs and panels that provide a clear and immediate view of the performance and costs of the monitored infrastructure.
5. **Data Availability:** Up to this stage, processed data is used internally by cloud providers. However, for this data to be consumed by external services, such as Grafana, it is made available through a secure authentication process. This process involves the use of a *Client ID* and a *Secret Key*, ensuring that only authorized services have access to the data, ensuring privacy and protection against unauthorized access.

Additionally, Wazuh, Zabbix, and CloudWatch (included in the AWS architecture) play a crucial role in data collection. They capture necessary metrics and security information, and send them directly to Grafana. Grafana acts as a central interface for consolidated visualization of data from different sources, but without generating or modifying them.

2.3.2 AWS Information Flow

In AWS, the first step was to analyze the available tools for information collection and processing. During this evaluation, it was highlighted that costs associated with the use of existing *Application Programming Interfaces* (APIs), such as Cost Explorer, were above the budget stipulated by PinPag. With this, an alternative architecture was proposed that would offer the same results, but using more economical methods adjusted to the company’s financial needs.

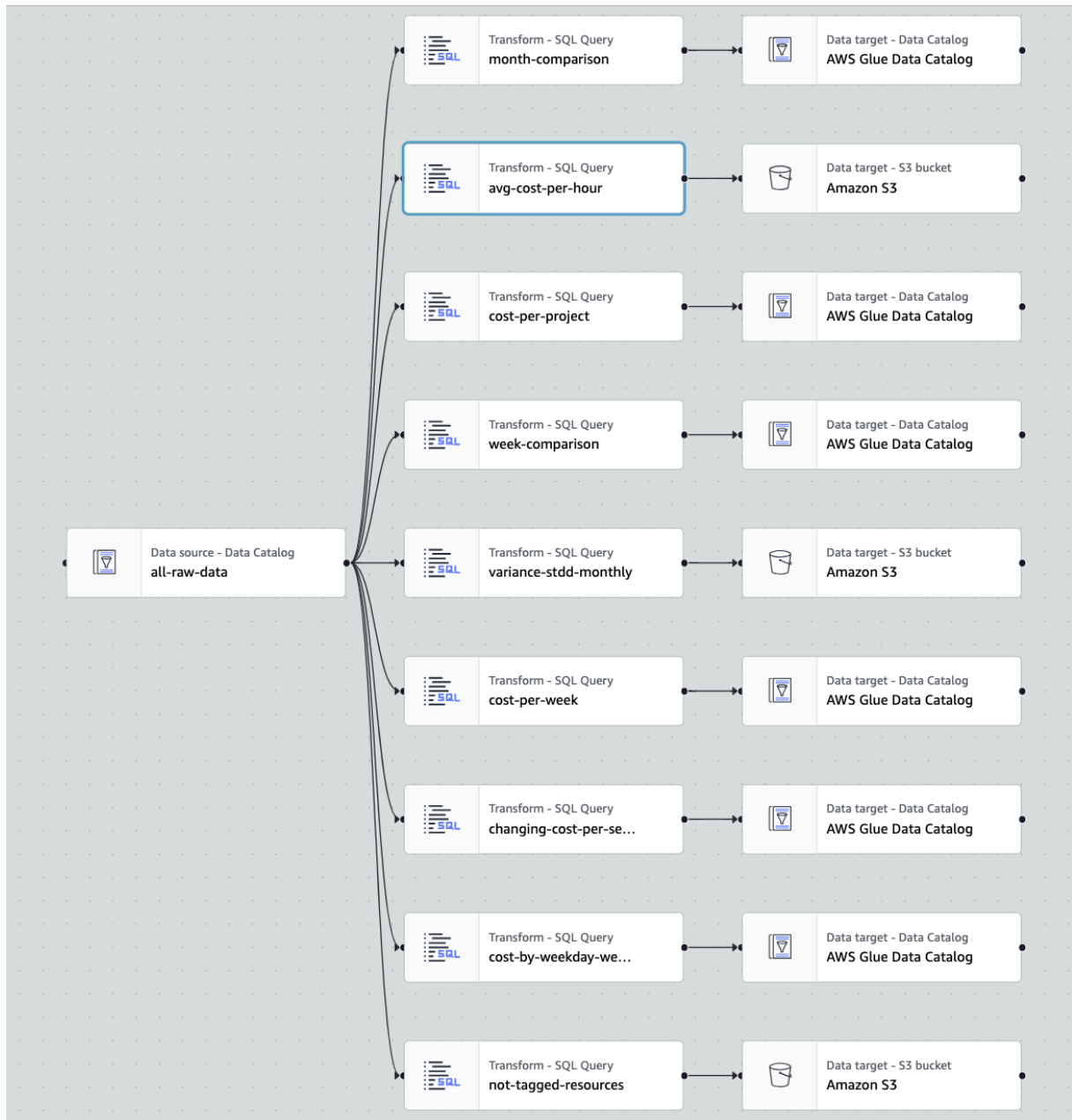
Figura 2.3 – AWS Architecture.



Source: Prepared by the authors.

1. **AWS Cost & Usage Report (CUR):** This service collects detailed information about the usage and costs of all active services in AWS over a specific period. It generates reports in Parquet format, which are processed in subsequent stages.
2. **Amazon S3:** Used to store raw data and processed KPIs. S3 stores cost reports, AWS Glue data, and serves as a base for AWS Athena queries.
3. **AWS Lambda:** Serverless compute service that is triggered by events. In this project, it is triggered after data is registered in AWS Glue, initiating processing through a *job* in AWS Glue.
4. **AWS Glue:** ETL service that uses *crawlers* to catalog data in S3 and process it. Glue transforms raw data into KPIs based on *Structured Query Language* (SQL) queries and stores the processed results in S3.
5. **AWS Athena:** Service that allows querying data stored in S3 using SQL. It accesses data processed by AWS Glue and facilitates efficient analysis without the need for additional infrastructure.

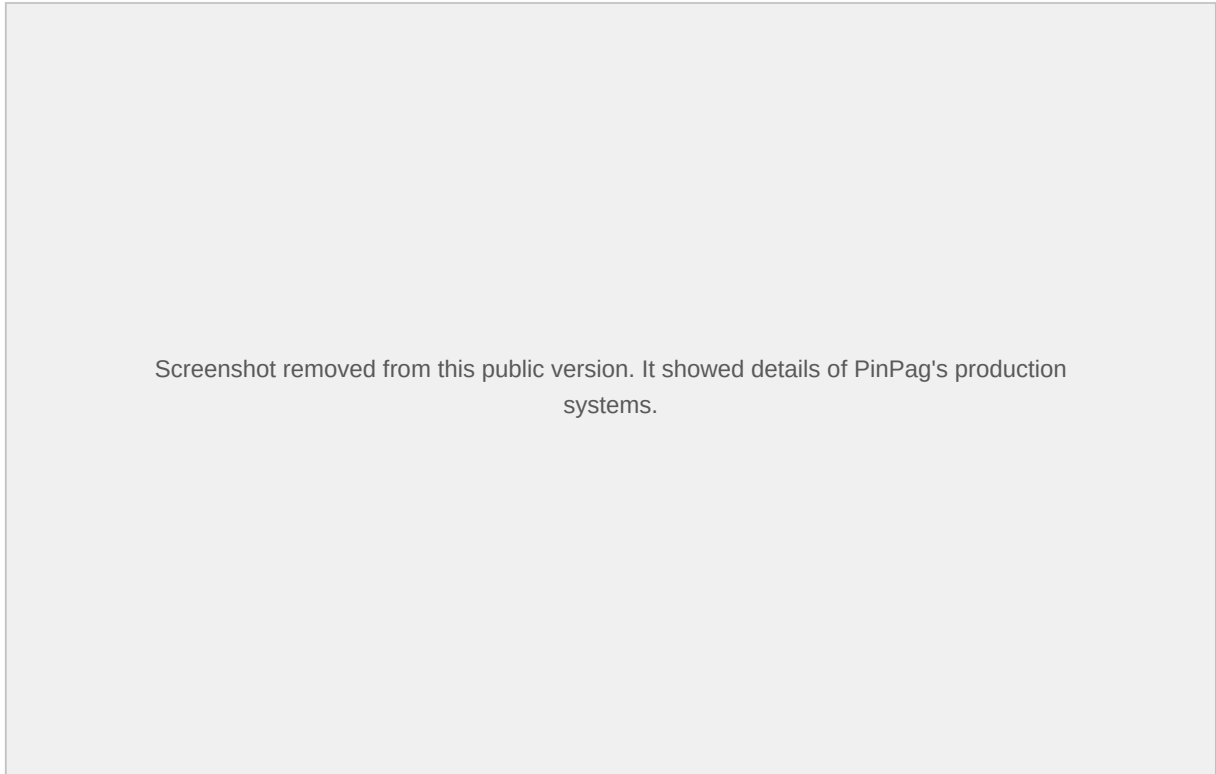
Figura 2.4 – AWS Glue Processing Job.



Source: Prepared by the authors.

To validate that all stages of the process are working correctly, it is possible to perform verification directly in Amazon S3, ensuring that data is being updated at least once a day. In addition, it is essential to analyze the processed KPIs, evaluating whether they accurately reflect the company's context in AWS.

Figura 2.5 – Example of Average Cost per Hour Table.



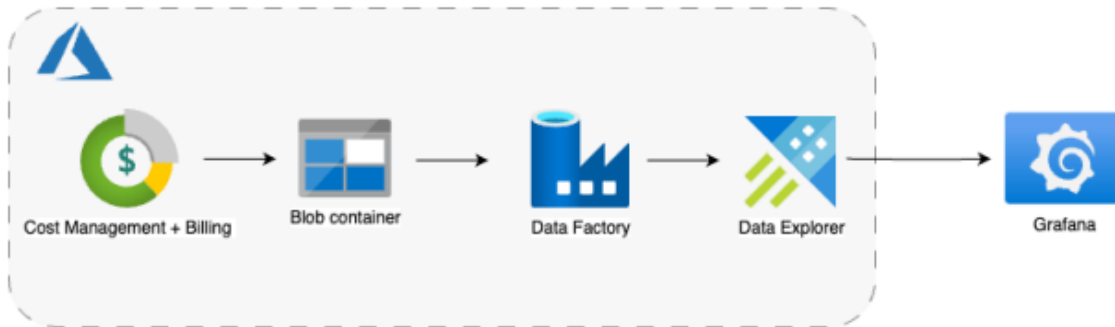
Source: Prepared by the authors.

As illustrated in Figure 2.5, AWS Athena displays multiple tables with the content of each processed KPI, which can be visualized by selecting in the left column. In the example shown, the **Average Cost per Hour** table is selected, and it is possible to observe, in the data displayed at the bottom, that results are being generated correctly, which validates the accuracy and consistency of the process.

2.3.3 Azure Information Flow

In Azure, as in AWS, the economic infeasibility of using proprietary Cost Management APIs was identified, which is responsible for providing detailed data on the usage and costs of cloud services. This API allows expense analysis, identification of optimization opportunities, and budget control in real time. However, due to the high costs involved, an architecture similar to that used in AWS was proposed. This new architecture exports cost and usage reports, which are stored in a database, processed, and made available for consumption in an economical and efficient manner.

Figura 2.6 – Azure Architecture.



Source: [Frezzatti et al. \(2023\)](#).

1. **Cost Management + Billing:** Provides a detailed view of cloud service costs and usage, helping to manage expenses and identify optimization opportunities. Due to the high cost of proprietary APIs, a more economical approach was chosen to capture and process data.
2. **Blob Container:** Equivalent to Amazon S3, Blob Storage was used to store both unprocessed cost reports and already processed KPIs, ensuring scalability and security for large volumes of data.
3. **Data Factory:** Data integration service that orchestrates processing flows. It was used to process collected data and generate KPIs, with the result stored in the Blob Container for later query.
4. **Data Explorer:** Data analysis platform that enables fast queries on large volumes of data. Used to create requests and visualizations, making processed data available to Grafana. Authentication follows the model seen in AWS of *Client ID* and *Secret Key*, ensuring secure access.

2.4 FINAL CONSIDERATIONS OF THE FIRST PHASE

2.4.1 Presentation of Results

With the completion of the AWS and Azure data flow, and the configuration of Zabbix and Wazuh monitoring tools, four visualizations were created in Grafana to facilitate data monitoring and analysis:

1. **Cost Explorer AWS:** The AWS Cost Explorer visualization, as shown in Figure 2.7, offers a unified view of AWS-related costs, based on previously defined KPIs (see Section 2.2). This visualization allows the company to monitor AWS infrastructure spending in real time.

2. **Cost Explorer Azure:** Similar to the AWS visualization, the Azure Cost Explorer, illustrated in Figure 2.8, offers a detailed view of Azure cloud infrastructure costs, enabling effective analysis and control of platform resources.
3. **Security Zabbix:** The Zabbix security visualization, as shown in Figure 2.9, focuses on infrastructure monitoring, highlighting resource usage such as CPU and RAM Memory. This visualization allows monitoring the performance and health of monitored servers.
4. **Security Wazuh:** The Wazuh security visualization, shown in Figure 2.10, is focused on cybersecurity, recording alert levels for each of the monitored computers. This visualization allows real-time tracking of identified threats and vulnerabilities, being crucial for security management.

Figura 2.7 – AWS Cost Explorer in Grafana.

Screenshot removed from this public version. It showed details of PinPag's production systems.

Source: Prepared by the authors.

Figura 2.8 – Azure Cost Explorer in Grafana.

Screenshot removed from this public version. It showed details of PinPag's production systems.

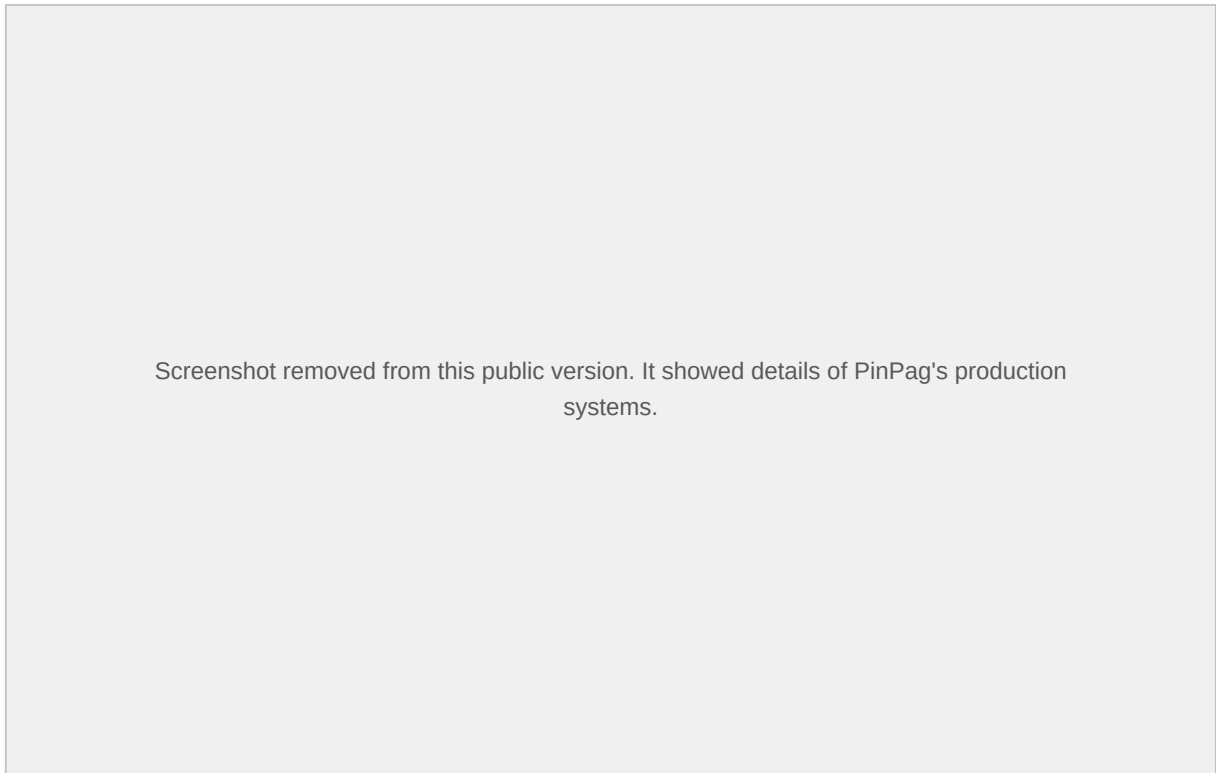
Source: Prepared by the authors.

Figura 2.9 – Security Monitoring with Zabbix in Grafana.

Screenshot removed from this public version. It showed details of PinPag's production systems.

Source: Prepared by the authors.

Figura 2.10 – Security Monitoring with Wazuh in Grafana.



Source: Prepared by the authors.

2.4.2 Objective Achievement

The objective of the First Phase of the Project was achieved, delivering a robust solution for monitoring and control of *multi-cloud* infrastructure. The implementation of tools such as Grafana, AWS Glue, Azure Data Factory, Zabbix, and Wazuh enabled integrated visualization of costs, resource monitoring, and cybersecurity of the environments.

Real-time tracking of KPIs provides efficient management of cloud resources and cost optimization. Security mechanisms provided continuous monitoring of vulnerabilities and threats, meeting PCI-DSS requirements and payment industry standards.

Data centralization in Grafana offers a consolidated view of AWS and Azure infrastructure. This integration brought strategic advantages in resource optimization and identification of excesses, while maintaining the security of sensitive data.

The implemented solutions not only fulfilled the initial objective but also provided a solid foundation for future optimizations of PinPag's cloud infrastructure, ensuring greater resilience and operational efficiency.

3 SECOND PHASE OF THE PROJECT

3.1 ANALYSIS OF PINPAG'S NEEDS AND CHALLENGES

As described in Section 2, the main objective of the First Phase of the project was to provide PinPag with a solid solution for monitoring and controlling its *multi-cloud* infrastructure, focusing on aspects related to costs, security, and compliance. Through the implementation of tools such as Grafana, Zabbix, and Wazuh, in addition to data collection flows in AWS and Azure, it was possible to centralize information and enable real-time tracking of established KPIs.

In the Second Phase of the project, the objective was to expand this solution, making KPIs and metrics more actionable. This allowed PinPag's team to use this data to make more assertive decisions and act directly to optimize results and achieve business objectives. The proposal is to ensure that metrics are not just performance indicators, but strategic tools that guide concrete actions based on collected data.

According to the concept of metrics presented by Eric Ries in the book *Lean Startup* ([RIES, 2011](#)), there are four main types of metrics that should be considered:

1. **Vanity Metrics:** Although visually attractive, these metrics do not provide useful information for decision-making. They may present an exaggerated or grandiose image of performance, but do not offer practical data for in-depth analysis.
2. **Actionable Metrics:** These are those where it is possible to establish a clear cause-and-effect relationship, allowing the team to identify specific actions based on the data presented.
3. **Accessible Metrics:** These metrics are presented in a simple and understandable way, allowing concrete conclusions to be drawn without the use of irrelevant or "vanity" metrics. They are often called "gold metrics" due to their clarity and value.
4. **Auditable Metrics:** These are metrics verified directly with customers, ensuring that the data presented is accurate and confirmed by external sources.

Therefore, the main focus of this phase of the project is to make existing metrics and KPIs more accessible and actionable, enabling effective use of this information when making PinPag's decisions. For this transformation to occur effectively, it is necessary to understand how these decisions are currently made within the organization.

3.1.1 Stakeholders and their Needs

In the context of contemporary business management, the term *stakeholders* refers to all interested parties who can affect or be affected by the development of a project (Exame, 2024). At PinPag, each *stakeholder* has distinct objectives and faces specific challenges, and the success in developing solutions depends directly on aligning them with these needs.

As shown in Table 3.1, *stakeholders* at PinPag were mapped, as well as their objectives, functions, and priorities. It is interpreted that objectives guide their decisions, functions determine how they work and, therefore, how they will use the proposed solutions, and priorities indicate which delivery points they may value most.

Quadro 3.1 – Stakeholders: Objectives, Functions, and Priorities in the project.

Stakeholder	Objectives and Functions	Priorities
Information security team	- Identify and mitigate security risks; - Monitor vulnerabilities and evaluate alerts.	Maintain compliance with regulations.
Infrastructure team	- Ensure infrastructure operation; - Monitor performance and visualize infrastructure.	Access information objectively and in a unified manner.
Management and FinOps team	- Supervise infrastructure; - Manage costs and visualize data for decision-making.	Simplicity of application and quick access to information.

Source: Prepared by the authors.

3.1.2 Main Challenges and Points of Attention Identified

After identifying *stakeholders* and conducting detailed conversations with each of them, it was possible to map several improvement fronts that underpin the main objective of the Second phase of the project. The focus of this phase is to make metrics and KPIs more accessible and actionable, providing more efficient decision-making and improved management of PinPag's infrastructure and security.

The main challenges found were organized into topics that clearly and objectively

summarize the identified points of attention. Each challenge is related to one or more groups of *stakeholders*, since the same issue can impact different groups and require different solutions according to their objectives. Thus, for each group of *stakeholders*, a representative phrase of the identified point of attention was described, followed by a complement that offers additional context.

Below are listed the main challenges and points of attention, organized according to the involved *stakeholders*:

1. System Restoration after Failures

- **Stakeholders:** Infrastructure Team, Information Security Team
- **Phrase:** "It is necessary to restore the functioning of the panel and the systems that feed it."
- **Complement:** Need for infrastructure corrections to ensure continuity of operations.

2. Visualization of Internal Firewall Rules

- **Stakeholders:** Information Security Team
- **Phrase:** "I need to visualize firewall rules between instances."
- **Complement:** Difficulty in correctly mapping internal firewall connections, which affects security.

3. WAF Implementation for Exposed Applications and Visualization of these Logs

- **Stakeholders:** Information Security Team
- **Phrase:** "I need to implement WAF rules on web-exposed applications and visualize traffic data generated from their logs."
- **Complement:** Focus on ensuring protection of web applications against attacks and understanding request behavior in these applications.

4. Vulnerability Monitoring at the Application Layer

- **Stakeholders:** Information Security Team

- **Phrase:** "There is no vulnerability monitoring at the application layer."
- **Complement:** The absence of continuous monitoring increases the risk of failures and security breaches in the code.

5. Real-time Communication about Errors and Failures

- **Stakeholders:** Infrastructure Team, Information Security Team
- **Phrase:** "I want to be notified in real time of errors, failures, and problems in the infrastructure."
- **Complement:** Urgent need for automatic alerts and efficient communication during incidents.

6. Panel Simplification for Consolidated Analysis

- **Stakeholders:** FinOps Team
- **Phrase:** "I need a visualization that aggregates all data sources and simplifies my analysis."
- **Complement:** Need to integrate various data sources into a simplified and easy-to-view panel.

In order of priority, the restoration of data collection and generation systems created in the first phase of the project described in Section 2 stands out. After PinPag's sale to Linx, and Linx's subsequent acquisition by Stone, multiple changes occurred in the functioning of cloud providers, as described in Section 1.1. These changes resulted in the removal of various file storage directories and interruption of services responsible for data processing, which made Grafana and its visualizations inaccessible.

Without resolving this problem, it becomes impossible to proceed with other improvement fronts. Therefore, restoration of the data collection and processing system is the main priority, since project continuity depends directly on this critical stage.

3.2 TECHNICAL, ETHICAL, AND RISK GUIDELINES

With priority established in restoring data collection and generation systems, it is essential that this process, as well as the development and implementation of other solutions proposed in this project, be guided by clear guidelines.

These guidelines include technical standards that ensure quality and compliance with industry standards, in addition to addressing ethical issues related to data protection and user privacy.

Furthermore, it is crucial to assess the risks involved in executing the proposed activities, aiming to mitigate possible operational, security, and compliance impacts.

3.2.1 Certifications and Standards Followed by the Company

By operating in the SPB as a sub-acquirer, PinPag is subject to a specific set of regulations:

- Bacen regulation for payment institutions
- SPB standards
- Guidelines of major card brands (Visa, Mastercard, Amex, etc.)

Thus, PinPag must maintain two essential certifications in information security management, PCI-DSS and ISO 27001. Both are indispensable to ensure compliance with financial data security and protection requirements, ensuring that best practices are followed in the development and operation process.

3.2.1.1 PCI-DSS

PinPag, by processing debit and credit card transactions, is subject to strict security regulation, required by the PCI Security Standards Council (PCI-SSC), established by major credit card companies, such as American Express, Discover, JCB International, MasterCard, and Visa ([PCI Security Standards Council, 2024](#)). All companies that process, store, or transmit payment card information must comply with PCI-DSS ([Cloudflare, 2024](#)).

To ensure protection of payment data, PCI-DSS includes a series of measures, such as:

- Use of encryption to protect sensitive information, such as card data
- Strict access control, with use of strong and non-default passwords
- Continuous monitoring and regular security testing of systems and networks
- Maintenance of *logs* and audit trails

Of the standard's requirements, **maintenance of *logs* and audit trails** was a critical requirement that directly impacted the project, as during the audit to obtain its certification, PinPag needs to demonstrate in detail all interactions with sensitive payment data and infrastructure changes. This mainly affected solutions such as Wazuh, which centralizes *logs*, and the Connection Map, for clear visualization of firewall rules, being discussed in later sections of the report, respectively, Section 4.1.3 and Section 3.3.3.

Non-compliance with PCI-DSS can result in penalties imposed by card companies, and may even lead to suspension of financial transaction processing ([PCI Security Standards Council, LLC, 2024](#)).

3.2.1.2 ISO 27001

ISO 27001 is an international standard that establishes standards for information security management. It defines an Information Security Management System (ISMS), which encompasses the implementation, monitoring, and continuous improvement of security controls, covering people, processes, and technologies ([ISO - International Organization for Standardization, 2022](#)).

This standard is vendor-independent, allowing each organization to implement processes according to its needs and technological characteristics. Adoption of ISO 27001 is directly connected to the objective of complying with PCI-DSS, as it offers the necessary governance to structure information security, manage risks, and protect financial data.

ISO 27001 provides the essential foundation for the company to organize itself efficiently, allowing more specific controls required by regulations such as PCI-DSS to be implemented later. In this way, the standard contributes to PinPag meeting PCI-SSC requirements and maintaining the integrity and security of processed financial data.

3.2.2 Ethical Guidelines Followed by the Project

With technical standards established, it is essential to consider the ethical guidelines and risks that guided this project. Despite being for internal use, tool development involves PinPag, which operates critical infrastructure as an acquirer and sub-acquirer. Any interruption, however brief, can generate financial impacts and damage the company's reputation. Therefore, all project actions must ensure operational integrity and continuity.

Several risks were identified and need to be managed cautiously:

1. **Lack of infrastructure documentation:** The absence of clear documentation compromises predictability in incident resolution, making it difficult to identify and

correct failures.

2. **Absence of versioning:** Lack of architecture versioning harms resilience, making it difficult to monitor and reverse critical changes.
3. **Data sensitivity:** PinPag deals with highly confidential information, such as security keys and customer data, which must be rigorously protected to prevent leaks.
4. **Production environment:** The infrastructure operates entirely in production, which means any modification can interrupt essential services and generate significant losses.

To mitigate leak and unauthorized access risks, best practices recommended by the market were implemented, as described in the book *Cloud Computing* (ERL et al., 2013):

1. **Key system:** Use of public and private key pairs for service authentication in providers.
2. **Identity and Access Management (IAM):** Granular assignment of permissions, limiting access to information and service modification. A specific profile was created for project members (Amazon Web Services, 2024c).

Authentication in *Grafana* was managed by *Client ID* and *Client Secret*, ensuring controlled access and minimizing risks. Use of *IAM* is maintained as a central security strategy, with access policies based on the principle of least privilege, mitigating risks and protecting infrastructure.

Finally, data displayed in panels and monitoring systems must be accurate, ensuring strategic decisions are well-founded. Information quality is vital to avoid adverse consequences for both PinPag and its customers.

Ethical guidelines must be in compliance with industry practices, such as data protection, operational security, and transparency. In addition, compliance with standards such as *PCI-DSS* and *ISO 27001* reinforces ethical commitment, ensuring safe and responsible operations.

3.3 PROPOSED SOLUTIONS

Based on the propositions presented in Section 3.1, PinPag's objectives and challenges directed the Second Phase of the project to restore infrastructure and data flows

in cloud service providers. In addition, the aim was to deepen metrics, visualizations, and available data, making them more actionable and accessible. To achieve these objectives, the following solutions were developed together with PinPag:

3.3.1 Alert System

The Alert System monitors and notifies real-time occurrences in PinPag's infrastructure through multiple communication channels (Twilio, Telegram, and SMTP). The system sends automatic notifications to the responsible team when detecting anomalies or problems, allowing incidents to be treated quickly, and their impacts minimized.

Integration of these communication channels offers flexibility in notifying the IT team. This makes monitoring metrics more accessible by providing important and clear information in real time, which facilitates interventions and keeps PinPag's environment proactive to incidents.

3.3.2 Web Application Firewall (WAF)

The *Web Application Firewall* (WAF) is a security tool positioned between the company's web applications and the external network or global internet. Its main objective is to monitor and control HTTPS traffic, analyzing all requests sent to applications and servers. Through predefined rules, the WAF seeks to identify patterns of malicious connections, protecting applications against cyberattacks. In addition to ensuring the security of sensitive data that travels through applications, the WAF allows generation of graphs and access statistics, which provide a detailed view of connection behavior and web service health.

WAF implementation in PinPag's infrastructure not only reinforces protection of web applications against vulnerabilities and threats, but also improves visibility of traffic-related activities, facilitating analysis of security metrics. By integrating WAF with tools such as Grafana, it is possible to create customized panels that display, in real time, attack attempts, traffic patterns, and other crucial information for security management. In this way, the IT team can act proactively to mitigate risks and optimize web service performance, in addition to ensuring compliance with regulatory standards, such as PCI-DSS and ISO 27001. The ability to generate detailed reports on invasion attempts and traffic behavior also contributes to continuous improvement of security policies, making metrics more accessible and actionable.

3.3.3 Connection Map

As described in Subsection 3.2.1, PinPag follows strict technical security and compliance standards. One of them is compliance with *firewall* rules and strict control of access permissions between resources and services. These rules are audited by third parties, and their access must be easy and direct.

The company established that, initially, rules should only be considered inadequate when infrastructure resources are accessible on ports that are not intended for web traffic. Thus, any open communication port that is not 80 or 443 should be highlighted for analysis.

Previously, verification of access rules and analysis of relationships between resources depended on manual processes, increasing the risk of errors and reducing operational efficiency.

Currently, PinPag uses mainly AWS to host the most critical services to its business. Although it has used Azure as a cloud provider, the current plan is to progressively migrate critical operations to AWS, consolidating infrastructure in a single environment more aligned with its security and compliance needs.

In AWS, PinPag's infrastructure is composed of several components, including *Security Groups*, EC2 instances, RDS instances, VPCs, and subnets. Below, we describe each of these components and their functions:

1. **Security Groups:** Function as virtual firewalls that control inbound and outbound traffic of AWS services, protecting them from unauthorized access. They allow definition of specific rules that restrict or authorize traffic based on IPs, ports, and protocols, providing a fundamental layer of security for the infrastructure. ([Amazon Web Services, 2024e](#))
2. **EC2 Instances (Elastic Compute Cloud):** Virtual servers that offer scalable computational capacity for data processing and application execution. They can be configured and adjusted to meet different types of workloads, from small applications to complex systems. ([Amazon Web Services, 2024a](#))
3. **RDS Instances (Relational Database Service):** Managed database service that supports multiple engines, such as MySQL, PostgreSQL, and Oracle, ensuring high availability and data security. Simplifies administration of relational databases used to store critical application data. ([Amazon Web Services, 2024b](#))
4. **Virtual Private Cloud (VPC):** Isolated virtual network where AWS resources are deployed and protected. Allows configuration of secure and controlled environments,

enabling complete management of IP addresses, subnets, and route tables to ensure isolation of each application. ([Amazon Web Services, 2024e](#))

5. **Subnets:** Subdivisions within a VPC that allow organizing resources into smaller networks and configuring access according to security needs. They can be classified as public or private, controlling access of the resources they host.

Managing this infrastructure presents significant challenges in complexity and time. [sentence removed]

With this number of components, there is no intuitive way to visualize which machines have permission to access which services, whether internal or external, or to quickly identify permissions assigned to each *security group*. This makes it difficult to detect vulnerabilities in the network, such as services with very broad permissions, where there is difficult control of machines that should or should not access them.

For this, graph modeling was adopted, providing a clear and efficient visualization of network connections, allowing rapid detection of inadequate security rules.

Graphs are essential structures in computer science and mathematics, composed of vertices (nodes) and edges that represent respectively entities and their connections. Used to model networks such as computer and social networks, graphs facilitate analysis of interactions between elements ([DIESTEL, 2005](#)). In the context of this project, each infrastructure resource is a node and each connection between them is an edge. Thus, the solution was structured as follows:

1. EC2 and RDS instances connect to their private IPs and associated security groups;
2. Security groups connect to EC2 instances, RDS, and port ranges, defined in their rule in the cloud provider;
3. Port ranges connect to security groups and IP ranges, also known as CIDR blocks (Classless Inter-Domain Routing), which define specific ranges of IP addresses in a subnet;
4. IP ranges, which encompass from individual IPs to subnets and VPCs, connect to instances and port ranges.

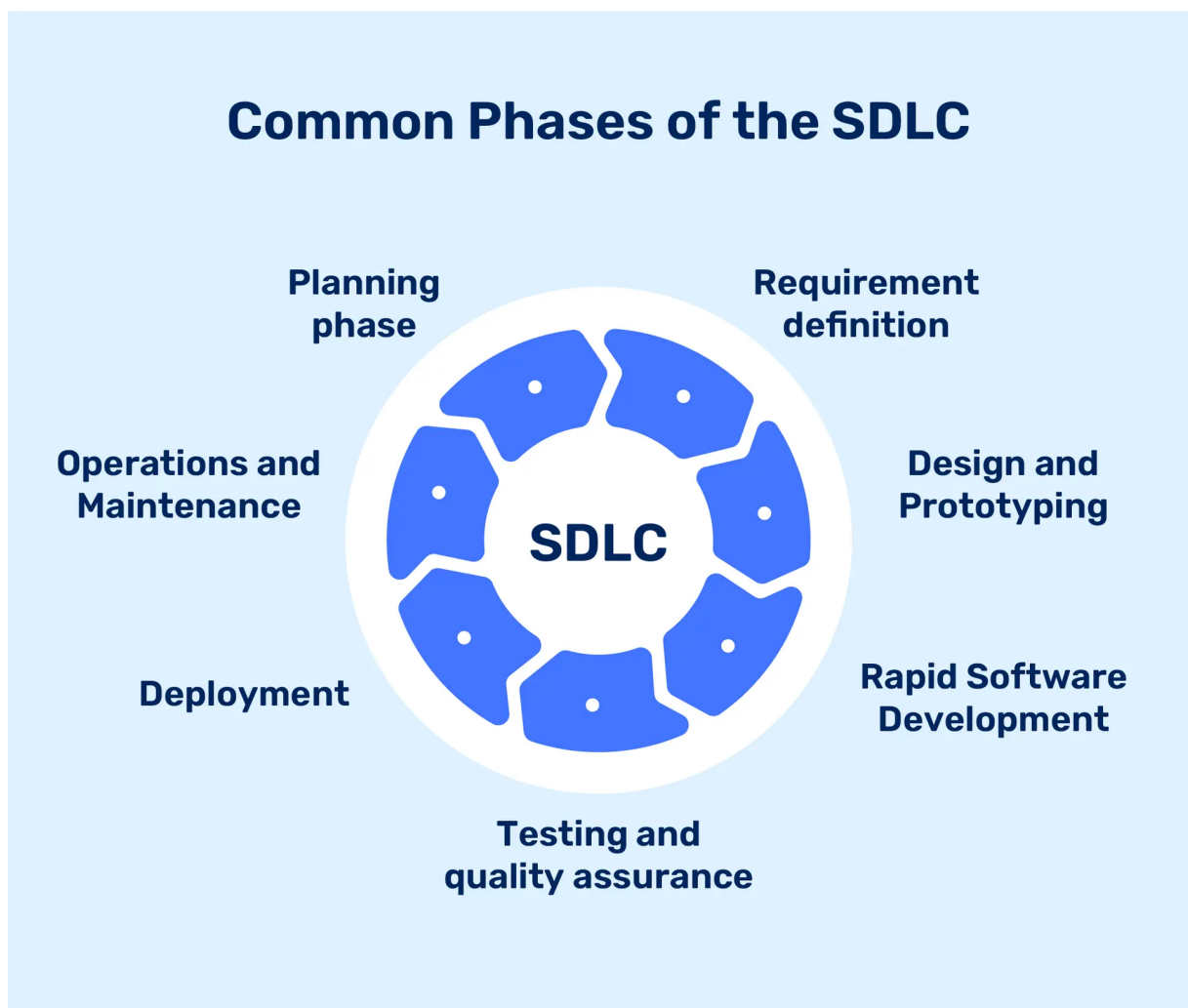
In this way, the Connection Map shows the connections of each service, detailing associated IPs and ports, and signals suspicious connections, according to criteria determined by the company, facilitating vulnerability correction.

This mitigates risks of incorrectly configured *firewall* rules and facilitates the compliance audit process, centralizing information and providing a broad view of the infrastructure.

3.3.4 Application Security

Application security involves a set of practices aimed at protecting software against vulnerabilities that may be exploited throughout its lifecycle. From the development phase to deployment, security is incorporated through analysis and verification techniques that seek to identify potential flaws and risks, mitigating them before they can be exploited (IBM, 2024). These practices are commonly integrated into the software development lifecycle, also known as *Software Development Lifecycle (SDLC)*, illustrated in Figure 3.1. It includes stages such as planning, design, development, testing, deployment, and maintenance, all oriented to ensure the quality and security of the final product.

Figura 3.1 – Application development lifecycle.



Source: [Agilie Team \(2024\)](#).

The SDLC was designed to improve development efficiency and quality, offering a structured process that helps minimize risks associated with software development. In addition to ensuring that stakeholder requirements are met, the SDLC facilitates collaboration between interested parties, promoting a clear understanding of expectations from the beginning of the project. This alignment is fundamental for the success of complex software projects, ensuring that all development phases have clear guidelines and standards to reduce failures ([Amazon Web Services, 2024f](#)).

3.3.4.1 Integration with *Continuous Integration* and *Continuous Delivery* (CI/CD) Practices

In addition to SDLC, integration of *Continuous Integration (CI)* and *Continuous Delivery (CD)* practices has become increasingly common as a way to accelerate the development cycle and improve software quality. These practices enable automation of code integration processes, executing unit tests, verifying compliance with standards, and integrating code changes automatically. This allows rapid error detection, which can be corrected continuously, keeping the software in a state ready for delivery at any time ([Red Hat, 2024](#)).

For development of its main solutions, PinPag uses the GitLab platform, an open-source tool widely adopted for code versioning and management. GitLab not only offers version control, but also incorporates a *CI/CD* system, which allows the team to develop in an agile and structured manner ([Deimos, 2024](#)). However, an important gap was identified in the current process: there was no security solution implemented to monitor vulnerabilities or verify code security throughout the development cycle.

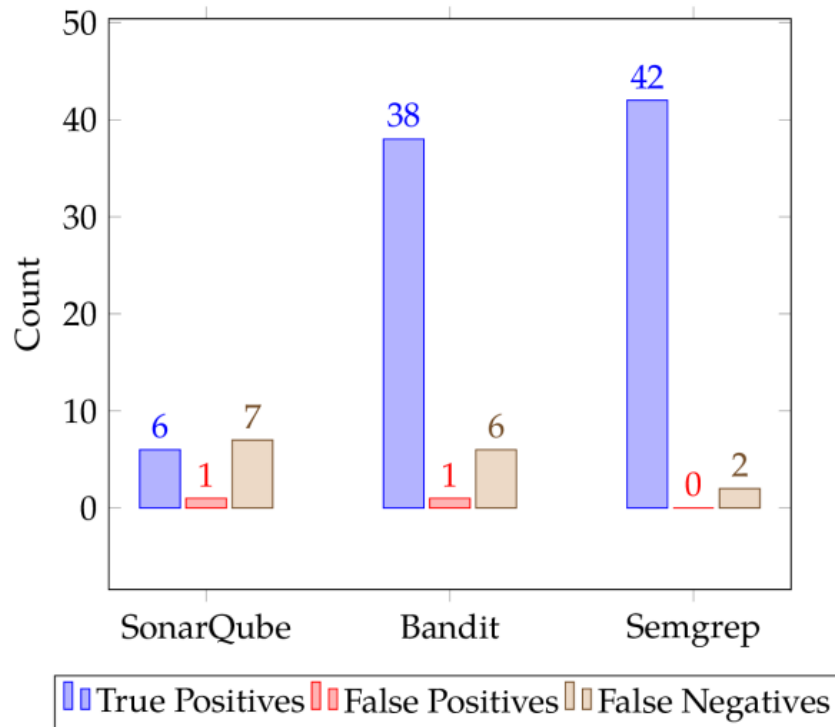
3.3.4.2 Market Analysis and SemGrep Selection

When evaluating the market for application security solutions, several security tools for code analysis were considered, among which Checkmarx and Veracode stand out, solutions widely used in the corporate sector for *Static Application Security Testing* (SAST), *Software Composition Analysis* (SCA), and *Dynamic Application Security Testing* (DAST), being effective for large organizations seeking complete and integrated protection for their systems. However, collaborators reported that costs are high - reaching about **\$500,000** for their infrastructure - making their adoption at PinPag unfeasible.

Thus, research was conducted to find a similar tool, but open-source. Available *open source* SAST applications were compared. As can be seen in figures 3.2 and 3.3, *SemGrep* was the SAST application with the best performance in Python and Javascript, which meets the client's needs ([SMYćKA, 2024](#)).

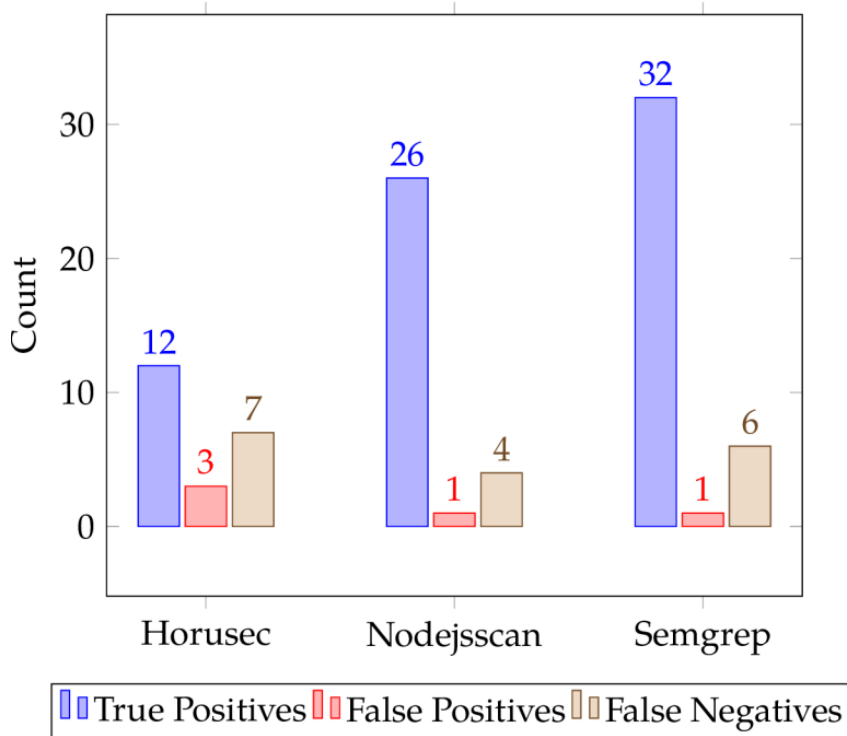
Given this scenario, Semgrep was adopted as a solution. In addition to being free, the tool is known for its flexibility, allowing development teams to customize their security rules according to the specific needs of the project. An example of this flexibility is integration with GitLab, which facilitates its inclusion in the existing CI/CD pipeline, with continuous security verification.

Figura 3.2 – Performance comparison for Python code.



Source: [Smyčka \(2024\)](#).

Figura 3.3 – Performance comparison for Javascript code.



Source: [Smyčka \(2024\)](#).

3.3.4.3 SemGrep Functionalities in PinPag's Context

SemGrep offers several security functionalities that cover different aspects of code analysis. For the *Atendedor* project, SAST was implemented, which allows identifying vulnerabilities directly in the source code. Below are some of SemGrep's approaches:

- **Software Composition Analysis (SCA):** Performs verification of libraries and external dependencies to identify known vulnerabilities. This is crucial for monitoring third-party components in code, ensuring that outdated or security-flawed versions are quickly identified and updated ([Black Duck by Synopsys, 2024](#)).
- **Dynamic Application Security Testing (DAST):** Allows analysis of vulnerabilities at runtime, which helps detect flaws that may arise when the application is in operation. Although SemGrep is not specifically a DAST tool, it can complement this analysis together with other tools ([OpenText, 2024a](#)).
- **Static Application Security Testing (SAST):** Focused on static analysis, SAST identifies vulnerabilities directly in the source code, including problems such as *SQL Injection* and *Cross-Site Scripting*. This analysis is performed without the need for code execution, allowing problems to be corrected before the testing or deployment phase ([OpenText, 2024b](#)).

SemGrep implementation in PinPag's CI/CD pipeline allows vulnerabilities to be detected continuously and proactively, contributing to greater agility in response and correction of security flaws. This practice not only strengthens security of the *Atendedor* project, but also helps PinPag meet compliance requirements with security standards such as PCI-DSS and ISO 27001, by ensuring that vulnerabilities are corrected before software is made available in production.

3.3.4.4 Security Monitoring Panel with Grafana

To make monitoring more visual and accessible, a panel in Grafana was integrated with SemGrep, offering a consolidated view of application security metrics, which includes the following KPIs:

1. **Files with most problems:** Identifies files with the highest number of vulnerabilities, allowing the team to focus on the most critical components with the greatest impact on the system.
2. **Most common vulnerabilities:** Shows which vulnerabilities occur most frequently, helping in detection of recurring security patterns and definition of preventive actions.
3. **Total Vulnerabilities Found:** Presents the total number of vulnerabilities identified over time, offering a clear view of the project's security panorama and allowing measurement of the effectiveness of adopted measures.
4. **Severity Distribution:** Classifies vulnerabilities by severity level (low, medium, high, critical), which helps the team prioritize corrections based on the severity of detected problems.
5. **SAST Vulnerabilities:** Focuses on specific vulnerabilities detected by SAST, providing a breakdown of security flaws related to static code analysis.

In addition to KPIs in Grafana, the alert system described in Section 4.6 was integrated with SemGrep to notify the IT team immediately about detected critical vulnerabilities. This integration reinforces the team's rapid response capacity, ensuring that any critical vulnerability is treated urgently.

3.3.4.5 Conclusion on Security Implementation with SemGrep

Adoption of SemGrep for application security at PinPag represents a strategic choice, combining cost efficiency and flexibility. Although tools such as Checkmarx and Veracode offer cutting-edge solutions in the market, the high cost of these platforms is not

sustainable for the company at the moment. SemGrep, being an accessible and open-source solution, provides an effective alternative that meets PinPag's security requirements, while offering the flexibility necessary for future adaptations.

With this implementation, PinPag now has a robust security solution for its main project, "Atendedor", and acquires the ability to expand use of SemGrep to other projects as needed. In addition to ensuring compliance with security best practices, use of a monitoring panel in Grafana allows visual and accessible tracking of security status, strengthening the culture of proactive security within the IT team.

3.3.5 Infrastructure Panel

The main client of this panel is the information security team, which seeks greater accessibility to data by simplifying and unifying visualization of main metrics in a single interface. This visualization consolidates multiple data sources, feeding critical KPIs for monitoring and strategic decision-making in real time.

For building this panel, the following KPIs were used:

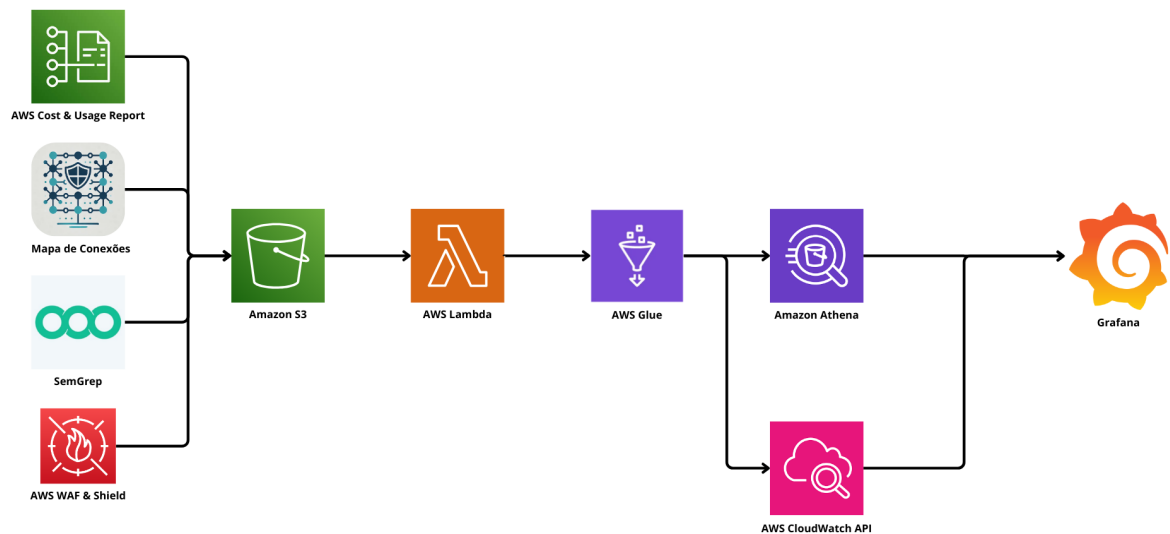
1. **Total Cloud Cost (Current Month vs Previous):** This metric, already existing in the previous phase, aims to compare total cloud costs between the current month and the previous month. It allows the security team to track cost trends and identify possible increases or reductions in resource usage.
2. **Resource Utilization (CPU and Memory) of Critical Instances:** Metric that monitors CPU and memory usage in the most critical instances of the infrastructure. Real-time monitoring allows optimizing performance and allocating resources efficiently, ensuring that critical instances operate stably.
3. **Percentage of Cost Change between Months:** This KPI, also deepened from the previous phase, measures the percentage variation of costs between months, helping the team quickly identify spending spikes and apply optimization strategies.
4. **Current Week Costs vs Previous Week:** A new metric implemented specifically for this infrastructure visualization. It compares current weekly costs with those of the previous week, providing a more granular and real-time analysis of cloud spending. This allows faster identification of anomalous cost increases, helping in weekly financial management and control.
5. **Costs in Business Hours vs Non-Business Hours:** Metric that compares the cost of resource usage during business hours with usage in non-business hours. This

KPI offers insights about resource consumption outside operating hours, allowing optimizations regarding resource usage in low-activity periods.

3.3.6 Solution Integration

All proposed solutions were integrated into existing AWS infrastructure. As illustrated in Figure 3.4, *AWS Cost & Usage Report*, *Connection Map*, *SemGrep*, and *AWS WAF & Shield* services store their data in *Amazon S3*. As needed, *Lambda* functions were configured to process stored data and activate *crawlers* in *AWS Glue* to catalog them. Processed data is made available in *Amazon Athena*, from where it is accessed by *Grafana*, the main interface for information visualization. This centralization of information in *Grafana* allows *PinPag* to have a unified and consolidated view of security, performance, costs, and connectivity, and facilitates monitoring of critical metrics in real time. In this way, the proposed integration optimizes analysis capacity, improves operational efficiency, and supports more agile and informed decision-making.

Figura 3.4 – Architecture Diagram of Proposed Solutions.



Source: Prepared by the authors.

3.3.7 Expected Benefits

From these changes, *PinPag* is expected to obtain several benefits, including:

- **Greater Visibility and Control:** Centralization of metrics and KPIs allows more efficient and detailed monitoring of *multi-cloud* infrastructure.

- **Security Improvement:** Tools such as WAF, SIEM, and SemGrep reinforce application and sensitive data security, reducing vulnerability and attack risks.
- **Cost Optimization:** Detailed monitoring of resources and costs enables identification of optimization opportunities and expense reduction.
- **Informed Decision Making:** Through the Infrastructure panel that provides focal operational information, facilitating strategic decisions based on concrete data.
- **Rapid Incident Response:** Real-time alert systems ensure that the team can react promptly to any problems or threats, minimizing operational impacts.
- **Compliance and Governance:** Implemented solutions ensure compliance with standards such as PCI-DSS and ISO 27001, reinforcing PinPag’s governance and responsibility.

3.3.8 Related Work

In order to deepen the state of the art cited in Section 1.3, related work to the project was researched, mainly about creating monitoring systems, with data generation, collection, and processing flows.

Several studies and comparisons have explored solutions for infrastructure and security monitoring in cloud computing environments. These works normally focus on fundamental areas for companies operating in *multi-cloud* environments, such as resource optimization, cost reduction, and compliance assurance. Among the main challenges highlighted in the literature are the need for tools that offer high flexibility, robustness in handling large volumes of data, and customization capability for different infrastructures.

Flexibility, in this context, refers to the tool’s ability to adapt to different operational needs and IT environments. This means that a flexible solution allows adjustments in monitored metrics, integration with various platforms, and operation. In an environment like PinPag’s, which uses multiple cloud service providers and varied operating systems, a solution with high flexibility is essential to ensure that the tool can be configured according to specific business demands, without limitations imposed by rigid or standardized solutions.

Robustness refers to the tool’s ability to handle large volumes of data without compromising performance or accuracy of collected information. In monitoring systems, robustness is a crucial attribute to ensure that data is processed efficiently, even when there are load spikes or significant variations in the volume of monitored data. This is especially important in *multi-cloud* infrastructures, such as PinPag’s, where different parts of the operation can generate large amounts of data in real time.

Finally, customization capability refers to the possibility of configuring the tool to meet specific monitoring and data management needs, adapting to each company's unique context. This includes creating customized metrics, modifying panels, and integrating different data sources. Highly customizable tools allow PinPag's IT team greater control over how information is presented and monitored, which facilitates strategic decision-making and continuous infrastructure adjustment as business requirements change.

In terms of infrastructure monitoring, literature demonstrates that open-source solutions, solutions to which any developer can contribute - are widely used due to their customization capability and the fact that they allow more granular control of metrics.

A study by Verinadis presented a comparison between the top 20 monitoring solutions, evaluating criteria such as the number of supported metrics, ease of integration, and compatibility with different operating systems ([VERGINADIS, 2023](#)). As illustrated in Figure 3.5, from this analysis, solutions such as Zabbix and its substitute Prometheus stood out for their robust functionalities and large configuration capacity. Considering that Zabbix is already implemented at PinPag and meets the company's needs, it was maintained as the main monitoring solution.

Regarding security monitoring, a critical aspect in financial environments, several open-source solutions for SIEM (Security Information and Event Management) were compared in recent studies. Tools such as Wazuh, OSSIM, and Elastic Security were evaluated for their threat detection capability, event correlation, and compliance with security standards, such as PCI-DSS. In an analysis conducted by Vazão and Bezas ([VAZÃO et al., 2019](#)) ([BEZAS; FILIPPIDOU, 2023](#)), Wazuh was identified as an efficient solution, offering advanced SIEM and XDR (Extended Detection and Response) functionalities, in addition to being widely used to ensure compliance in security audits.

From the comparison between Wazuh and other solutions such as OSSIM, SIE-Monster, and Elastic Security, Manzoor's study concluded that Wazuh stands out for its performance in different operational scenarios and for its flexibility in terms of compatibility with Windows and Linux operating systems ([MANZOOR et al., 2024](#)). It proved superior in aspects such as event correlation, vulnerability scanner, and compliance, being an appropriate choice to meet PinPag's security requirements and ensure compliance with financial sector standards. Thus, the choice of Wazuh for continuous monitoring of PinPag's infrastructure security is appropriate and recommended, ensuring protection against threats and compliance with security standards required in the sector.

Figura 3.5 – Table with monitoring solutions evaluated in Verinadis’s article.

Monitoring Tool	Monitoring Level	Processing	Output Types	Distribution	Number of metrics	Configuration	Custom Metrics	Foot-print	Provider Dependent
Amazon CloudWatch	host/database/application (- functions)	basic/alerts	push-based/ persisted (DB)	centralized	medium	medium	yes	medium	yes
Apache Chukwa	big data framework	basic	pull-based/ persisted (file)	centralized	low	low	yes	medium	no
Azure Cloud Monitoring	host/application/ database	basic/alerts	pull-based/ persisted (file/DB)	centralized	high	medium	yes	medium	yes
Cacti	network/partially host	alerts	pull-based/persisted (DB)	statically distributed	low	medium	yes	low/ medium	no
Collectd	host/network/ext ensible to big data frameworks	no	push-based/ persisted (file)	centralized	medium	low	yes	low	no
Datadog	host/network/app lication/ database	advanced/root cause analysis	push-based/persisted (DB)/time-series	centralized	high	high	yes	medium	no
Dynatrace	host/application/ network/ database	alerts/root cause analysis/ forecasting	pull-based/ persisted(file/DB)/ time-series	distributed	high	high	yes	medium	no
Google Cloud Monitoring	host/application/ network/ database	basic/alerts	push-based/ persisted (file/DB)	centralized	high	medium	yes	medium	yes
Hadoop Perf. Monitoring	big data framework	basic	pull-based/ persisted (DB)	statically distributed	low	low	yes	medium	no
Icinga	host/network	alerts	push-based/ persisted (DB)	dynamically distributed	high	medium	yes	medium	no
Logic Monitor	host/application/ network/ database	alerts/ forecasting	pull-based/ persisted (DB)	centralized	high	high	yes	medium	no
Munin	host/network	alerts	pull-based/ persisted (file)	centralized	high	medium	yes	low	no
Nagios	host/network	basic/alerts	pull-based/ persisted (DB)	centralized/ statically distributed	high	medium	no	medium	no
Netdata	host/network/dat abase/ extensible to application, big data frameworks	basic/define thresholds/ alerts	pull-based/ persisted(file/DB)/ time-series	centralized	high	high	yes	low	no
New Relic	host/network/app lication	advanced/root cause analysis	pull-based/ persisted(DB)/ time-series	centralized	high	high	yes	medium	no
Openstack Telemetry	host/network	alerts	push-based/ persisted (DB)	centralized	medium	low	yes	medium	yes
Opsview	host/application/ network/ database	alerts	push-based/persisted (file/DB)/time-series	statically distributed	high	high	yes	medium	no
Prometheus	host/network/app lication	alerts	pull-based/push-base/persisted (DB)/time-series	centralized/ statically distributed	high	high	yes	low	no
SequenceIQ	big data framework	basic	push-based/ persisted (DB)	dynamically distributed	medium	low	yes	medium	no
Shinken	network/partially host	basic	pull-based/ persisted (file/DB)	statically distributed	medium	medium	yes	medium	no
Zabbix	host/network	basic/alerts	pull-based/ persisted (DB)	statically distributed	medium	medium	no	low	no

Source: Verginadis (2023).

3.4 METHODOLOGY

To achieve the objectives established in this stage of the project and maximize the quality of deliverables, a methodology was adopted that aims to cause the greatest positive impact on PinPag's *stakeholders*. The main focus is to develop solutions that not only meet the challenges faced by the company, but also solve them effectively. The chosen approach aims to ensure efficient coordination among all parties involved in the project, promoting smooth integration between development, communication, and management phases.

This section addresses the methodologies applied at different moments of the project, detailing how each one directly impacts development, communication, and general management. The choice of these methodologies was guided by the need to create an organized process, with high adaptability and that ensures delivery of quality results, while ensuring collaboration between teams and compliance with established deadlines.

3.4.1 Inspirations

3.4.1.1 Agile Manifesto

To maximize quality and efficiency in project management, the development process was inspired by the Agile Manifesto ([Agile Alliance, 2001](#)), which of the 12 general principles, prioritized those most aligned with the challenges faced by the team and *stakeholders*. These principles are:

- **Focus on customer satisfaction:** Continuous contact with *stakeholders* was maintained to ensure that solutions met their expectations and needs. With this, deliveries were adjusted according to received *feedback*, ensuring project alignment with PinPag's real demands.
- **"Changes in requirements are welcome, even late in development":** Requirement changes were accepted even in advanced development phases, as the project should not be limited by a rigid set of predefined requirements. Thus, the scope of this project evolved beyond the improvements proposed at the end of the previous one, meeting needs presented during its development.
- **"Build projects around motivated individuals, giving them necessary environment and support":** The project was organized by assigning specific responsibilities to each member, maintaining the global vision. Although each had their work front, mutual support and collaboration were valued, aiming for group success.

- **"At regular intervals, the team reflects on how to become more effective and refines its behavior"**: This principle was combined, in a structured way, with the *Scrum framework* (SCHWABER; SUTHERLAND, 2020), conducting weekly *sprints*, development periods, with all team members. In meetings, completed deliverables, their quality, and planning of next steps were discussed. To promote diversity of ideas, the role of *Scrum Master* was rotated weekly, being responsible for maintaining *Scrum* values in the team, removing impediments, and concentrating focus on defined goals.
- **"Working software is the primary measure of progress"**: Focus was always on delivering functional parts of solutions, with emphasis on components that could be evaluated during the development cycle, which is described in Section 3.4.1.2. Thus, delivery integrity was ensured, anchored by continuous validation with *stakeholders*.

3.4.1.2 Complete Bug Solution Cycle

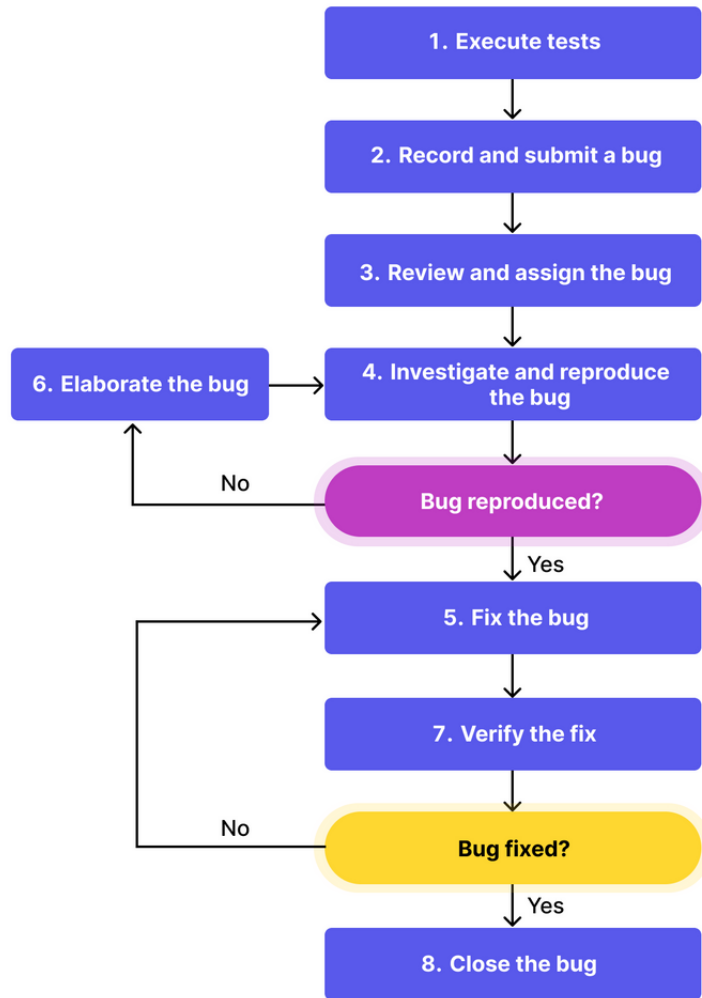
The approach to solving *bugs* was inspired by the complete management cycle described by (Katalon, 2023), as illustrated in Figure 3.6. In this context, *bugs* are all unwanted behaviors that occur in a digital (*software*) or physical (*hardware*) system and that do not meet expected results.

Due to the specific nature of problems faced in this project, mainly related to restoring data flows from cloud providers, a simplified cycle was developed. The challenge involved a high degree of uncertainty regarding the complexity of corrections and the time needed for their completion, requiring an agile, yet methodical approach. It was essential to individually test each system module to ensure they were returning satisfactory results, according to their responsibilities defined in Section 2.

The process used, *detect-debug-resolve*, began with systematic exploration of subsystems through *troubleshooting*, thoroughly checking data flows, integrations between services, and dependencies between systems to identify root causes of data ingestion and monitoring failures.

After this initial analysis, an iterative cycle was implemented that covers the stages of detection, error understanding, and resolution. For each subsystem, *logs* were reviewed, network and monitoring configurations validated, and anomalies in data flow diagnosed, ensuring rigorous control over identified problems.

Figura 3.6 – Complete *Bug* Solution Cycle.



Source: [Katalon \(2023\)](#).

3.4.2 Communication

Effective communication between team members, the supervising professor, and the client was a fundamental element for project success. To ensure agility in information exchange and question resolution, different communication tools were used, according to the specific needs of each interaction.

For quick and informal communications between group members and the supervising professor, the Signal tool was adopted, while WhatsApp was used for direct and quick communication with the client. Both tools allowed agile exchange of messages and files, facilitating real-time decision-making and resolution of emergency issues during development.

In addition, for more structured meetings, such as daily group meetings (*daily*s),

weekly *sprints*, and periodic meetings with the client, the Google Meet platform was used. This tool provided a stable environment for videoconferences, allowing alignment between team members, delivery planning, and discussion of *feedback* with the client. Synchronous meetings ensured that all involved were aware of project progress and that any adjustments or changes were discussed quickly and collaboratively.

These communication practices ensured that the flow of information was continuous and efficient, minimizing noise and impediments throughout project development.

4 IMPLEMENTATION OF PROPOSED SOLUTIONS

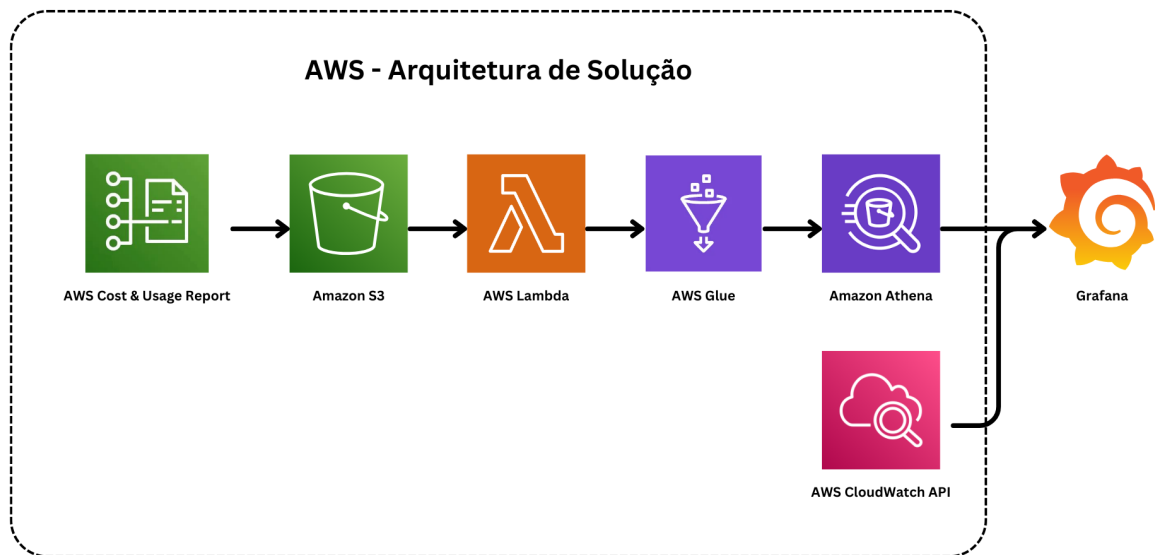
In the development of the 2nd stage of the project with PinPag, the first priority is to ensure the recovery of data collection and processing flows from AWS and Azure providers.

4.1 REESTABLISHMENT OF MONITORING INFRASTRUCTURE

4.1.1 AWS Infrastructure Correction

This section describes the correction of AWS infrastructure, adopting a *Top-Down* approach, which begins the analysis at the highest levels of the system and progresses toward more specific and detailed levels. The data collection, processing, and visualization flow was examined sequentially, starting from *Grafana* and passing through intermediate services such as Amazon Athena, AWS Glue, and Lambda, until identifying the root of the problem in cost reports, as demonstrated in Figure 4.1.

Figura 4.1 – AWS cloud solution architecture.



Source: Prepared by the authors.

Below are the identified problems and respective implemented solutions, presented in an organized and numbered manner.

4.1.1.1 Identified Problems

1. Error in Grafana Dashboard:

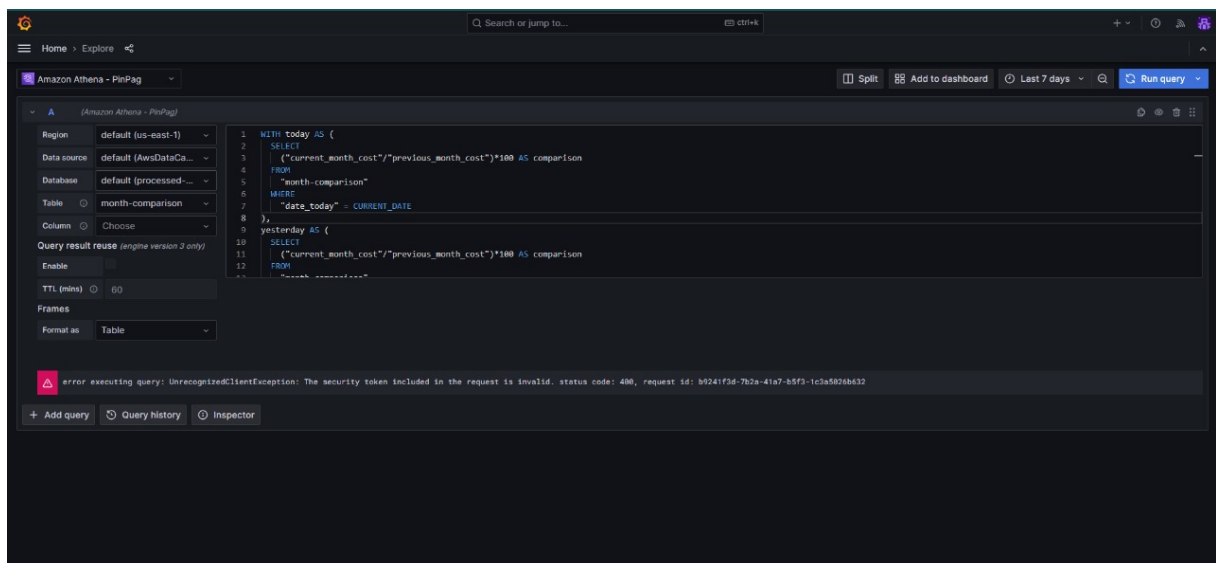
During the initial analysis in the AWS dashboard in Grafana, it was observed that no data was being loaded correctly, and multiple errors were displayed. These errors ranged from lack of data to authentication and connection problems with Amazon Athena. Initially, it was considered that these problems could be caused by incorrect SQL requests, such as errors in column names or syntax.

To validate this hypothesis, the SQL requests were reviewed and the columns queried were verified in comparison with the data tables available in Amazon Athena. After this verification, it was possible to confirm that the requests were correct and that the requested columns corresponded to those existing in Athena.

With this validation, the next step was to investigate possible problems related to user permissions. The focus shifted to the IAM profile used to manage Grafana's access to AWS. Although the permissions were reviewed and corrected, the error persisted, which indicated the possibility of a problem related to authentication.

After verifying that the permissions were correct, it was identified that the access keys associated with the *IAM* profile had expired. However, even after updating the keys, the error continued to occur, suggesting that the problem could be at another level. Finally, it was concluded that the Grafana *cache* still stored the old information from the expired keys.

Figure 4.2 – Credential invalidation error in Grafana.



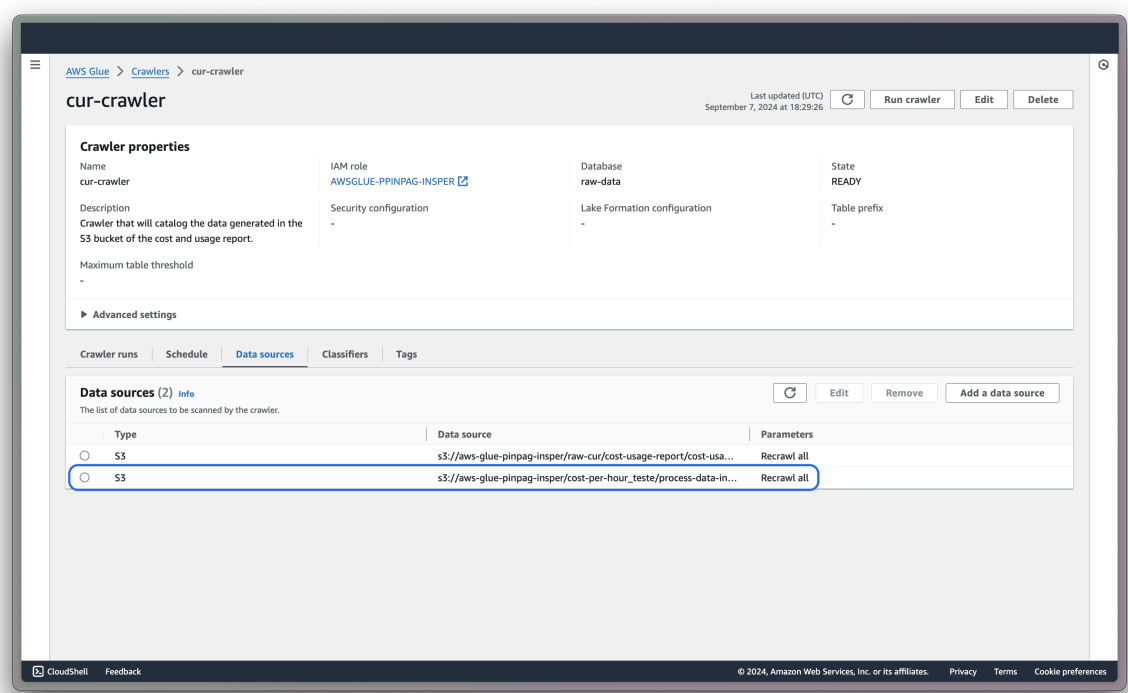
Source: Prepared by the authors.

2. AWS Glue *Crawler* Configuration Failure:

A significant failure was identified in the configuration of the AWS Glue *crawler*, responsible for processing data stored in S3. The directory configured for data search, **aws-glue-pinpag-insper/raw-cur/cost-usage-report/cost-usage-report/**, did not exist. This incorrect configuration resulted in the *crawler* being unable to locate and process the data adequately.

When investigating the cause of the problem, it was verified that the specified directory had been removed or was incorrect, which interrupted the processing flow. Since the *crawler* is responsible for collecting and preparing data for the next transformation stages, the absence of the correct directory prevented the data from being made available for the subsequent flow.

Figure 4.3 – Configuration of the *crawler* cur-raw with corrected directory.



Source: Prepared by the authors.

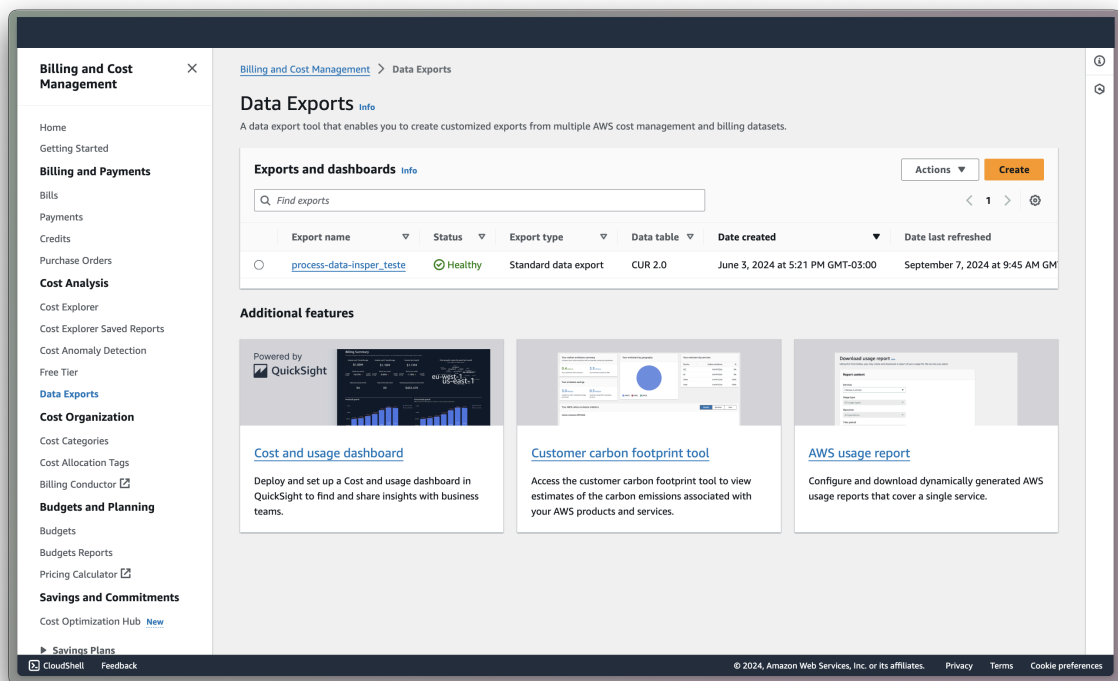
3. Inadequate Configuration of Cost Reports:

An incorrect configuration was identified in the *AWS Cost & Usage Report*, which generates infrastructure usage and cost reports. During the analysis, it was verified that only one report was active, and it was named in a way that suggested it was a test

environment. This inadequate configuration was causing the interruption of data flow to the S3 *bucket* designated to store the reports.

Upon examining more closely, it was found that the current configuration did not meet the requirements for a production environment, being configured to generate data for another repository that was not being used by the *crawler*.

Figura 4.4 – Configuration of cost report generation and export.



Source: Prepared by the authors.

4. Removal of Processing *Job* in AWS Glue:

Finally, the process executed in AWS Glue, responsible for transforming data to feed the dashboard, had been removed, resulting in the interruption of data flow and the unavailability of information in Grafana.

4.1.1.2 Implemented Solutions

1. Solution for *Cache* Problem in Grafana:

The error in Grafana was resolved after clearing the *cache*, followed by recreating the data source and reinserting the connection credentials. These actions allowed the connection between Grafana and Amazon Athena to be reestablished, ensuring that SQL

requests were executed correctly and the dashboard returned to displaying the processed data.

2. Correction of Directory in AWS Glue *Crawler*:

With the identification of the non-existence of the directory configured for the *crawler* in AWS Glue, it was necessary to reconfigure the search path to the correct location in S3. The new configuration was validated, and the *crawler* began to locate and process data correctly, reestablishing the data flow between S3 and the next stages of the flow.

3. Adjustment in Cost Report Generation:

After identifying that the cost report was configured for a test environment and storing data in an incorrect repository, the *AWS Cost & Usage Report* was reconfigured. The report was adjusted to send data to the correct directory in S3, allowing the *crawler* to process it and the cost flow to return to normal operation.

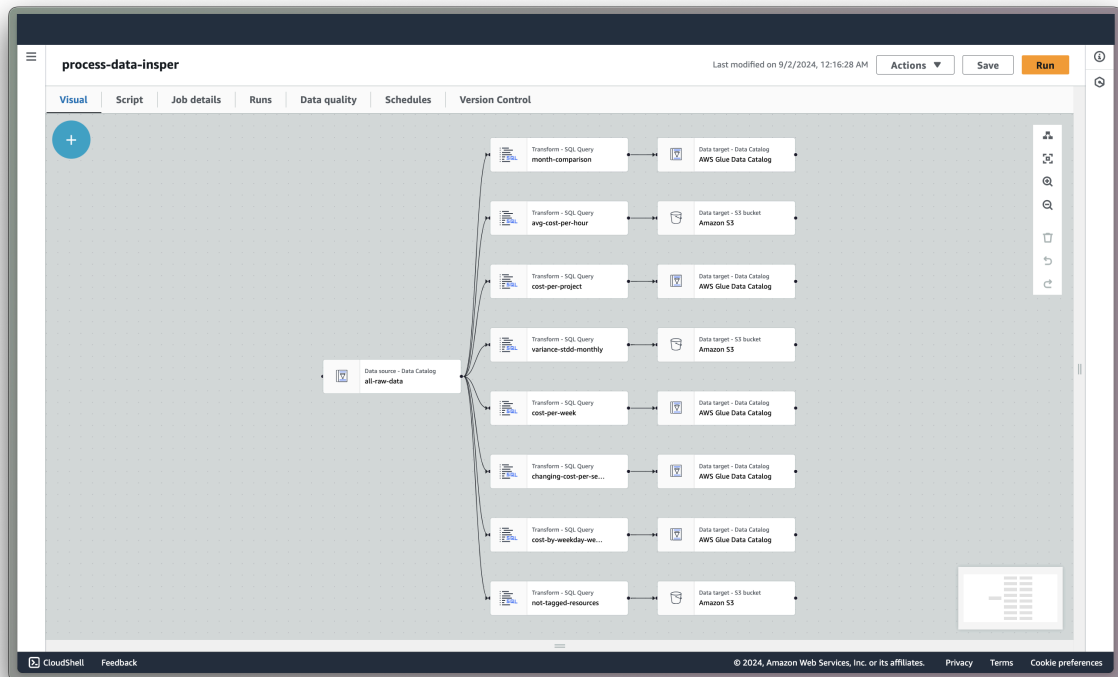
4. Reconstruction of Processing *Job* in AWS Glue:

The processing *Job* in AWS Glue, which had been removed, was completely rebuilt, requiring the reconfiguration of all blocks that compose the ETL (*Extract, Transform, Load*) flow. This *job* is responsible for processing the data captured by the *crawler* daily and performing the necessary transformations to calculate KPIs from AWS usage and cost data.

The *job* operation begins with the structuring of raw data in the database, which is done by the AWS Glue *crawler*. This *crawler* scans the specified directory in S3, identifying data and creating or updating the table catalog in AWS Glue, ensuring that data is ready to be accessed and transformed. With the data in the table, it is possible to access the data through a data source node (*Data Source*), which was configured to use the **raw-data** database and the **Data** table. This table contains all data relating to AWS cost and usage reports.

From this data source node, several transformation nodes are created (called *Transform - SQL Query*), as shown in Figure 4.5. In each transformation node, SQL queries are applied to manipulate the data and extract the necessary KPIs. During reconstruction, it was necessary to recreate the 9 SQL queries, since the final versions had been deleted and only test requests were available from the first phase of the project. These requests were recreated to ensure that the processed data could generate consistent results.

Figura 4.5 – AWS Glue processing *Job*.



Source: Prepared by the authors.

After executing the SQL queries, the resulting data needs to be stored. The storage of processed data is done both in S3 and in the AWS Glue *Data Catalog*. In some cases, the *Data Target S3 Bucket* node was used, which allows saving information directly to S3 and, optionally, creating a new table in the *Data Catalog* or updating the schema of an existing table. For this task, the option "*Create a table in the Data Catalog and on subsequent runs, update the schema and add new partitions*" was enabled, selecting the **processed-data** database and the table corresponding to the KPI.

However, in some situations, the option of creating or updating tables in the *Data Catalog* could not be used directly via the S3 node. In these cases, the ***Data Target of type Data Catalog*** node was used, which prioritizes the registration of data in the Glue database. This procedure ensured that the data was available and organized to be used later by Amazon Athena, which performs the final analyses and visualizations in Grafana.

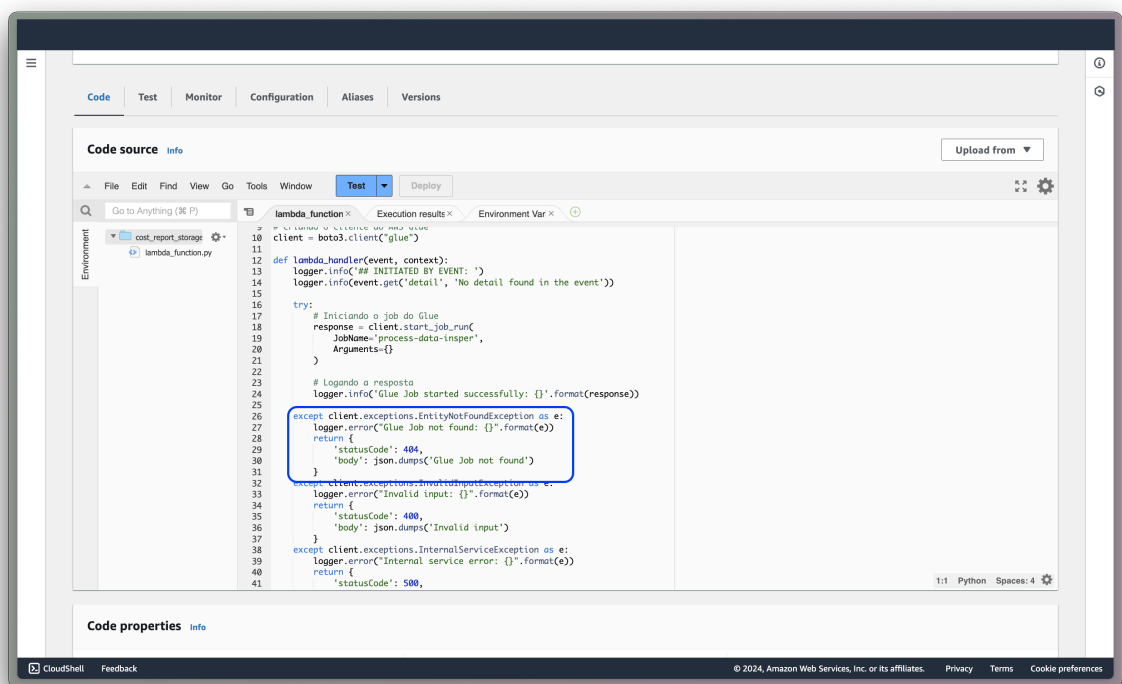
To ensure that the KPIs were being processed correctly, the *job* was executed multiple times. However, the initial execution presented a high time, varying between 15 and 25 minutes per execution. To improve performance, the processing machine type (***Worker Type***) was updated to **G 2x**, which offers greater processing capacity and reduced the *job* execution time, maintaining the same cost in terms of processing units (0.8-0.97 DPU per hour). This provided more efficient and agile execution, without increasing

the processing cost.

5. Improvement in Lambda Function:

The Lambda Function, responsible for triggering the **process-data-insper** job in AWS Glue, was improved. *Try-except* blocks were added to improve exception handling, and multiple conditional entries were configured to handle different failure scenarios. This improvement increased the function's resilience and facilitated diagnosis in case of future failures.

Figura 4.6 – Lambda function source code.



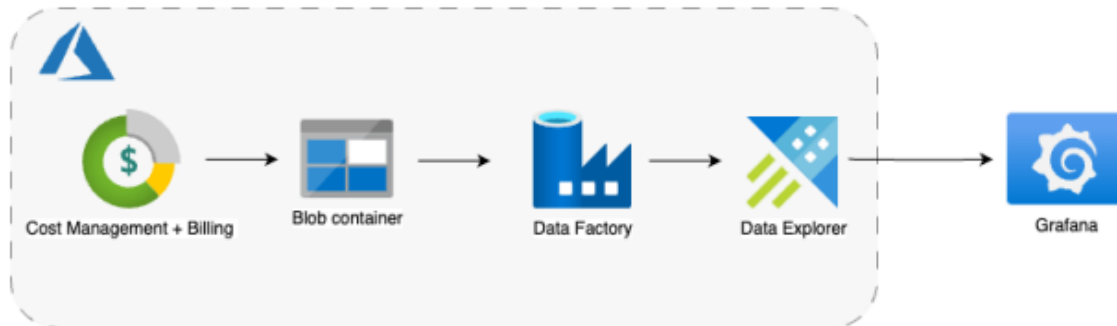
Source: Prepared by the authors.

4.1.1.3 Conclusion

With the implementation of the described solutions, the data flow in AWS was reestablished, and critical services, such as the Grafana dashboard, returned to functioning correctly. The *Top-Down* approach allowed problems to be identified and corrected efficiently, minimizing downtime and ensuring the continuity of PinPag operations. The data processing flow now operates stably, with due improvements implemented, ensuring continuous monitoring and the generation of cost reports and KPIs.

4.1.2 Azure Correction

Figura 4.7 – Diagram of the ingestion and processing architecture developed in Microsoft Azure.



Source: Frezzatti et al. (2023)

As with AWS, Azure's monitoring infrastructure also underwent a complete analysis and was diagnosed with a series of problems related to cost and usage data export and processing. The diagram in Figure 4.7 illustrates the Azure solution architecture, which includes Cost Management + Billing, Blob Container, Data Factory, Data Explorer, and Grafana.

4.1.2.1 Cost Data Export

The first problem detected in Azure infrastructure was related to cost data exports, which were configured to occur daily at 13:55. However, the exports were failing. The initial analysis revealed that, to reduce costs, the export directory in Blob Container had been deleted months earlier, which caused failures in the execution of scheduled tasks.

It was necessary to recreate the directory in Blob Container, thus reestablishing the data export functionality. With this, daily exports returned to occurring correctly, allowing data to be processed again without interruptions.

4.1.2.2 Credential Update

In addition to the failure in the export directory, the investigation revealed that some credentials, such as the *client id*, had been changed during an internal company restructuring. The old credentials, which were still configured in the monitoring services, were responsible for connection and authentication failures.

To correct this issue, all old credentials were replaced with new authentication information in each service. This update was necessary in both Data Factory and Data Explorer, ensuring the continuity of integration processes between the different services.

4.1.2.3 Reconnection between Data Factory and Data Explorer

Another critical point identified was the removal of the application responsible for the connection between Azure Data Factory and Azure Data Explorer. During the restructuring, the application was deleted, which caused the interruption in data flow between these two platforms.

To solve the problem, the application was recreated from scratch. This process involved reinstalling the connectors and redefining access permissions, which reestablished communication between Data Factory and Data Explorer. After completing this stage, the data flow returned to functioning stably, without further interruptions.

4.1.2.4 Diagnosis in Grafana Dashboard

The dashboard in Grafana configured for Azure also presented problems similar to those found in AWS. The credentials used to connect with Azure services were outdated, referring to information prior to the company restructuring. This problem prevented Grafana from correctly accessing the data stored in Azure.

After updating the credentials and validating the accesses, the dashboard was successfully reconfigured, reestablishing the connections and allowing SQL queries to be executed correctly. As in the AWS scenario, it was necessary to recreate some of the SQL requests for data monitoring to return to functioning.

4.1.3 Wazuh Correction

The *Security Information and Event Management* (SIEM) collects, stores, and provides *logs* from the main instances of PinPag's IT infrastructure. With the use of Wazuh, an open-source SIEM tool, the analysis of these *logs* is centralized, facilitating the identification and response to security incidents. Thus, it correlates events from various sources, offering a detailed view of activities in the environment, which improves threat and anomaly detection ([Wazuh, 2024](#)).

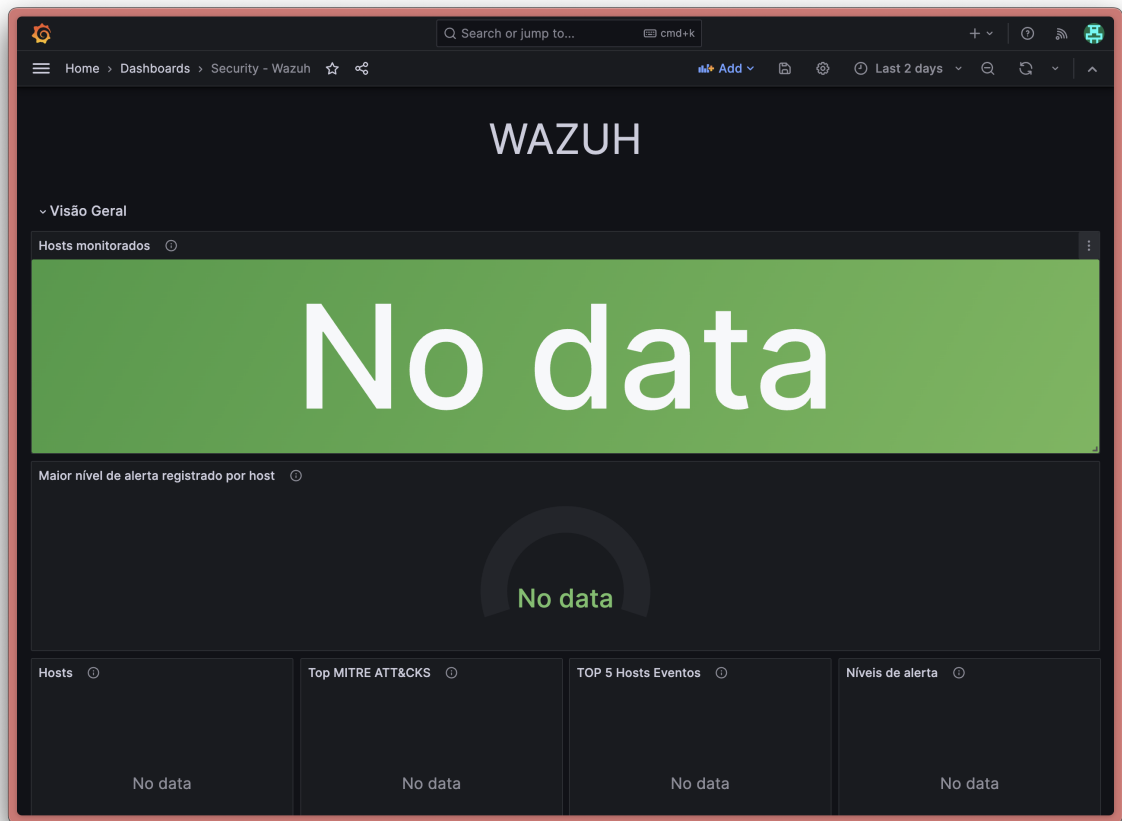
To access the data collected by Wazuh in a customized way, a dashboard was created in Grafana, which provides a personalized view of security events, detected vulnerabilities, and the general state of the infrastructure, contributing to more efficient management of critical assets.

4.1.3.1 Previous State

Upon receiving access to the Grafana system linked to Wazuh Indexer, it was noticed that data was no longer being received in Grafana. We proceeded to diagnose and

correct the service, preventing this error from occurring again. Figure 4.8 presents the consequence of the problem in the visualization of *logs* generated by Wazuh Indexer.

Figura 4.8 – Capture of Grafana dashboard, in state prior to its correction.

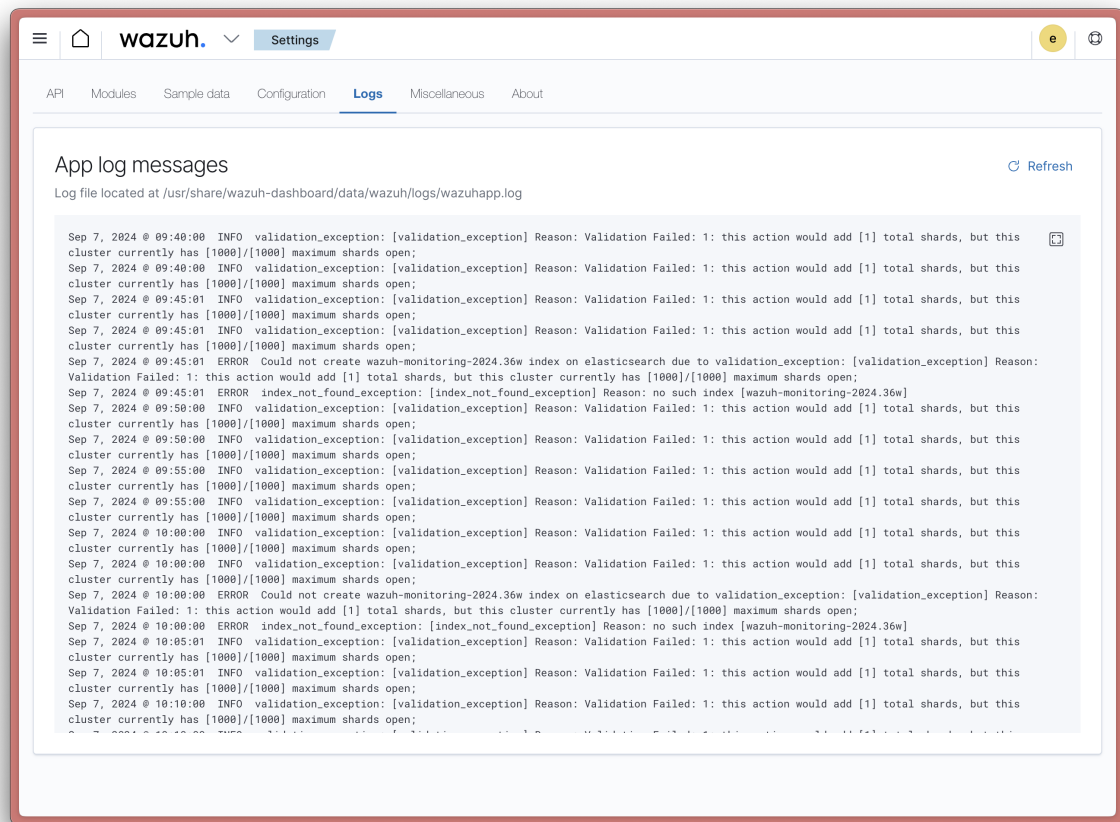


Source: Prepared by the authors.

4.1.3.2 Diagnosis

When investigating Wazuh Indexer, it was identified that the *cluster* had reached the limit of 1000 *shards*, preventing the creation of new indices. That is, although the system was generating new *logs*, they were not being stored. The mentioned *shards* are subdivisions, in indices, that distribute stored data across different *cluster* nodes, increasing search performance within this data, at a higher storage cost. In Figure 4.9, the error is verified in detail:

Figura 4.9 – Capture of *Wazuh dashboard*, in the *logs* section, extracted from the instance where the service is installed.



Source: Prepared by the authors.

Another piece of evidence acquired during the diagnosis phase was performed through the *OpenSearch Dashboards* sub-interface, which allowed confirming the blockage in the creation of new indices.

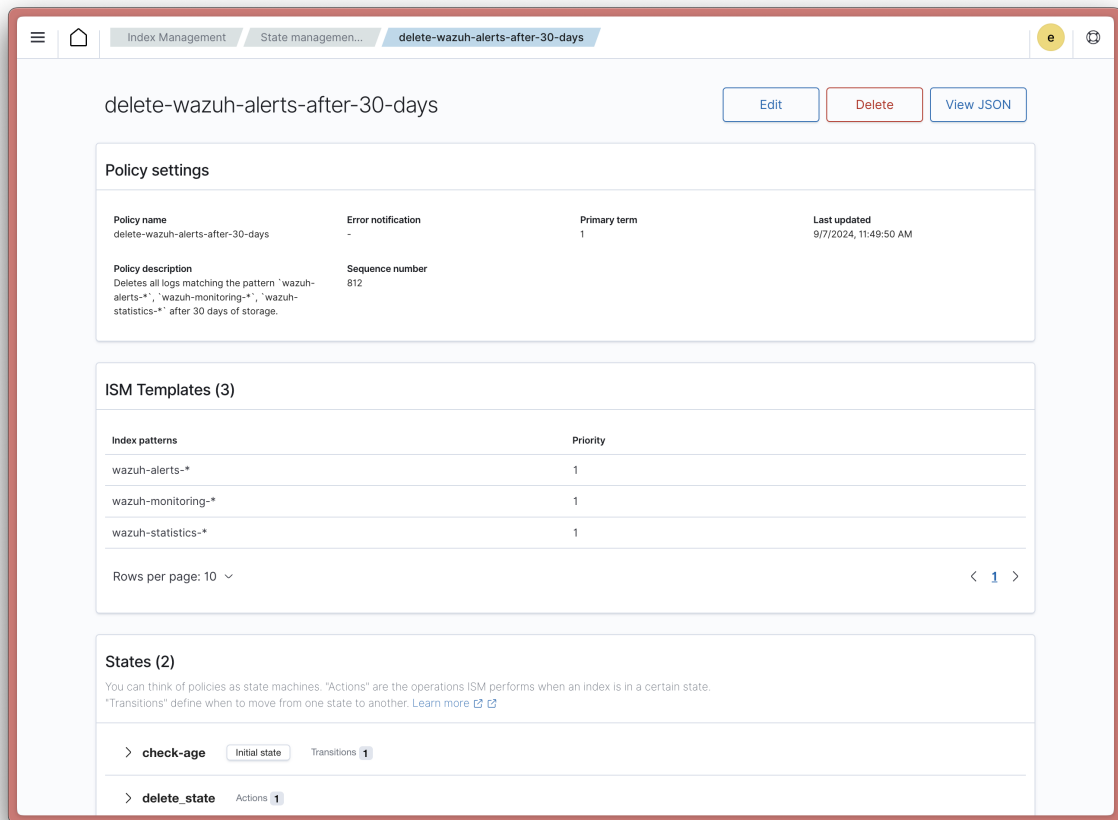
4.1.3.3 Creation of Log Management Policy

Facing the limit of *shards*, an index management policy was developed, a concept based on *Index State Management (ISM)* from *OpenSearch*. The policy aims to delete *logs* older than 30 days from their generation, since the company advised that data older than this limit has no utility, thus allowing automatic space release in the *cluster*, without the need to increase the *shards* limit.

The policy was created through the Wazuh interface, configured to apply to all indices that followed the following patterns: **wazuh-alerts-***, **wazuh-monitoring-*** or **wazuh-statistics-***. For this, a retention rule was created, implementing the state

transition **check-age** and **delete-state**, ensuring the correct transition of *logs*. The complete policy configuration can be observed in Figure 4.10.

Figura 4.10 – Capture of *Wazuh dashboard*, in the ISM section, showing the created policy.



Source: Prepared by the authors.

4.1.3.4 Retroactive Application of Policy

To release the space occupied by *logs* already existing before the limit overflow, it was necessary to apply the policy retroactively to existing indices. This measure, in addition to releasing the necessary space, confirmed the good functioning of the policy. As all *logs* dated more than 30 days from their creation, the space was completely released.

4.1.3.5 Validation

To ensure the system's functioning, its behavior was monitored for retention and deletion of *logs*. The policy functioned as expected, with old *logs* deleted and new *logs* correctly processed, and it was confirmed that there was no need to increase the number of *shards*. Figure 4.11 shows the *Grafana dashboard* with continuous access to *Wazuh Indexer* indices.

Figura 4.11 – Capture of *Grafana dashboard*, showing data provided by *Wazuh Indexer*.



Source: Prepared by the authors.

4.2 WAF

4.2.1 ModSecurity 3.0

After alignment with the company, it was decided to implement *ModSecurity 3.0* initially in a test environment [removed]

. This environment was configured in an external cloud provider, offering a controlled and isolated scenario to evaluate the effectiveness of ModSecurity as a security solution for the Apache server. The implementation was based on the documentation available on the OWASP ModSecurity GitHub ([OWASP ModSecurity, 2024](#)).

The complete implementation is available in Appendix Section A.1.

4.2.1.1 Test Environment Configuration

To ensure accurate validation, the test machine was configured with the following specifications, [removed] :

- **CPU Platform:** Intel Broadwell
- **Operating System:** Ubuntu 22.04 (Jammy Jellyfish)
- **Web Server:** Apache 2.4.52
- **Location:** Company's Azure platform

[sentence removed]

In initial tests, ModSecurity was configured in *DetectionOnly* mode (detection only), allowing the recording of suspicious access attempts without directly interfering with requests. This mode was chosen to avoid impact on legitimate traffic while monitoring system performance and behavior, providing an initial basis for security rule adjustments and evaluation of the WAF's blocking potential.

4.2.1.2 Validation and Tests

During the validation process, we performed tests with simulations of common attacks, such as *Cross-Site Scripting (XSS)*, sending suspicious requests to the server to verify if ModSecurity identified and recorded these activities. The tests were successful: ModSecurity recorded in detail each suspicious request in its audit *log*, including the source IP address, the rule triggered, the type of threat detected, and the anomaly score attributed to the request.

Figure 4.12 illustrates an example of the generated audit *log*:

Figure 4.12 – ModSecurity Audit Log showing detection of an XSS attack attempt in *DETECTION ONLY* mode.

```

~$ curl -I "http://10.128.0.3/?test=<script>alert(1)</script>"
HTTP/1.1 200 OK
Date: Sun, 10 Nov 2024 00:58:36 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Sat, 09 Nov 2024 21:28:20 GMT
ETag: "29af-626818bedf4e"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html

~$ sudo tail -f /var/log/apache2/modsec_audit.log
Apache-Error: [file "apache2_util.c"] [line 271] [level 3] [client 10.128.0.3] ModSecurity: Warning. Operator GE matched 5 at TX:inbound_anomaly_score. [file "/usr/share/modsecurity-crs/rules/RESPONSE-980-CORRELATION.conf"] [line "91"] [id "980130"] [msg "Inbound Anomaly Score Exceeded (Total Inbound Score: 18 - SQLI=0,XSS=15,RFI=0,LFI=0,RCE=0,PHP=0,HTTP=0,SESS=0); individual paranoia level scores: 18, 0, 0, 0"] [ver "OWASP_CRS/3.3.2"] [tag "event-correlation"] [hostname "10.128.0.3"] [uri "/index.html"] [unique_id "2aAFFQMDHXUollM4K5ZGpAhaaE*"]
Stopwatch: 1731200316582590 4571 (- - -)
Stopwatch2: 1731200316582590 4571; combined=3489, p1=644, p2=1359, p3=46, p4=1083, p5=357, sr=87, sw=0, l=0, gc=0
Response-Body-Transformed: Dechunked
Producer: ModSecurity for Apache/2.9.5 (http://www.modsecurity.org/); OWASP_CRS/3.3.2.
Server: Apache/2.4.52 (Ubuntu)
Engine-Mode: "DETECTION_ONLY"
--3b445439-2--

```

Source: Prepared by the authors.

In the audit *log*, it is observed that ModSecurity recorded an **anomaly score exceeded** alert, indicating that the simulated request was identified as a possible XSS attack attempt. As expected for detection mode, ModSecurity did not block the request, but recorded the event with all relevant details, providing a solid basis for future adjustments and improvements in the security of the production environment.

4.2.1.3 Validation Conclusion and Next Steps

Based on the tests performed in the Azure test environment, it was possible to confirm that ModSecurity was correctly configured and operational, fulfilling its role of recording attack attempts without interfering with legitimate traffic. This detailed monitoring was essential to evaluate system behavior and identify opportunities for adjustment in security rules, reinforcing confidence in ModSecurity's effectiveness.

[sentence removed]

ModSecurity 3.0 proved to be a robust and efficient solution, capable of identifying threats with precision, providing an essential additional layer of security for the company's infrastructure. All documentation of this process, including test captures and audit *logs*, provides a solid basis for effective implementation in production.

4.2.2 WAF Dashboard

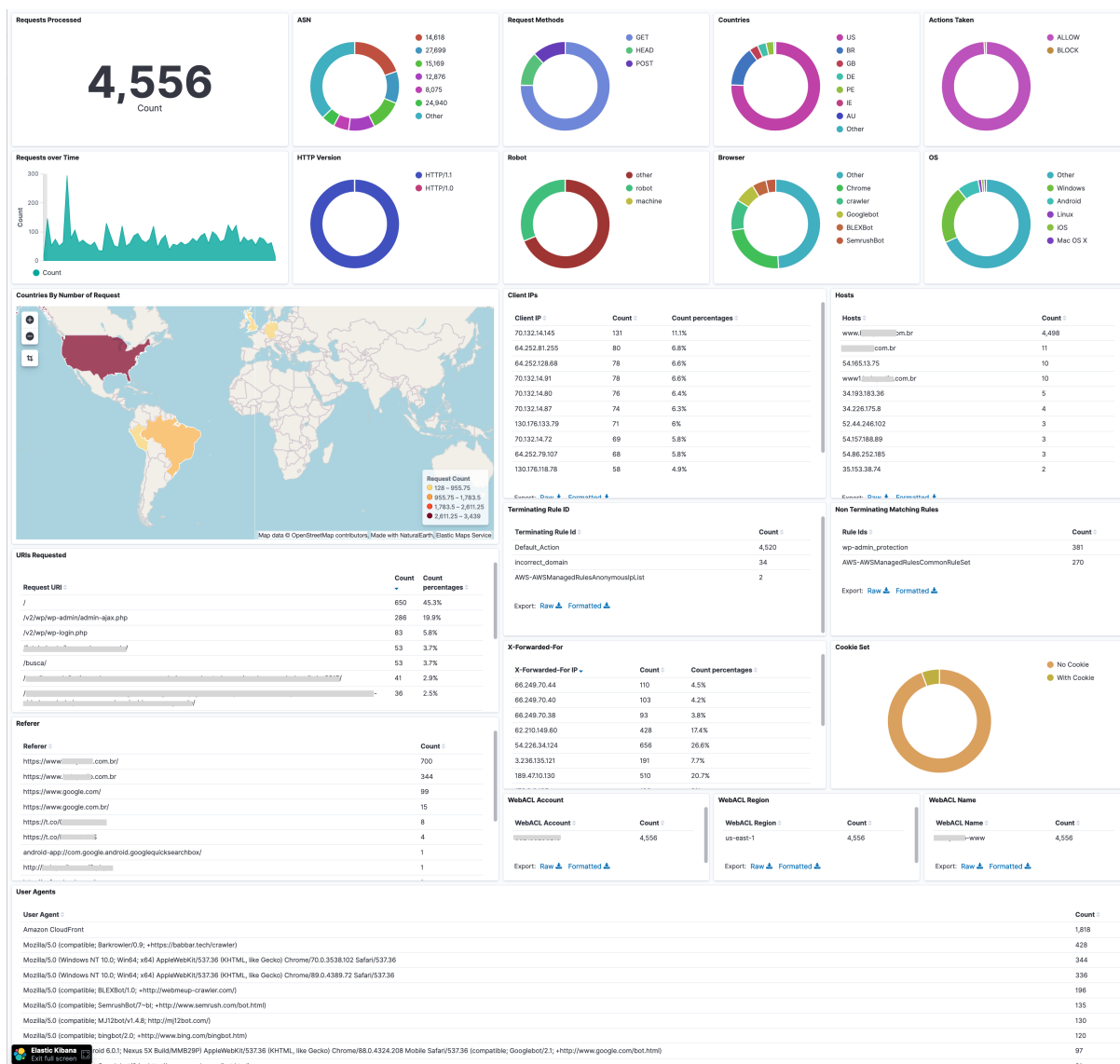
[sentence removed]

According to the pain demonstrated by the client, it was proposed to implement a dashboard, to show *insights* and a general overview of the *logs* collected by the WAF service.

Initially, the dashboard implementation was based on the AWS example repository on GitHub, the *AWS WAF Operations Dashboards* ([SAMPLES](#), 2021). However, due to the outdated content, it was not possible to completely follow the step-by-step of the proposed solution.

Nevertheless, Figure 4.13, extracted from the tutorial, served as a reference for the creation of KPIs and for the development of the dashboard in Grafana.

Figura 4.13 – Example of WAF dashboard.

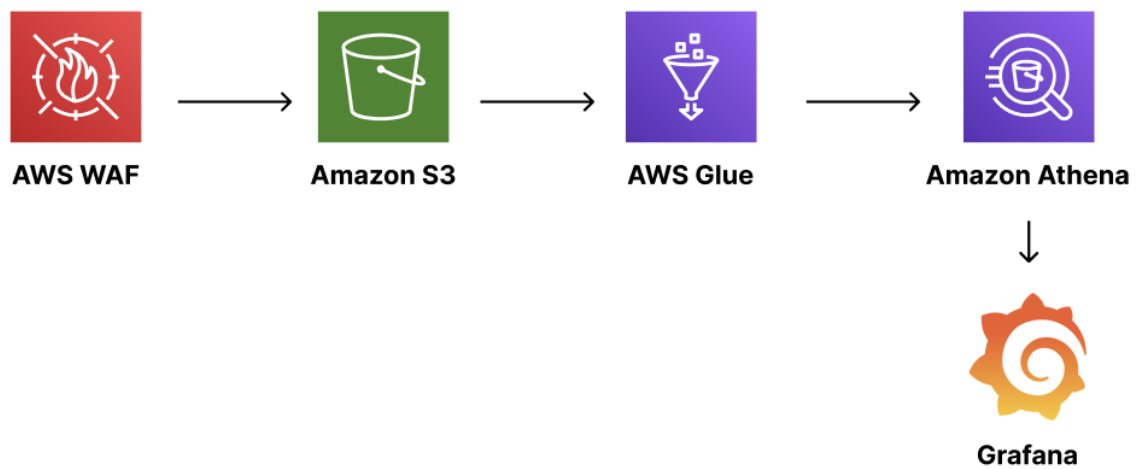


Source: **Samples (2021)**.

Maintaining coherence with the architecture already implemented in other solutions, for data capture, processing, and availability, the WAF dashboard followed the flow shown in Figure 4.14.

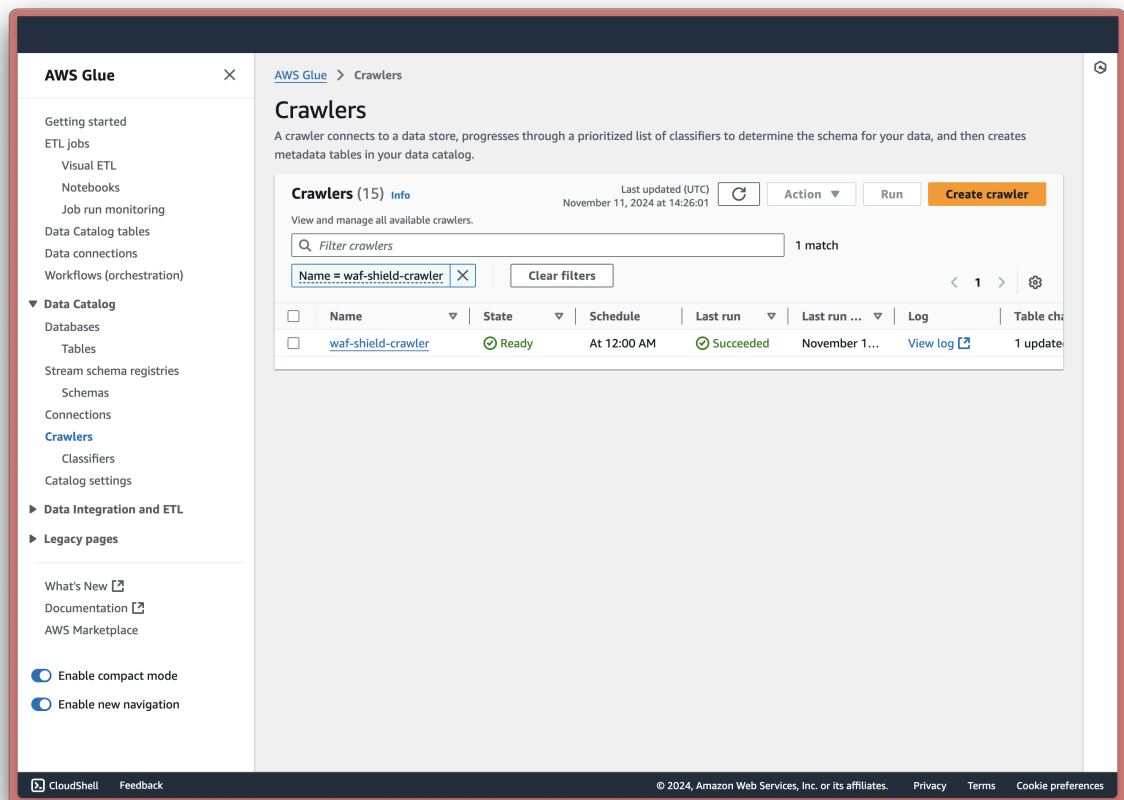
Within the *Web Access Control List* (WebACL) settings, in the *Logging and metrics* tab, *log* generation was activated and their destination was configured to **aws-waf-logs-aws-us-east-1-waf-ops-pinpag**, an S3 *bucket* created only to store *logs* from WAF services. In AWS Glue, the *crawler* **waf-shield-crawler** was configured within the *Data Catalog*, as in Figure 4.15, which daily retrieves and compiles data from the S3 *bucket*, and stores it in the **waf-shield-database** database in the **awslogs** table, with the configuration of its columns shown in Figure 4.16.

Figura 4.14 – Diagram of WAF log capture and processing flow in AWS.



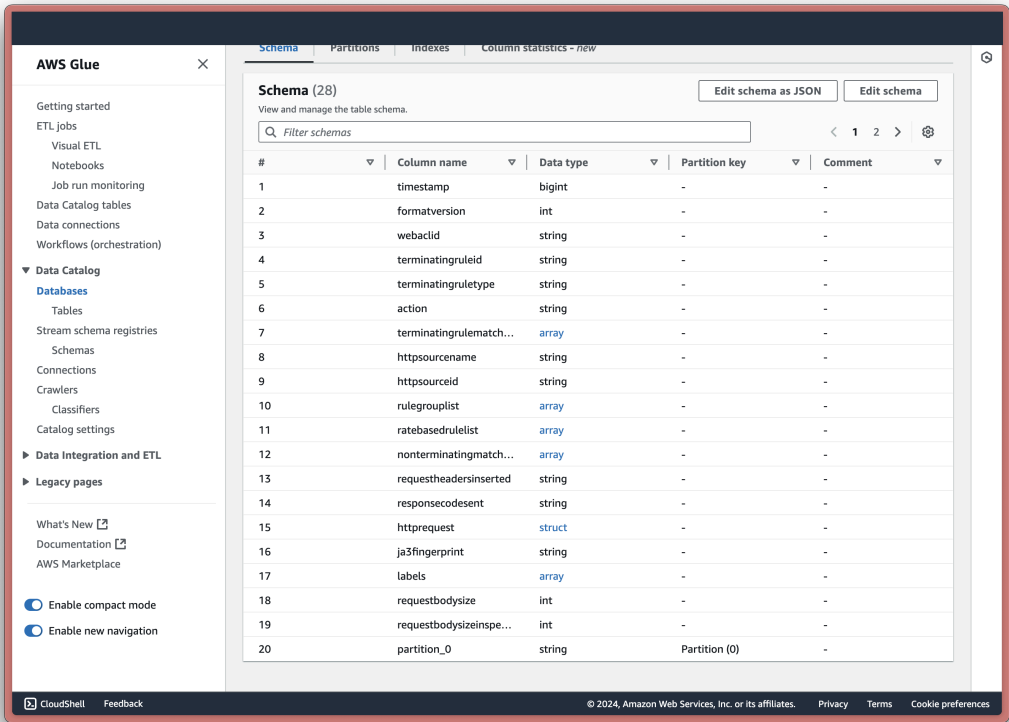
Source: Prepared by the authors.

Figura 4.15 – AWS Glue *crawler* visualization panel.



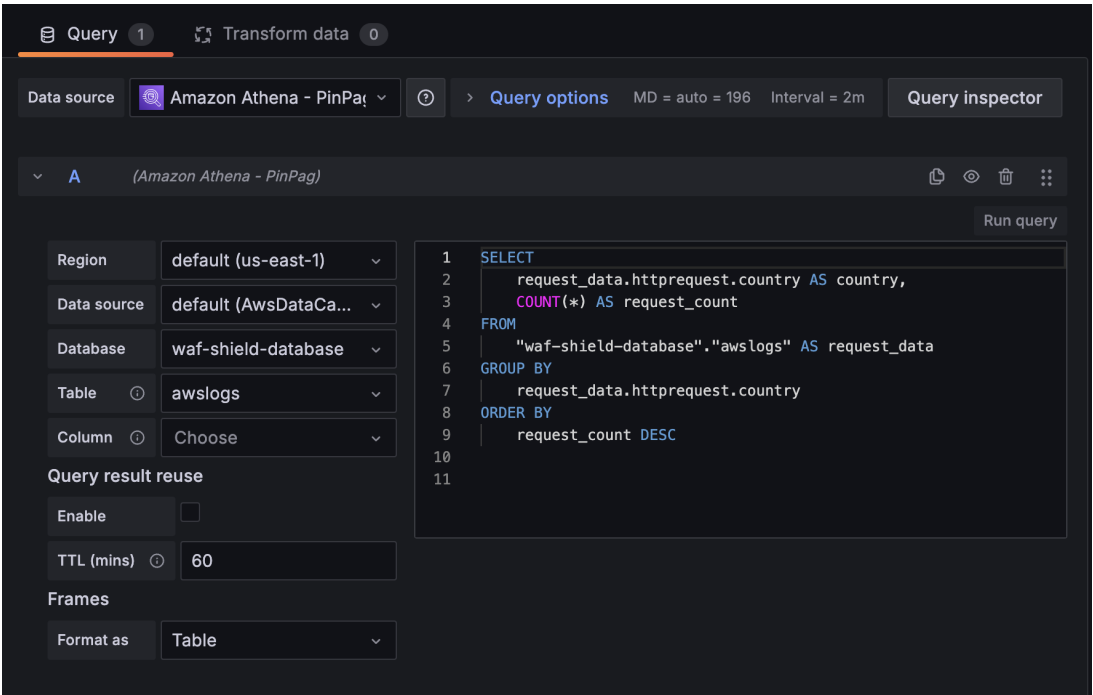
Source: Prepared by the authors.

Figura 4.16 – Column schema configuration panel for the waf-shield-database database in AWS Glue.



Source: Prepared by the authors.

Figura 4.17 – Amazon Athena *Data Source* configuration panel in Grafana for one of the metrics.



Source: Prepared by the authors.

Finally, Amazon Athena was configured as *Data Source*, in each module of the Security - WAF Operations dashboard in Grafana, each performing specific SQL *SELECT queries*, as in Figure 4.17.

To configure each module, the KPIs to be implemented were determined, as follows:

1. ***Requests Processed***: Total number of requests processed by the WAF during the analyzed period. This metric is obtained by counting all records in the `awslogs` table in `waf-shield-database`;
2. ***Autonomous System Number (ASN)***: Count of requests by IP addresses from autonomous systems, which are managed by a single entity (such as an internet service provider, company, or other type of institution) and follow a common routing policy. This metric is calculated by summing the number of requests originated from each IP address (`clientip`) in `httprequest`;
3. ***Request Methods***: Count of requests for each HTTP method processed by the WAF, such as *GET*, *POST*, *PUT*, and others. This metric is obtained by grouping and summing the number of requests for each HTTP method in the `httpMethod` column in `httprequest`;
4. ***Countries***: Count of requests processed by the WAF service, grouped by country of origin. This metric is obtained by grouping and summing the number of requests for each country in the `country` column in `httprequest`;
5. ***Requests over Time***: Number of requests processed by the WAF over time. This metric is calculated by grouping requests based on `timestamp`, converting it to local time, and summing the total records per time interval in the `timestamp` column in `awslogs`;
6. ***HTTP Version***: Count of requests by HTTP protocol version used. This metric is obtained by grouping the `httpVersion` records in `httprequest` in the `awslogs` table;
7. ***Browser***: Count of requests grouped by origin browser. This metric is obtained through the `User-Agent` field, categorizing into specific browsers, such as Chrome, Firefox, Safari, Edge, Googlebot, among others;
8. ***Countries by Number of Requests***: Number of requests grouped by country of origin. This metric is obtained by grouping the `country` field in `httprequest` and counting it, thus representing the data on a geographic map;

9. **Actions Taken:** Count of actions applied by the WAF, such as *BLOCK* or *ALLOW*. This metric is obtained by grouping the `action` field in `terminatingRule` within `rulegrouplist` and counting it;
10. **Operating System:** Count of requests by client operating system. This metric is obtained through the `User-Agent` field, grouping it by operating system, such as Windows, Linux, Android, iOS, among others;
11. **Client IPs:** Count and percentage of requests by client IP address, within the total processed requests. This metric is obtained through the `clientip` field in `httprequest`;
12. **Hosts:** Count of requests by *host* address. This metric uses the `Host` field extracted from HTTP requests to identify the traffic destination, for domains and IPs;
13. **URIs Requested:** Count and percentage of requests grouped by requested URI, within the total processed requests. This metric is calculated based on the `uri` field of each request in `httprequest`;
14. **Terminating Rule ID:** Count of requests by terminating rule identifier, defined according to that WAF's traffic policies, which is applied to each request. This metric is calculated from the `terminatingruleid` field within `awslogs`;
15. **Non Terminating Matching Rules:** Count of requests with non-terminating rules, that is, which do not interrupt analysis in the WAF, allowing other rules to be evaluated in sequence. This metric is extracted from the `nonterminatingmatchingrules` field of each `rulegrouplist` entry in `awslogs`;
16. **Cookie Set:** Count of requests according to the presence or absence of *cookies*. This metric identifies whether the `Cookie` field is filled or not;
17. **WebACL Account:** Count of requests by account responsible for the AWS WebACL. The metric identifies the AWS account associated with each `webaclid`, extracting it from the ARN (Amazon Resource Name). It is especially useful for cases where more than one account is used for AWS services;
18. **WebACL Region:** Count of requests by AWS WebACL region where they were processed. This metric is extracted from the `webaclid` field and identifies accesses in different AWS availability regions where the service is being made available, such as, for example, `us-east-1` or `us-west-1`. It is especially useful for cases where the WebACL is configured in more than one AWS availability region;
19. **WebACL Name:** Count of requests by WebACL name where they were processed. This metric is obtained from the `webaclid` field in the `webacl` key. It is especially

useful for cases where there are different WebACLs configured to monitor instance traffic;

20. ***User Agents***: Count of requests by user agent, identifying accesses from different devices, browsers, and robots. This metric is extracted from the `header.value` field, with `User-Agent` in `header.name`.

Finally, the proper functioning of the KPIs in the WAF dashboard in Grafana can be verified in Figure 4.18.

Figura 4.18 – Implementation of AWS WAF dashboard in Grafana.



Source: Prepared by the authors.

4.3 CONNECTION MAP

The development of the Connection Map was performed using AWS's existing infrastructure to collect, process, and make available data about the company's network and security configuration. However, since this data is not natively available in AWS, it was necessary to build a Python program, using specific libraries to access and manipulate infrastructure information.

4.3.1 General Operation and Data Structuring

The developed function uses the `boto3` library, native to AWS, to access infrastructure resources and perform programmatic queries to network components, such as *Elastic Compute Cloud* (EC2) instances, security groups (SG), and subnets. To ensure unique identifiers for each node and edge, the `uuid` library was employed, allowing clear distinction between the various entities represented in the connection graph. The `pandas` library enabled the manipulation and export of collected data in `parquet` format, which is ideal for AWS's data availability architecture and facilitates integration with visualization tools, such as Grafana.

The data collection and organization flow was structured in four main stages:

- **Data Collection:** The `lambda_handler()` function performs infrastructure data collection through `boto3` methods, such as `describe_security_groups()`, `describe_instances()`, and `describe_subnets()`. Each call returns details of network components, which are stored in temporary dictionaries for later processing.
- **Node Construction:** Each network component, such as EC2 instances, SG, and subnets, is represented by a node in the graph, created by the `create_node()` function. Each node is assigned a unique identifier, a representative color, and contextual information, such as title and IP address.
- **Edge Construction:** The connections between nodes, or edges, represent interactions and dependencies between components. Using the `create_edges()` function, connections between EC2 instances and their IPs, security groups, and subnets were mapped. In it, compliance with the rules presented by the company was evaluated.
- **Data Storage and Export:** After building nodes and edges, data is organized into *dataframes* with `pandas` and exported in `parquet` format. The `Sessions.save_to_bucket()` function stores the `parquet` files in an S3 bucket, organizing the data for the Connection Map.

Figura 4.19 – Data structure of the `edges.parquet` file, containing connections between components.

Screenshot removed from this public version. It showed details of PinPag's production systems.

Source: Prepared by the authors.

Figura 4.20 – Data structure of the `nodes.parquet` file, containing node information.

Screenshot removed from this public version. It showed details of PinPag's production systems.

Source: Prepared by the authors.

It is important to note that there may be `NaN` or `None` fields in the data files, especially in the `edges.parquet` file. This occurs because not all edges have attributes such as `thickness` or `color`, which are applicable only in specific cases, such as when vulnerabilities are detected. However, to maintain the symmetry of the data structure, it is necessary that all fields be present, filling absent values with `NaN` to ensure consistency and compatibility of data throughout the storage and analysis process.

4.3.2 Algorithm Detail

To implement the Connection Map, it was necessary to understand how the *node graph* visualization works in Grafana. Based on the documentation analysis ([Grafana Labs, 2024](#)), we defined the essential parameters to generate a clear and informative visualization. Table 4.1 lists the fields selected for node configuration, highlighting that the only mandatory field to create a node is the unique identifier (`id`). The `title` and `subtitle` fields are used to display the name and details of the resource, when applicable. Colors were assigned according to resource category: EC2 instances in yellow, *Relational Database Service* (RDS) instances in orange, security groups in dark blue, port ranges in green, internal IP ranges in light blue and external ones in pink. Edges connect nodes with default values, except in cases of inadequate configuration, where red color and greater thickness are used.

Due to the customization limitations of the Athena data source in Grafana, the graph visualization was implemented statically. Thus, the algorithm previously defines visual attributes such as colors, thicknesses, and transparencies, while Grafana is responsible only for the final rendering of the graph.

Quadro 4.1 – Field Specifications for the node graph.

Field name	Type	Description
Node Fields:		
id	string	Unique identifier of the node. This ID is referenced by the edge in the source and target fields.
Edge Fields:		
id	string	Unique identifier of the edge.
source	string	Id of the source node.
target	string	Id of the target node.
Optional Node Fields:		
title	string	Node name visible just below the node.
subtitle	string	Additional name, type, or other identifier displayed below the title.
Optional Edge Fields:		
thickness	number	The thickness of the edge. Default: 1.
Optional Fields:		
color	string/number	Can be used to specify a single color for the node or edge.

Source: [Grafana Labs \(2024\)](#).

Initially, the algorithm stores all security groups in a dictionary (hash table), recording information such as group identifier, name, associated VPC ID, and IP permission rules contained in the *IpPermissions* field, which determine which port ranges can be accessed by a given IP. Each security group is then represented by a node in the graph.

Next, the list of EC2 instances is traversed. For each instance, nodes representing the instance itself and its private IP are created, connected by an edge. For each security group associated with the instance, an edge is created between the instance node and the respective security group node. The same procedure is applied to RDS instances.

Subsequently, nodes are created for the IP ranges of each *subnet* present in the account, ensuring that the company's internal infrastructure is completely represented in the graph.

After representing resources as nodes, the algorithm builds edges based on security group rules. Each rule generates a new node for the port range, following the criterion that each port range is connected to only one node. If the rule is related to an IP range, it

is verified whether the IP is within a mapped range; otherwise, it is an external connection and a new node is created. The port range node is then connected to the corresponding IP range node.

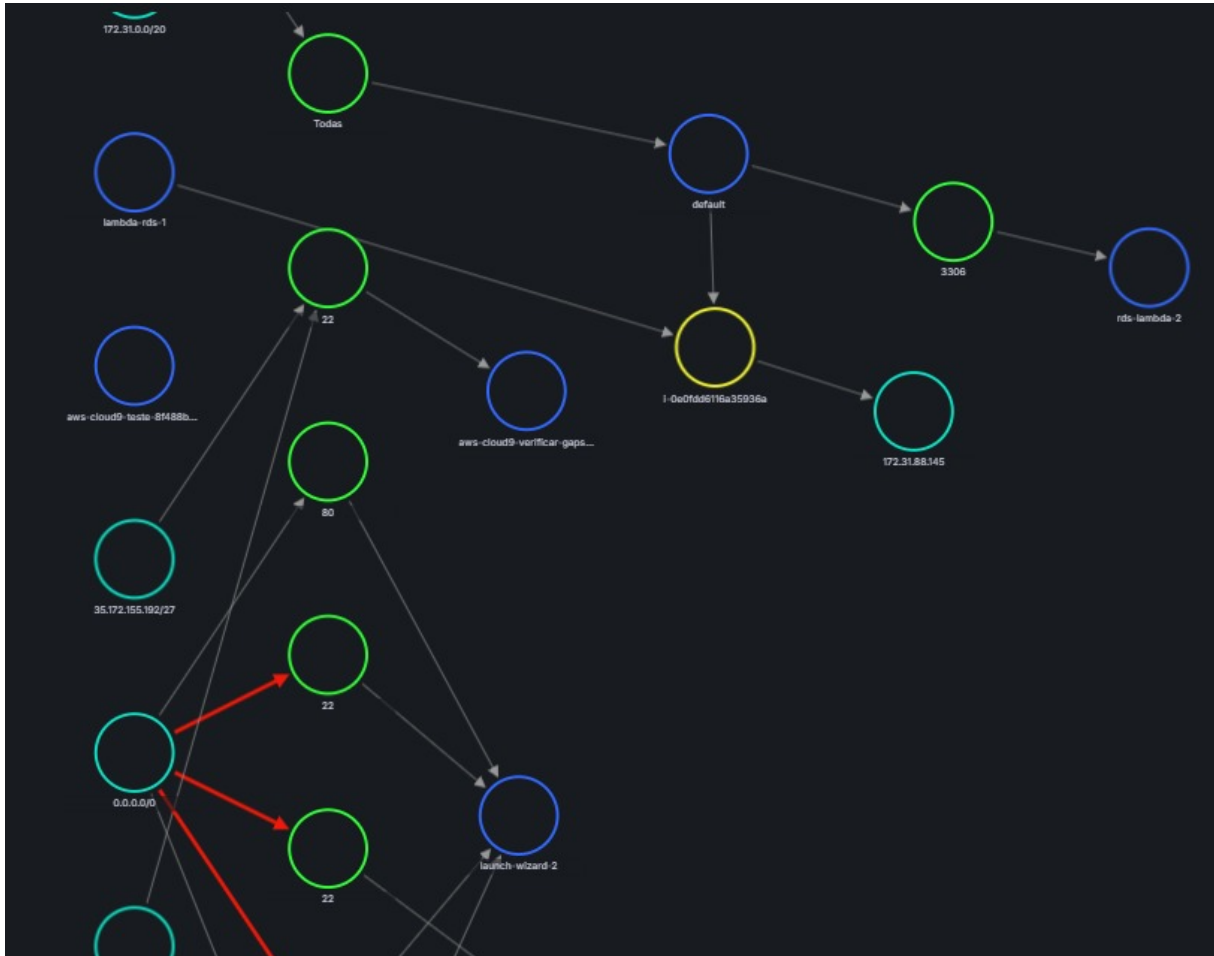
In cases where the IP range is "0.0.0.0/0" and the ports do not correspond to those used for secure traffic (80 and 443), the edge is highlighted in red and receives greater thickness, indicating an insecure configuration. For rules that direct to another security group, a direct connection is created between the port range node and the security group node.

The final stage consists of adjusting the integration to the company's environment. Data is converted into a `pandas dataframe` and saved in `.parquet` format in the `s3://aws-glue-pinpag-insper/mapa-conexoes-insper/` directory. After code execution, *crawlers* are automatically triggered to scan the directory and register information in databases.

4.3.3 Implementation and Tests in Insper Environment

For initial tests, we used Insper's internal cloud computing environment, which contains a limited amount of nodes and configuration data. This simplified environment provided an ideal base to validate the main function functionalities, allowing adjustments in data collection and structuring, as illustrated in Figure 4.21.

Figura 4.21 – Connection Map dashboard in Grafana, demonstrating the test environment at Inspier.



Source: Prepared by the authors.

4.3.4 Migration and Implementation in PinPag Infrastructure

After test validation, the function migration process to PinPag infrastructure was initiated, using AWS Lambda service, which allows on-demand processing without the need for a dedicated machine. Lambda automatically manages the necessary infrastructure, executing code whenever activated, ensuring efficient and economical implementation for this task.

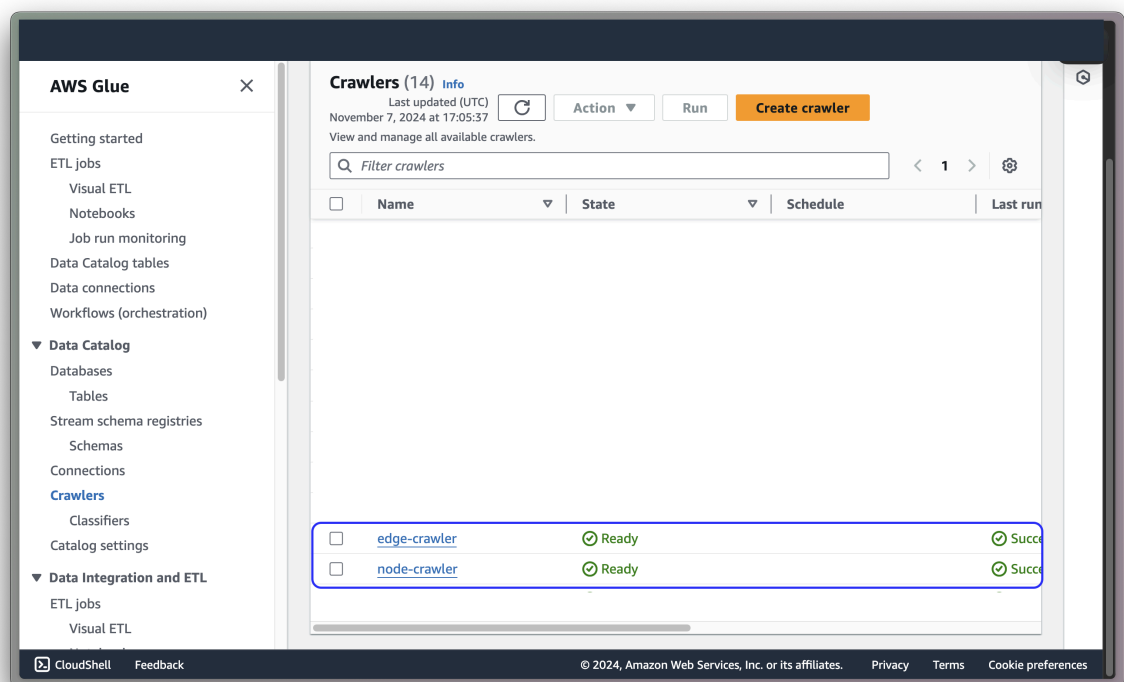
The processed data is stored in an S3 *bucket* named `aws-glue-pinpag-insper`, with a specific directory called `mapa-conexoes-insper`. Within this directory, there is an organized structure with a folder for raw data (`raw`), containing separate subdirectories for storage of nodes and `edges`.

After function execution and data storage in S3, the `edge-crawler`, one of the dedicated *crawlers*, is activated for data monitoring and updating. This *crawler* is responsible for scanning the `edges` directory in S3, identifying updates in `edges` files and

automatically saving information in the `mapa-conexoes-database` database. The same occurs for nodes, making data available in the same base, in the `nodes` table.

The database has two main tables, `nodes` and `edges`, which contain the columns and data types necessary to ensure a consistent structure optimized for querying. This architecture allows efficient integration with Grafana for visualization of updated data in real time, providing continuous monitoring of connections and helping the team identify possible vulnerabilities.

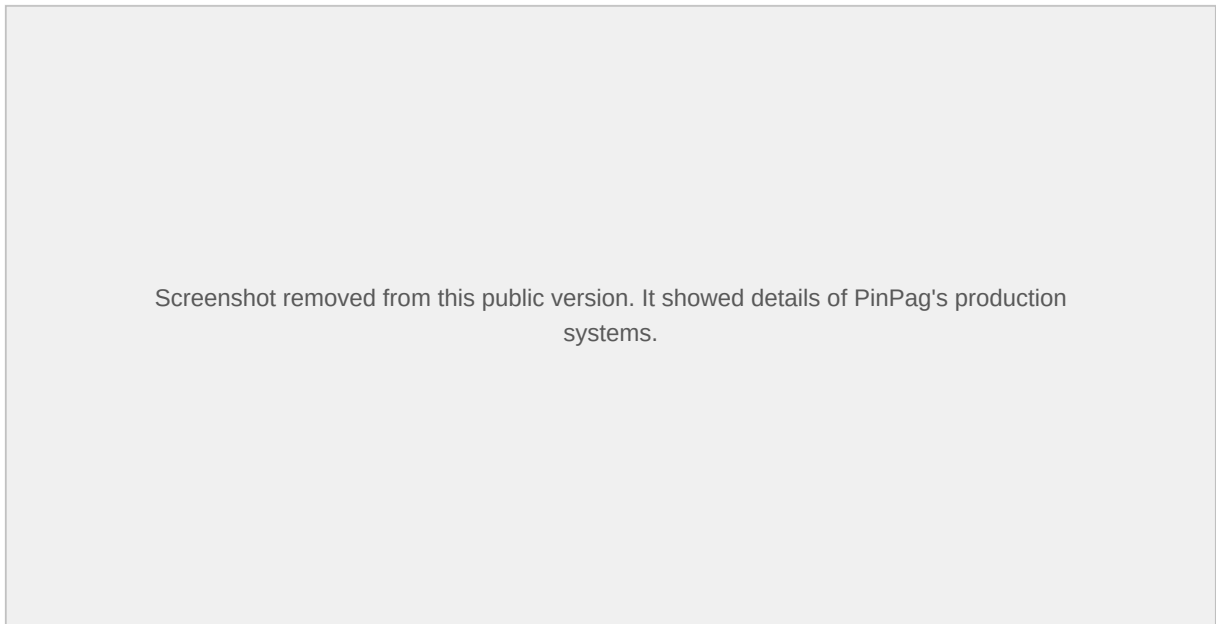
Figura 4.22 – Configuration of edge-crawler in AWS Glue for automatic table updates.



Source: Prepared by the authors.

After these configurations and correct data loading into tables, it is possible to access them through Grafana. For this, a dashboard called **Mapa de Conexões** was created, which makes all information available in a graph-style visualization.

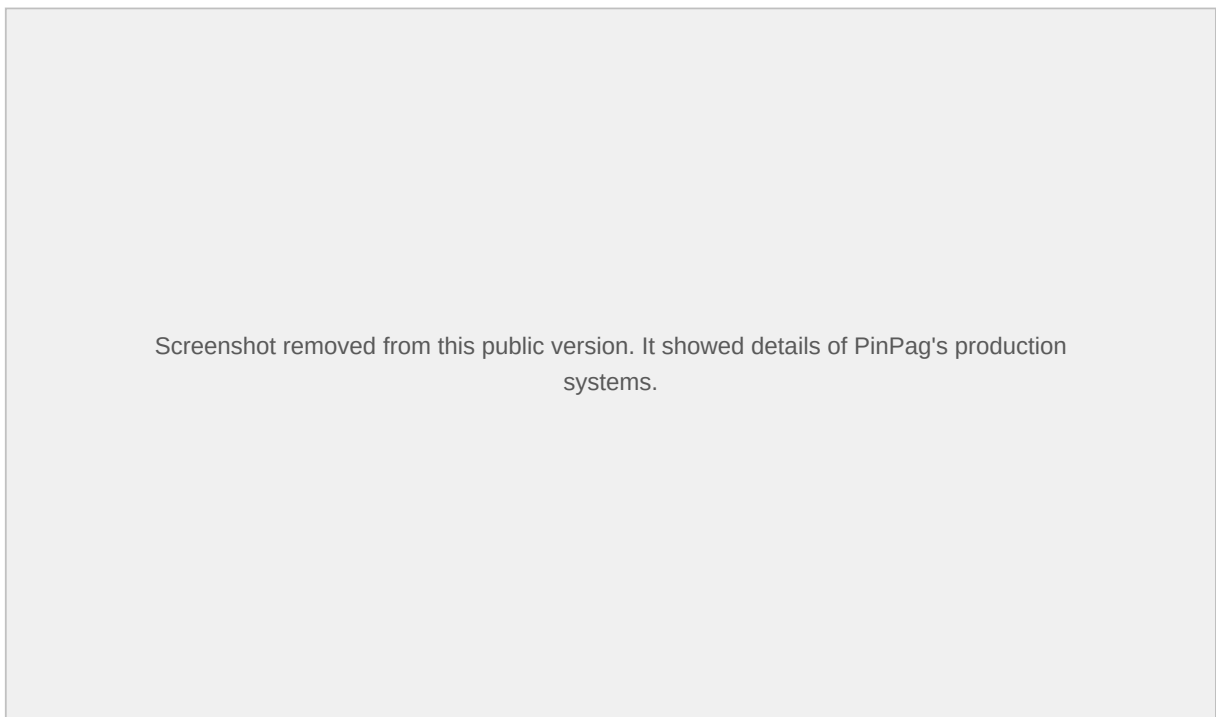
Figura 4.23 – Visualization of Connection Map in Grafana, presenting network connections in graph format.



Source: Prepared by the authors.

4.4 APPLICATION SECURITY

Figura 4.24 – Dashboard of vulnerabilities identified by SemGrep in Grafana.



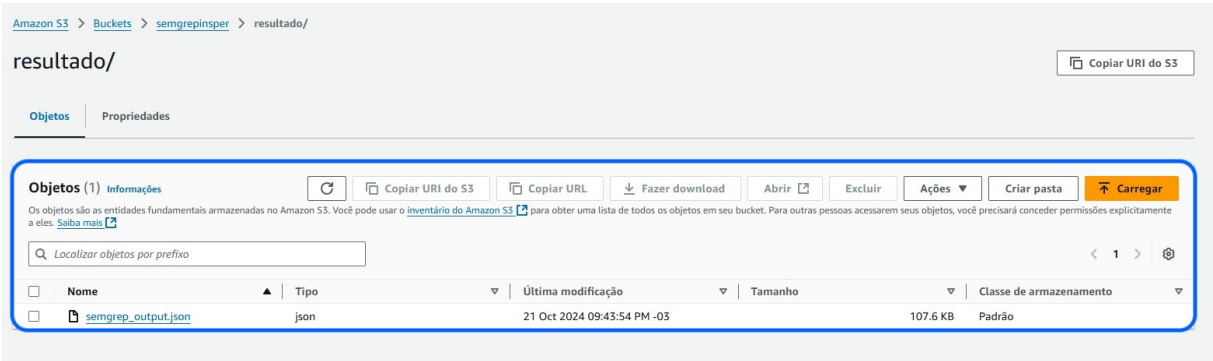
Source: Prepared by the authors.

To optimize SemGrep data flow, the data collection architecture already implemented in AWS was leveraged (see Subsection 2.3.2). The implementation process involved the following steps:

1. **Configuration in GitLab:** Integration of SemGrep into the CI/CD flow, automating the execution of security *scans* at each *build*.
2. **Storage in S3:** Creation of a dedicated *bucket*, `semgrepinsper`, to store the SemGrep output file, `semgrep_output.json`.
3. **Data Registration with AWS Glue:** Configuration of a *crawler* in AWS Glue, called `semgrep-crawler`, which searches for data in S3 and saves it to an SQL database.
4. **Cataloging in AWS Glue:** Creation of the `semgrep-database` database in AWS Glue, with cataloging of the following columns:
 - **cwe:** Unique code for each type of vulnerability (*Common Weakness Enumeration*).
 - **cwe_title:** Vulnerability description.
 - **impact:** Vulnerability impact.
 - **likelihood:** Likelihood of exploitation.
 - **message:** Explanatory message about the vulnerability.
 - **line:** Affected code line.
 - **path:** Source code file path.
5. **Data Processing with AWS Glue ETL:** Creation of a processing *job* that uses SQL queries to transform data into KPIs, storing them in S3 and in the AWS Glue Data Catalog.
6. **Making Available for Queries in AWS Athena:** Athena configuration for access to processed data, facilitating queries and vulnerability analyses.

PinPag segregates machines and infrastructure by responsibility, that is, each machine executes specific services, requiring scheduling for access to the GitLab machine and SemGrep installation (Semgrep, 2024). It was configured so that, at each activation of the development flow in GitLab, a security *scan* is performed. The result, in *.json* format, is automatically transferred to the S3 *bucket* `semgrepinsper`, stored in the `semgrep_output.json` file, as in Figure 4.25.

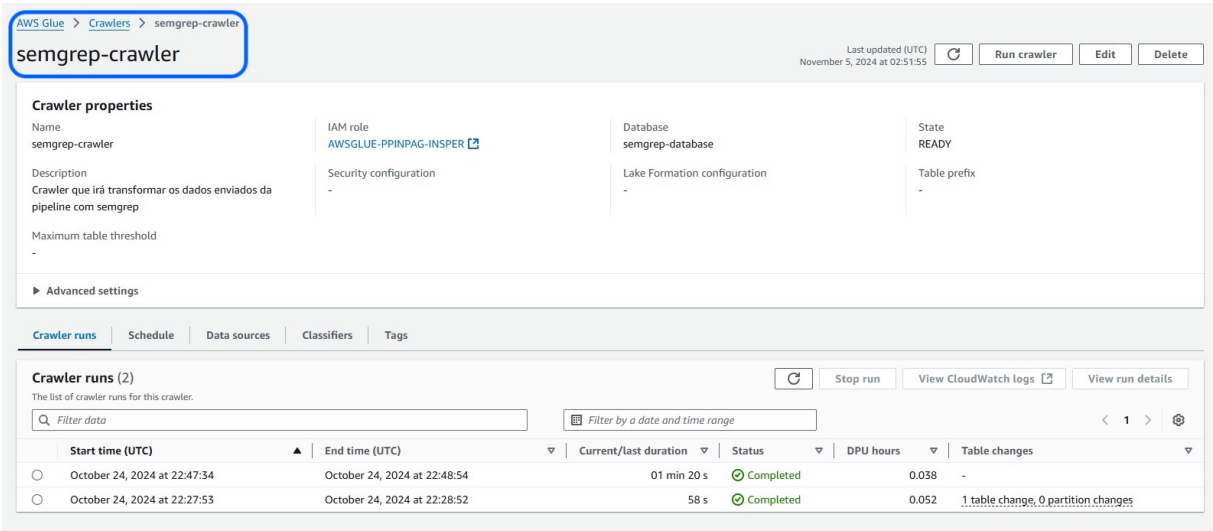
Figura 4.25 – Amazon S3 interface displaying the *bucket* dedicated to storing SemGrep results.



Source: Prepared by the authors.

After creating the *bucket*, the next step was to register the data in AWS Glue, following the same infrastructure reestablishment flow. For this, a *crawler* called **semgrep-crawler** was created, shown in Figure 4.26, to search for data in S3 and structure it in a table of the **semgrep-database** database, composed of the previously described columns, resulting in the table shown in Figure 4.27.

Figura 4.26 – AWS Glue interface displaying the *crawler* **semgrep-crawler**, responsible for registering security data in the S3 *bucket*.



Source: Prepared by the authors.

Figura 4.27 – Visualization of the semgrep-table table in AWS Glue.

AWS Glue > Tables > semgrep-table

Last updated (UTC)

November 5, 2024 at 02:52:52

Version 4 (Current version)

Actions

semgrep-table

Table overview

Data quality - new

Table details

Name

semgrep-table

Database

semgrep-database

Description

-

Last updated

October 25, 2024 at 01:25:51

Classification

JSON

Location

s3://aws-glue-plinpag-insper/processed/semgrep/

Connection

-

Deprecated

-

Column statistics

No statistics

Advanced properties

Schema

Partitions

Indexes

Column statistics - new

Schema (7)

View and manage the table schema.

Q / filter schemas

Edit schema as JSON

Edit schema

#

▼

Column name

▼

Data type

▼

Partition key

▼

Comment

▼

1

cwe

string

-

-

2

cwe_title

string

-

-

3

impact

string

-

-

4

likelihood

string

-

-

5

message

string

-

-

6

line

int

-

-

7

path

string

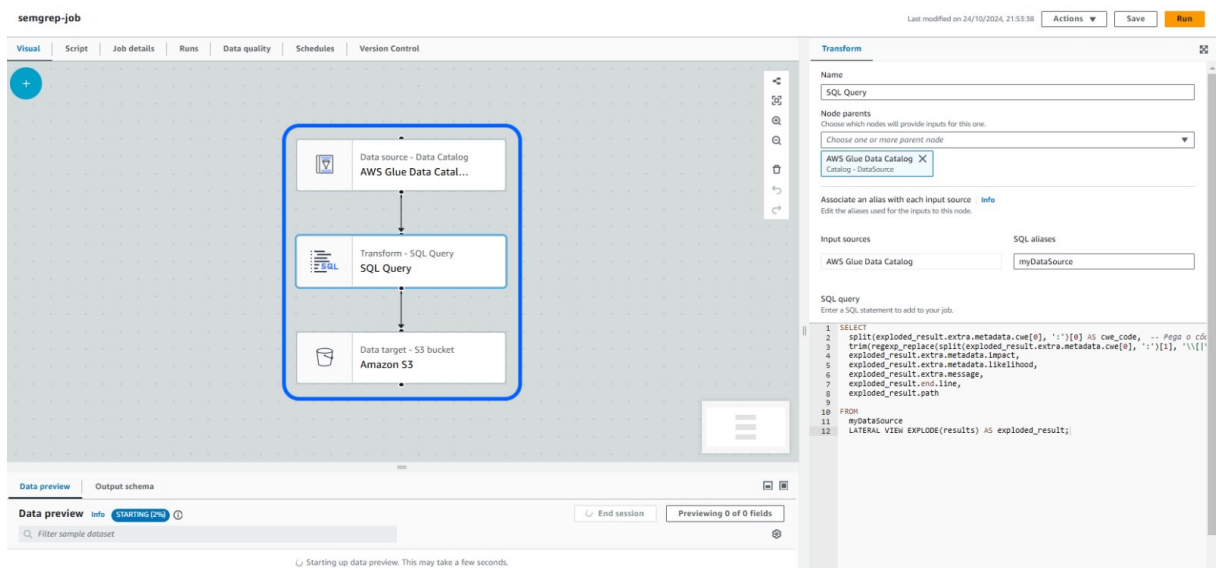
-

-

Source: Prepared by the authors.

Next, an ETL *job* was developed in AWS Glue, to perform a data transformation, and allow information to be extracted in the correct columns. As can be seen in Figure 4.28, the *Data Source* to access information that came in vector form, data is transformed by the *SQL Query* and the processed version is kept in an S3 directory. This structure ensures that data is ready for queries in AWS Athena.

Figura 4.28 – Configuration of ETL *job* in AWS Glue for processing and storing SemGrep vulnerability data.

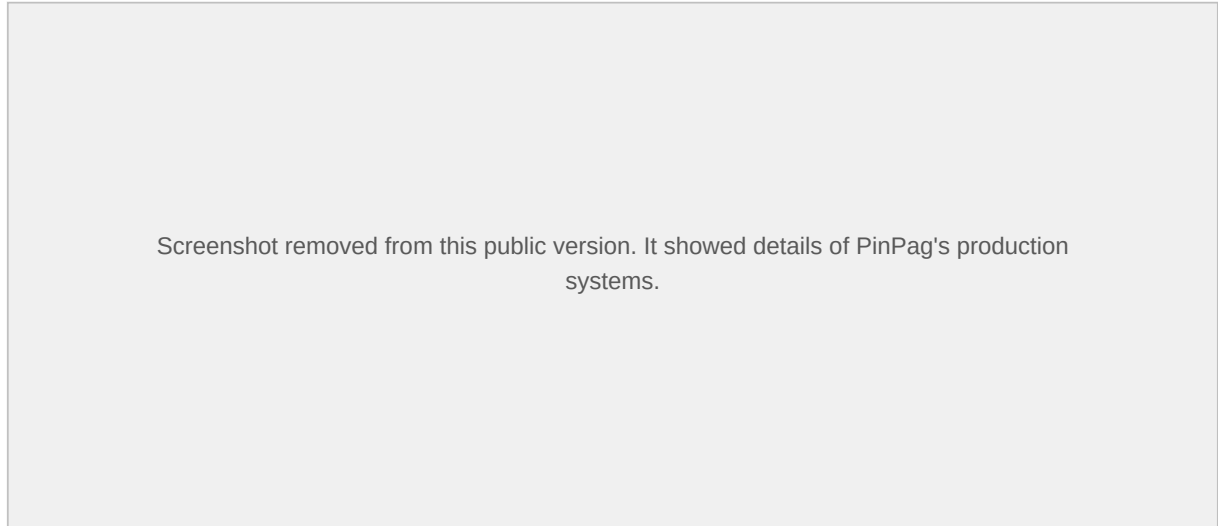


Source: Prepared by the authors.

To validate correct access to data by AWS Athena, a query was performed to

retrieve all information from the `semgrep-table` table. The verification showed that the columns are properly cataloged and accessible, as illustrated in Figure 4.29.

Figura 4.29 – AWS Athena interface showing data from the `semgrep-table` table, confirming correct access and cataloging of columns.



Source: Prepared by the authors.

This integrated flow between SemGrep, AWS Glue, Athena, and Grafana allows PinPag to monitor and respond to security risks in a proactive and effective manner. Additionally, the alert system detailed in Subsection 4.6.5 complements this integration, ensuring that any detected vulnerability is immediately notified to the IT team for prompt mitigation.

4.5 INFRASTRUCTURE DASHBOARD

The infrastructure dashboard was developed to meet the needs of the FinOps and Information Security team, described in Subsection 3.1.2, consolidating various data sources into a unified and simplified interface. The objective is to provide a centralized visualization of the main infrastructure metrics, facilitating real-time monitoring and strategic decision-making.

The KPIs used include:

1. Total Cloud Cost (Current Month vs Previous)
2. Resource Utilization (CPU and Memory) of Critical Instances
3. Percentage of Cost Change between Months

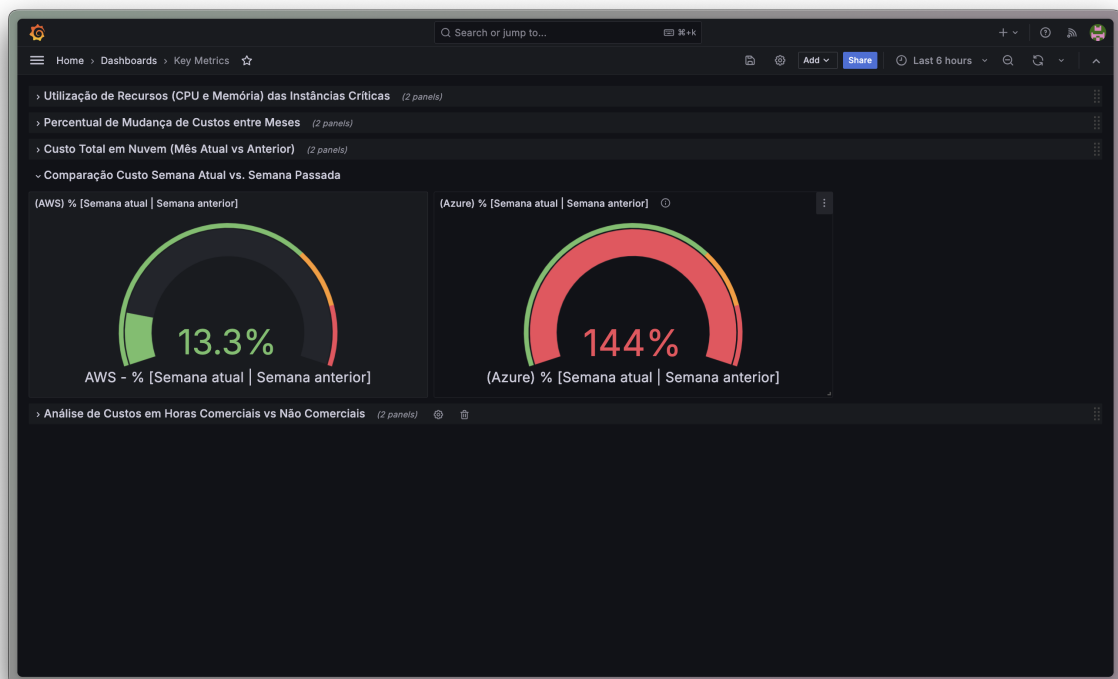
4. Current Week Costs vs Previous Week
5. Costs in Business Hours vs Non-Business Hours

For the creation of this dashboard, the granularity of data generated by AWS and Azure was validated, confirming the possibility of implementing the new weekly cost KPI. With this, it was possible to create the **"Key Metrics"** visualization in Grafana, where all existing metrics and KPIs were reused without the need for modifications in requests or data processing.

For the weekly cost KPI, two new requests were created: an SQL query for AWS Athena and a KQL for Azure, without the need for new processing levels.

The dashboard organizes visualizations into sections, allowing access only to KPIs of interest, behavior analogous to folders on a computer. The layout was designed to simplify the interface and facilitate navigation.

Figura 4.30 – Visualization of Infrastructure Dashboard: *Key Metrics*.



Source: Prepared by the authors.

4.6 ALERT SYSTEM

PinPag's Alert System was developed to communicate occurrences in real time, requiring immediate actions in response to infrastructure incidents. Using tools such as

Twilio, Telegram, and SMTP, the system sends automatic notifications to the responsible team as soon as an anomaly or problem is identified. This ensures a rapid response, minimizing operational impacts and ensuring service continuity.

The integration of these tools allows the IT team to be notified instantly through multiple channels, increasing flexibility and agility in incident resolution. Additionally, monitoring metrics become more actionable, providing real-time information that facilitates immediate interventions, reducing risks and improving infrastructure management efficiency. This mechanism contributes to quickly mitigating the impacts of possible failures, ensuring a secure and reliable environment.

4.6.1 Metrics Selected for the Alert System

The following metrics were selected, together with PinPag, for monitoring, aiming to ensure infrastructure performance and security:

1. **CPU Usage per Instance:** Monitors server overload, identifying moments of excess load and assisting in resource redistribution, optimizing infrastructure performance.
2. **RAM Usage per Instance:** Detects memory leaks and performance degradation, preventing crashes or unexpected reboots, ensuring service stability.
3. **Vulnerability Levels:** Evaluates suspicious activities and compliance with security policies, allowing rapid detection and correction of threats, ensuring that infrastructure meets security standards.
4. **Monthly Cost Comparison:** Monitors monthly expenses, assisting in budget control and identification of inefficiencies, such as unexpected cost spikes, to optimize financial management.
5. **Daily Cost Standard Deviation:** Detects anomalous patterns in daily costs, contributing to more precise financial planning and risk mitigation.
6. **Application Security Vulnerability Limit:** Monitors the number of vulnerabilities detected during the CI/CD process, ensuring that the agreed maximum limit of vulnerabilities (25) is not exceeded. This metric facilitates proactive security management and mitigation of risks related to application security.

4.6.2 Alert System Operation

The alert system monitors these metrics in real time and triggers notifications based on established conditions. Three main variables regulate operation:

1. **Rules:** Define the conditions that data must meet to trigger an alert.
2. ***Evaluation Period:*** The time interval to evaluate whether a condition is being met.
3. ***Pending Period:*** The time required for the condition to be maintained before the alert is triggered.

These parameters allow the system to detect incidents efficiently, ensuring that the IT team is notified in time to mitigate problems.

4.6.3 Infrastructure and Cost Alerts Configured in Grafana

After defining the metrics, the alert system for infrastructure and costs was configured in Grafana. The operation is supported by the following components:

1. **Alert Rules (*Alert Rules*):** Define the criteria that, when met, trigger an alert. The rules use configured requests and expressions to determine the conditions to be monitored. Once triggered, they generate alert instances (*Alert Instances*) that are sent according to established notification policies.
2. **Notification Policies (*Contact Points*):** Define how users are notified about alerts. In Grafana, methods such as email, Telegram, and Twilio were configured. The SMTP service was configured in the Grafana instance to allow communication via email using PinPag accounts.
3. **Sending *Templates*:** Each notification policy has customized *templates* for sending messages via Telegram, Twilio, and email. These *templates* were configured to provide detailed information about alerts, facilitating immediate understanding of the problem.
4. ***Labels:*** *Labels* facilitate the identification and categorization of alerts by type of service, cloud, and priority. The main *labels* include:
 - ***Cloud:*** AWS or Azure
 - ***Priority:*** *Low*, *Medium*, *High*, or *Max*
 - ***Subtype:*** Service Specifications (e.g., RAM)
 - ***Type:*** Service (e.g., Zabbix)
5. **Alert Priority Levels:** Alerts were prioritized based on two factors:
 - **Urgency:** The speed needed to resolve the problem.

- **Impact:** The severity of consequences for company operation.

The priority levels and their respective notification methods are:

- **Low Priority:** Email notification.
- **Medium Priority:** Email notification.
- **High Priority:** Notification via Telegram.
- **Maximum Priority:** Notification via phone call (Twilio).

4.6.4 Initial Tests and Alert Adjustments in Grafana

Initially, 28 alert rules were configured with reduced *Evaluation* and *Pending* periods to test infrastructure and configuration in Grafana. These tests ensured that notifications worked correctly and that rules were triggered as expected.

After tests, rules were adjusted together with PinPag to ensure that alerts were triggered only when necessary, avoiding notification overload. The objective was to find the right balance, where alerts were triggered in the correct quantity and frequency, without generating multiple daily triggers for metrics that do not require immediate action.

The *Evaluation* and *Pending* periods were adjusted as follows:

- **CPU and RAM:** *Evaluation Period* of 30 minutes and *Pending* of 30 minutes, aiming to reduce false positives and optimize alert frequency.
- **Vulnerabilities:** *Evaluation Period* of 1 day and *Pending* of 1 day, reflecting the time needed to correct vulnerabilities.
- **Cost Analysis:** *Evaluation Period* of 1 week and *Pending* of 1 week, as it is a qualitative metric that does not require daily evaluation.
- **Daily Cost Standard Deviation:** *Evaluation Period* and *Pending* of 1 week, equally as it is a qualitative metric that does not need frequent evaluations.

These adjustments ensured that alerts were triggered efficiently and relevantly, aligning with PinPag's operational needs.

4.6.5 Application Security Alerts

To complement application security monitoring, a vulnerability limit metric was defined, established together with the client. Initially, 20 vulnerabilities were detected, and

the agreed limit is 25. If the number of vulnerabilities increases by 25% relative to the initial value, an alert is triggered to the responsible team.

As these are high-priority alerts, notification sending is performed exclusively via Telegram, ensuring that the team receives notifications wherever they are immediately.

To integrate these alerts into the system, it was necessary to adjust the current infrastructure of notification *templates*, adding a new comparison key for the Semgrep *Type*. When this key is detected, the system dynamically builds the alert message, providing a personalized and relevant notification about identified vulnerabilities. This configuration ensures that application security alerts are triggered consistently and relevantly, allowing immediate interventions to mitigate risks.

4.7 ITEM REMOVAL POLICY

The item removal policy, developed in the First Phase of the Project, was already in operation at PinPag. Initially, it was configured for certain *buckets* and repositories that are no longer in use, such as `grafana-query-results-insper`. With the expansion of solutions and the increased complexity of the `aws-glue-pinpag-insper` storage directory, which now contains data from multiple solutions, the need arose to update and improve the existing policy.

4.7.1 Updates Implemented in Code

Several changes were made to the existing code, focusing on improving efficiency, security, and process flexibility:

- **Extension to New Repositories:** The code was adapted to include new *buckets* such as, for example, `aws-waf-logs-insper` responsible for storing WAF data.
- **Implementation of Advanced Logging:** Simple print statements were replaced by Python's `logging` module. This change enables more detailed control over log levels and facilitates monitoring of performed operations, contributing to rapid problem identification and preventive maintenance.
- **Performance Optimization:** To increase efficiency, the function now performs batch deletions of objects, instead of deleting them individually. By grouping up to 1,000 objects per request, the number of calls to the S3 API is reduced, improving performance and decreasing resource load.
- **Preservation of Directories with Files:** Logic was added to ensure that directories that still contain files are not deleted. The function previously verifies *prefixes* that

have content, avoiding accidental removal of folder structures necessary for system operation or data organization.

- **Improvement in Error Handling:** Exception handling was improved to capture and record in detail any errors that occur during execution. This includes capturing complete *tracebacks*, facilitating problem diagnosis and correction, in addition to ensuring that failures do not interrupt the complete execution of the process.

5 COST ESTIMATION

In this chapter, we present the updated cost estimate for the infrastructure used in the project, considering AWS and Azure services. The objective is to provide a clear view of the necessary investments, allowing the company to adequately plan its financial resources.

5.1 AWS COST ESTIMATION

As previously discussed, PinPag’s AWS infrastructure for this project uses a variety of services, such as Amazon S3, AWS Lambda, AWS Glue, Amazon Athena, AWS WAF and Shield. In this section, the estimated costs for each service are presented and, for this calculation, the AWS pricing calculator was used ([Amazon Web Services, 2024d](#)), based on the current usage estimate of the services.

5.1.1 Infrastructure Overview

The project uses the following services and resources:

- **Amazon S3:** Two *buckets* are used:
 - **aws-glue-pinpag-insper:** Contains 41,332 objects, totaling 7.2 GB of storage.
 - **aws-waf-logs-insper:** Approximately 50,000 objects were estimated, totaling 8.7 GB of storage.
- **AWS Lambda:** Three Lambda functions are in operation:
 - **MapaDeConexoes_CriacaoDosNosELinhas:** Executed once a week (Sunday at 00:00), with average duration of 35 seconds.
 - **cost_report_storage_function:** Executed daily, with average execution time of 0.323 seconds.
 - **delete_old_cur:** Executed daily, with average execution time of 14 seconds.
- **AWS Glue:** Two *jobs* and several *crawlers* are used:
 - **Jobs:**
 - * **process-data-insper:** Executed daily, with average time of 15 minutes and use of 4 *Data Processing Units* (DPUs).
 - * **semgrep-job:** Executed sporadically, approximately once a week, with average time of 3 minutes and use of 2 DPUs.

– ***Crawlers:***

- * **cur-crawler:** Executed daily, with average execution time of 58.85 seconds and use of 0.039 DPU.
- * **edge-crawler:** Executed weekly, with average execution time of 63.73 seconds and use of 0.041 DPU.
- * **node-crawler:** Executed weekly, with average execution time of 64.38 seconds and use of 0.041 DPU.
- * **semgrep-crawler:** Executed weekly, with average execution time of 64.56 seconds and use of 0.041 DPU.
- * **waf-shield-crawler:** Executed weekly, with average execution time of 73.85 seconds and use of 0.043 DPU.

– ***Databases:***

- * **mapa-conexoes-database**
- * **processed-data**
- * **raw-data**
- * **semgrep-database**
- * **waf-shield-database**

It is observed that Glue *databases* do not store duplicate data, as they all use Amazon S3 as underlying storage.

- **Amazon Athena:** Used to perform queries on data stored in Amazon S3. Based on usage, 5 active users monthly were estimated, each accessing daily all 8 dashboards available in Grafana, totaling 40 daily queries.
- **AWS WAF and AWS Shield:** The log generation and export service is used, at no additional cost.

Below, the estimated costs for each of these services are detailed, considering the specific configurations and usage of the project.

5.1.2 Amazon S3

Amazon S3 is used to store data from various services, including AWS WAF logs and data processed by AWS Glue. Two *buckets* are used:

- **aws-glue-pinpag-insper:** Contains 41,332 objects, totaling 7.2 GB of storage.
- **aws-waf-logs-insper:** Approximately 50,000 objects were estimated, totaling 8.7 GB of storage.

Totaling, the *buckets* occupy 15.9 GB of storage. As the services use less than 1,000 monthly requests and there is no data transfer out of the AWS region, additional costs of requests and data transfer are negligible.

Storage cost calculation:

- Total storage: 15.9 GB.
- Price per GB (S3 Standard): \$0.023 per GB/month.
- **Monthly cost:** $15.9 \text{ GB} \times \$0.023/\text{GB} = \0.37 .

Rounding, the estimated monthly cost is \$0.39/month.

5.1.3 AWS Lambda

AWS Lambda is used for *serverless* code execution, with three Lambda functions in operation:

- **MapaDeConexoes_CriacaoDosNosELinhas:** Executed once a week (4 times a month), with average duration of 35 seconds.
- **cost_report_storage_function:** Executed daily (30 times a month), with average duration of 0.323 seconds.
- **delete_old_cur:** Executed daily (30 times a month), with average duration of 14 seconds.

Totaling 64 monthly executions, with average duration of 8.9 seconds per execution. All functions are configured with 128 MB of RAM memory and 512 MB of storage.

Cost calculation:

- Total monthly executions: 64.
- Average duration per execution: 8.9 seconds.
- Configured memory: 128 MB.
- GB-seconds calculation: $64 \times 8.9 \text{ seconds} \times \frac{128 \text{ MB}}{1024 \text{ MB}} = 71.2 \text{ GB-seconds}$.

- AWS Lambda offers 400,000 free GB-seconds per month. As usage is well below this limit, the cost is negligible.
- **Estimated monthly cost:** \$0.00/month.

5.1.4 AWS Glue

AWS Glue is used for ETL activities and data cataloging. Two *jobs* and several *crawlers* are used.

Jobs:

- **process-data-insper:**
 - Executed daily, totaling 30 monthly executions.
 - Average duration per execution: 15 minutes (0.25 hours).
 - DPUs used: 4.
 - Total monthly DPU-hours: $30 \times 0.25 \text{ hours} \times 4 \text{ DPUs} = 30 \text{ DPU-hours}$.
 - Cost per DPU-hour: \$0.44.
 - **Monthly cost:** $30 \text{ DPU-hours} \times \$0.44/\text{DPU-hour} = \$13.20$.
- **semgrep-job:**
 - Executed weekly, totaling 4 monthly executions.
 - Average duration per execution: 3 minutes (0.05 hours).
 - DPUs used: 2.
 - Total monthly DPU-hours: $4 \times 0.05 \text{ hours} \times 2 \text{ DPUs} = 0.4 \text{ DPU-hours}$.
 - Cost per DPU-hour: \$0.44.
 - **Monthly cost:** $0.4 \text{ DPU-hours} \times \$0.44/\text{DPU-hour} = \$0.18$.

DPU-hour represents how many processing units are consumed in one hour of execution of a given service.

Crawlers:

The *crawlers* are executed according to Table 5.1 below:

Tabela 5.1 – Execution details of AWS Glue *crawlers*

Crawler	Monthly Executions	Total Duration (s)	DPUs
cur-crawler	30	$30 \times 58,85$	0,039
edge-crawler	4	$4 \times 63,73$	0,041
node-crawler	4	$4 \times 64,38$	0,041
semgrep-crawler	4	$4 \times 64,56$	0,041
waf-shield-crawler	4	$4 \times 73,85$	0,043

Source: Prepared by the authors.

The total monthly execution time of *crawlers* is 2,831 seconds.

Converting to hours:

$$\text{Total hours} = \frac{2,831 \text{ seconds}}{3,600} = 0.7864 \text{ hours}$$

Assuming an average of 0.041 DPU, the total monthly DPU-hours is:

$$\text{Total DPU-hours} = 0.7864 \text{ hours} \times 0.041 \text{ DPU} = 0.0322 \text{ DPU-hours}$$

Cost calculation:

$$\text{Monthly cost} : 0.0322 \text{ DPU-hours} \times \$0.44/\text{DPU-hour} = \$0.0142$$

Considering the minimum charge per execution, the estimated cost is \$1.74/month.

Total AWS Glue cost:

- process-data-insper: \$13.20/month
- semgrep-job: \$0.18/month
- *Crawlers*: \$1.74/month

Total monthly cost: \$15.12/month

5.1.5 Amazon Athena

Amazon Athena is used to perform queries on data stored in Amazon S3. Based on usage, 5 active users monthly were estimated, each accessing daily all 8 dashboards available in Grafana, resulting in 40 daily queries.

Data consumption calculation:

- Total data in S3: 15.9 GB.
- Number of daily queries: 40.
- Data processed per query: $\frac{15.9 \text{ GB}}{40} = 0.3975 \text{ GB}$.

Monthly cost calculation:

- Total monthly processed data: $0.3975 \text{ GB/query} \times 40 \text{ queries/day} \times 30 \text{ days} = 477 \text{ GB}$.
- Amazon Athena charges \$5.00 per TB of processed data.
- Total TB processed: $\frac{477 \text{ GB}}{1,024} = 0.4658 \text{ TB}$.
- **Monthly cost:** $0.4658 \text{ TB} \times \$5.00/\text{TB} = \$2.33$.

Rounding, the estimated monthly cost is \$2.36/month.

5.1.6 Total Estimated Monthly Cost in AWS

Based on previous estimates, the total monthly cost for AWS infrastructure and services is:

- **Amazon S3:** \$0.39/month
- **AWS Lambda:** \$0.00/month
- **AWS Glue:** \$15.12/month
- **Amazon Athena:** \$2.36/month

Total Estimated Monthly: \$17.87/month

This estimated cost considers an optimistic scenario, in which the amount of data stored in *buckets* reflects the stable volume projected for continuous system use. However, to accommodate possible scenarios in which the data volume is doubled or even tripled, it is important to consider how these increases would impact the complexity and costs of services. With data growth, AWS Glue *crawlers* and *jobs*, among other components, have longer execution and processing time. Below are the cost scenarios for the case of doubling and tripling data, without detailing all calculations again.

5.1.7 Cost Scenarios

- **Scenario 1 - Doubled Usage:** With doubled data volume, the total estimated monthly cost would increase to \$32.17/month.
- **Scenario 2 - Tripled Usage (Worst Case):** If data volume is tripled, the total estimated monthly cost would increase to \$48.26/month.

5.2 AZURE COST ESTIMATION

The Azure infrastructure used in the project includes several services, such as Cost Management + Billing, Azure Storage Blob, Azure Data Factory, and Azure Data Explorer. In this section, the estimated costs for each of these services are presented, considering the specific configurations and usage of the project. For cost calculation, usage estimates of services were made, based on current infrastructure, and the Azure pricing calculator was used ([Microsoft Azure, n.d.](#)). The infrastructure configurations remain the same as defined in the First Phase of the Project ([FREZZATTI et al., 2023](#)).

5.2.1 Infrastructure Overview

The project uses the following services and resources:

- **Cost Management + Billing:** Service used for cost data export. There is no cost associated with cost report export.
- **Azure Storage Blob:** A storage module called `pinpaginsper` is used to store data.
- **Azure Data Factory:** A service called `Cost-DataFactory-Pinpag-insper` is used for daily processing of infrastructure costs and KPIs.
- **Azure Data Explorer:** A service called `pinpag-insper` is used for data ingestion and querying.

There is no need to explain in detail the operation of each of these services, as they have already been explored in depth previously.

Below, the estimated costs for each of these services are detailed, considering the specific configurations and usage of the project.

5.2.2 Azure Storage Blob

Azure Storage Blob is used to store data in a scalable and secure way. The configurations are as follows:

- Total capacity of 50 GB.
- 100 GB of data recovery.
- Up to 100,000 monthly write and read operations.
- *Cool* (cold) access tier, suitable for data accessed less frequently.

Cost calculation:

The estimated monthly cost is \$2.08/month, composed of:

- \$1.04 referring to storage of 50 GB in *Cool* tier.
- \$0.50 for write operations (100,000 operations).
- \$0.50 for creation operations (100,000 operations).
- \$0.04 for read operations (100,000 operations).
- \$0.01 for other included operations.

All operations are at the minimum possible, as defined in the configurations.

5.2.3 Azure Data Factory

Azure Data Factory is used to orchestrate and automate data workflows. The configurations are as follows:

- Instance of 8 vCores.

- Average daily utilization of 2 hours.
- The instance is used for daily processing of infrastructure costs and KPIs.

Cost calculation:

The estimated monthly cost is \$131.91/month, composed of:

- \$131.16 for utilization of 8 vCore instance for 60 monthly hours (2 daily hours for 30 days).
- \$0.75 referring to operations and information saving.

This value is the same estimated in the previous project ([FREZZATTI et al., 2023](#)).

5.2.4 Azure Data Explorer

Azure Data Explorer is used for ingestion and querying of large data volumes. The configurations are as follows:

- Ingestion of 1 GB per day.
- Hot cache retention of 7 days.
- Total retention of 30 days.
- Use of an E2ads v5 instance, running 24 hours a day.
- Up to 22,500 write and read operations.
- Development environment (not production).
- Use of a single instance, but costs are distributed into two categories when processing values.

Cost calculation:

The estimated monthly cost is \$144.99/month, composed of:

- \$103.68 referring to processing.

- \$41.31 referring to *Data Management*.

Although only one machine is used, costs are allocated into two categories during calculation: processing and *Data Management*. The price disparity occurs because it was not possible to find the previously chosen machine (Dev(No SLA)_Standard_E2a_v4), and an available alternative was used.

5.2.5 Total Estimated Monthly Cost in Azure

Based on previous estimates, the total monthly cost for Azure infrastructure and services is:

- **Azure Storage Blob:** \$2.08/month
- **Azure Data Factory:** \$131.91/month
- **Azure Data Explorer:** \$144.99/month
- **Cost Management + Billing:** \$0.00/month (no associated costs)

Total Estimated Monthly: \$278.98/month

This estimated cost considers a scenario in which the amount of data and service usage remain stable, as projected for continuous system use. For the price calculations, the Azure pricing calculator was used ([Microsoft Azure](#), n.d.).

As the Azure environment is simple and there is only one service with data from multiple months, there is no need to calculate multiple scenarios. The presented cost reflects the estimate for continuous and stable service usage, without considering significant variations in data volume or resource usage.

6 CONCLUSION

The second phase of the project with PinPag represented a substantial advance in the optimization of monitoring infrastructure and in strengthening the company's security practices. Based on the needs and challenges identified in Section 3.1.2, comprehensive solutions were implemented that meet not only current demands, but also prepare PinPag for future expansions and improvements in a *multi-cloud* environment.

The main challenges faced were addressed in a systematic and efficient manner, as detailed below:

1. **Reestablishment of Systems after Failures:** The initial priority was to restore the data collection and generation systems interrupted due to changes in cloud infrastructure. The reconstruction of data flows on AWS and Azure platforms, as described in Sections 4.1.1 and 3.1.2, ensured the continuity of operations and the accessibility of visualizations in Grafana.
2. **Visualization of Internal Firewall Rules:** To meet the need to map *firewall* rules between instances, the *Connection Map* was developed (Section 4.3). Using Grafana and the `boto3` library, this interactive tool allowed identifying and monitoring internal connections, highlighting potential vulnerabilities according to established security policies.
3. **Monitoring Vulnerabilities at Application Layer:** The implementation of SAST tools, such as SemGrep (Section 4.4), enabled the proactive identification of vulnerabilities in application source code. Integration with Grafana allowed a clear visualization of detected flaws, facilitating the management and mitigation of security risks.
4. **Real-Time Communication about Errors and Failures:** The creation of the Alert System (Section 4.6) ensured that the IT team received instant notifications about any infrastructure incidents. Using tools such as Twilio, Telegram, and SMTP, the system ensured a quick and effective response, minimizing operational impacts and maintaining service continuity.
5. **Protection against Attacks with WAF:** The implementation of WAF (Section 3.3.2) enabled monitoring and controlling HTTPS traffic, protecting web applications against malicious connections and cyber attacks. The integration of WAF with Grafana allowed a detailed real-time visualization of attack attempts and traffic

patterns, increasing security and compliance with regulatory standards, such as PCI-DSS and ISO 27001.

6. **Simplification of Dashboards for Consolidated Analysis:** The development of the infrastructure dashboard (Section 4.5) allowed the centralization and simplification of visualizations from multiple data sources. The selected *KPIs* facilitated consolidated analysis and informed decision-making, meeting the needs of the *FinOps* team and improving operational efficiency.

In addition to these specific solutions, the integration of various tools and systems in Grafana (Section 3.3.6) provided a unified and detailed view of PinPag’s infrastructure. The use of agile practices in the development methodology (Section 3.4) ensured an efficient implementation adaptable to emerging changes and challenges.

The benefits achieved include:

- **Greater Visibility and Control:** The centralization of metrics and *KPIs* in Grafana allowed more efficient and detailed monitoring of *multi-cloud* infrastructure.
- **Improvement in Security:** The implementation of *SAST* and *SIEM* tools strengthened the protection of applications and sensitive data, reducing risks of vulnerabilities and attacks.
- **Cost Optimization:** Detailed monitoring of resources and costs enabled the identification of opportunities for optimization and expense reduction.
- **Informed Decision-Making:** Through the infrastructure dashboard, the *FinOps* team could make strategic decisions based on concrete data.
- **Rapid Response to Incidents:** The Alert System ensured that the team could react promptly to any problems or threats, minimizing operational impacts.
- **Compliance and Governance:** The implemented solutions ensured compliance with standards such as PCI-DSS and ISO 27001, strengthening PinPag’s governance and responsibility.

During implementation, several challenges were overcome, such as the restructuring of infrastructure on AWS and Azure platforms, the proper configuration of security tools in the CI/CD flow, and the efficient management of indices in Wazuh to avoid the *shards* limit. Each of these challenges was faced with a methodical and collaborative approach,

involving detailed meetings with *stakeholders* and the application of recommended security and monitoring practices.

Among the lessons learned, the importance of clear and updated infrastructure documentation stands out, the need for effective communication between the teams involved, and the relevance of adopting flexible and robust tools that can adapt to the company's specific needs. The adopted agile methodology allowed a rapid response to necessary changes and adaptations, ensuring that the project remained aligned with PinPag's strategic objectives.

As future perspectives, it is recommended to continue the expansion of implemented solutions, including the adoption of other SemGrep functionalities, such as SCA and DAST, for even more comprehensive security coverage. Additionally, the continuous integration of new metrics and *KPIs* in Grafana allows for increasingly detailed and informed analysis, supporting the evolution of PinPag's business strategies.

In summary, the second phase of the project contributed significantly to the strengthening of PinPag's monitoring and security infrastructure, providing tools and processes that ensure efficient, secure operation aligned with market best practices. The success achieved in this stage establishes a solid foundation for future continuous improvement and innovation initiatives in the company.

6.1 NEXT STEPS

The next steps aim to deepen the integration between monitoring tools and expand the security coverage of applications.

6.1.1 [section removed]

6.1.2 WAF Enhancement with New KPIs, Centralized Integration^[removed]

To improve monitoring and security analysis, it is recommended to implement new KPIs and consolidate WAF information in the security dashboard.

The specific next steps for WAF are:

1. **Implementation of new, more advanced, KPIs:** Add metrics such as:
 - *Risk Score by IP*: scoring system based on suspicious request history.
 - *X-Forwarded-For*: list of IPs, count, and percentage.
 - *Referer*: request origin, with count.
 - *Robot Detection*: detection and count of requests made by robots.
 - *Temporal metrics*, such as *Actions Over Time*, *Terminating Rule ID Over Time*, and *User Agent Over Time*, among others, as indicated in (SAMPLES, 2021).
2. **Re-evaluation of implemented KPIs:** Reorganize the dashboard prioritizing more critical metrics, such as *Actions Taken*, *Countries by Number of Request*, and *Terminating Rule ID*, consolidate correlated metrics in unified visualizations, reducing the number of redundant charts, and verify the false positive rate, that is, monitoring the proportion of incorrect blocks to adjust WAF rules.
3. **Integration of all WAFs in Dashboard:** Consolidate information from all implemented WAFs in the existing dashboard, providing a general and centralized view of infrastructure and accesses, facilitating security monitoring and analysis.
4. [item removed]

6.1.3 Enhancement of Connection Map

For the Connection Map, it is recommended to add new infrastructure components, regular adjustments to connections, based on identified vulnerabilities, and develop new rules for possible vulnerable connections, as pointed out by the company.

One of the components suggested for inclusion are *Load Balancers*. The representation of these elements allows a clear view of application instances connected to them, in addition to providing a better understanding of configured traffic distribution rules. This addition improves the ability to identify bottlenecks and critical points, strengthening control over access flow and traffic distribution in infrastructure.

It is also suggested to include nodes representing each VPC, with sub-nodes for associated *subnets*. From these *subnets*, connections with the rest of the infrastructure, including application instances and databases, should be structured hierarchically. This

approach, with VPC as the starting point, creates a segmented network visualization, highlighting the isolation and security of resources at each connectivity layer.

Currently, the Connection Map visualization is limited regarding the application of advanced filters, as it requires multiple requests, each with specific parameters. Although functional, this approach is not scalable and limits analysis customization.

To overcome this limitation, the next steps involve developing a function that allows dynamic filters directly in the visualization, exploring a graph approach. This can be done in two ways:

1. **Expansion of Current Architecture:** Develop a function that applies graph algorithms over the existing structure, allowing the creation of dynamic filters directly in the relationships of each node and connection. Thus, the existing data structure is preserved while expanding flexibility in the visualization of connections and vulnerabilities.
2. **Implementation of a Graph Database:** Consider using a graph database, such as Neo4j or Amazon Neptune, for direct queries and filters in the graph structure. This type of database is optimized to handle complex relationships, allowing advanced visualization, where connections between nodes can be explored in greater depth and filters applied more efficiently and intuitively.

6.1.4 Expansion of Application Security with SemGrep and Complementary Tools

To complement application security, it is suggested to expand the use of SemGrep, also integrating tools such as SCA for analysis of third-party components. The creation of alerts that detect sudden increments in the number of vulnerabilities is also useful for maintaining code security.

1. **Include SCA:** In addition to SAST, implement SCA in SemGrep to monitor vulnerabilities in external libraries and dependencies, increasing security scope by analyzing third-party components.
2. **Integrate DAST:** Complement SAST with DAST, allowing security tests at runtime. DAST enables the identification of contextual vulnerabilities that arise only in production environment, offering an additional security layer that detects dynamic failures and unexpected behaviors during application operation.

3. **Improve Granular Severity Notifications:** Currently, the alert system is configured only for sending messages via Telegram. It is recommended to increase the granularity of notifications, differentiating them by severity level and implementing, through adjustments in message *templates*, sending additional notifications via email and calls through Twilio for critical vulnerabilities.
4. **Blocking CI/CD Flow in GitLab:** To ensure that code meets security standards, it is recommended to block the development flow in GitLab whenever multiple vulnerabilities are detected or an increasing number of failures throughout the CI/CD stages.

6.1.5 Enhancement of Analysis Capability in Infrastructure Dashboard

With the infrastructure dashboard consolidated, the next phase does not involve simply expanding visualization modules, but rather improving the information analysis capability. As discussed and validated with PinPag in this second stage of the project, an approach for this improvement is the implementation of artificial intelligence to ingest infrastructure data and generate a high-level report. This report, prepared weekly or monthly, provides the Management and FinOps team with a detailed view of how infrastructure is behaving over time, in addition to including critical analyses to support decision-making.

The proposed approach involves the use of *Large Language Models* (LLMs), specifically ChatGPT and Claude AI, to process and interpret infrastructure data in an intelligent and contextualized way. However, this approach was not included in the scope of this report and is registered as a next step to be considered.

6.1.6 Refinement and Optimization of Alert System

With the alert system already in operation and configured for current infrastructure, a semi-annual review of metrics and alert intervals is recommended, adjusting them based on observed incident patterns to reduce false positives and ensure that alerts are triggered at necessary times.

Additionally, Twilio is currently on a development account. It is necessary to transfer the service to an official PinPag account and update the contact phone number to ensure continuity and adequacy of notifications in production environment.

6.1.7 Implementation of Staging Environment and Resilience Tests

To ensure the stability and resilience of infrastructure implemented in both parts of the project, it is recommended to create a staging environment that replicates production

conditions. This will allow controlled simulation of failures, testing infrastructure changes and, mainly, will increase the company's response capacity to interruptions.

The proposed approach consists of replicating the failure scenarios faced during phases 1 and 2 of the project, such as credential expiration, accidental directory removal, and shard limit in Wazuh. Thus, it will be simpler to validate the effectiveness of changes that are applied to the developed solutions.

REFERÊNCIAS

Agile Alliance. **Manifesto for Agile Software Development**. 2001. Disponível em: <https://agilemanifesto.org/>. Acesso em: 22 out. 2024.

Agilie Team. **What is the Software Development Life Cycle (SDLC) and How Does It Work?** 2024. Disponível em: <https://agilie.com/blog/what-is-the-software-development-life-cycle-sdlc-and-how-does-it-work>. Acesso em: 4 nov. 2024.

Alert Logic Product Management Team. **Multi-Cloud vs. Single Cloud: What's the Difference?** 2022. Disponível em: <https://www.alertlogic.com/blog/multi-cloud-vs-single-cloud-whats-the-difference/>. Acesso em: 11 nov. 2024.

Algar Telecom. **Mercado de Meios de Pagamento: Entenda como as transações são processadas**. 2017. Disponível em: <https://blog.algartelecom.com.br/mercado-de-meios-de-pagamento-entenda-como-as-transacoes-sao-processadas/>. Acesso em: 13 out. 2024.

Amazon Web Services. **Amazon EC2 Concepts - User Guide**. 2024. Disponível em: https://docs.aws.amazon.com/pt_br/AWSEC2/latest/UserGuide/concepts.html. Acesso em: 7 nov. 2024.

Amazon Web Services. **Amazon RDS - User Guide**. 2024. Disponível em: https://docs.aws.amazon.com/pt_br/AmazonRDS/latest/UserGuide/Welcome.html. Acesso em: 7 nov. 2024.

Amazon Web Services. **AWS Identity and Access Management (IAM)**. 2024. Disponível em: <https://aws.amazon.com/pt/iam/>. Acesso em: 22 out. 2024.

Amazon Web Services. **AWS Pricing Calculator**. 2024. Disponível em: <https://calculator.aws/#/>. Acesso em: 12 nov. 2024.

Amazon Web Services. **VPC Security Groups**. 2024. Disponível em: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>. Acesso em: 7 nov. 2024.

Amazon Web Services. **What is the Software Development Lifecycle (SDLC)?** 2024. Disponível em: <https://aws.amazon.com/what-is/sdlc/>. Acesso em: 4 nov. 2024.

[reference removed]

BEYER, B.; JONES, C.; PETOFF, J.; MURPHY, N. R. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016. 603 p. ISBN 9781491929124.

BEZAS, K.; FILIPPIDOU, F. Comparative analysis of open source security information & event management systems (siems). **The Indonesian Journal**

of **Computer Science**, v. 12, n. 2, p. 443–468, 2023. Disponível em: https://www.researchgate.net/publication/372732872_Comparative_Analysis_of_Open_Source_Security_Information_Event_Management_Systems_SIEMs. Acesso em: 12 nov. 2024.

Black Duck by Synopsys. **What is Software Composition Analysis?** 2024. Disponível em: <https://www.blackduck.com/glossary/what-is-software-composition-analysis.html>. Acesso em: 4 nov. 2024.

Brazil Journal. **Stone acquires Linux in US \$1.1 billion deal.** 2020. Disponível em: <https://braziljournal.com/stone-acquires-linux-in-us-11-billion-deal/>. Acesso em: 13 out. 2024.

Cloudflare. **O que é conformidade com PCI-DSS?** 2024. Disponível em: <https://www.cloudflare.com/pt-br/learning/privacy/what-is-pci-dss-compliance/>. Acesso em: 16 out. 2024.

Cockroach Labs. **Why Fintech Companies Are Choosing Multi-Cloud Architecture.** 2023. Disponível em: <https://www.cockroachlabs.com/blog/fintech-multi-cloud-architecture/>. Acesso em: 13 out. 2024.

Deimos. **Understanding GitLab: What is GitLab and How is it Used?** 2024. Disponível em: <https://www.deimos.io/blog-posts/understanding-gitlab-what-is-gitlab-and-how-is-it-used>. Acesso em: 4 nov. 2024.

DIESTEL, R. **Graph Theory**. 3. ed. Springer, 2005. 322 p. ISBN 3540261826. Disponível em: <https://www.emis.de/monographs/Diestel/en/GraphTheoryII.pdf>. Acesso em: 9 nov. 2024.

ERL, T.; PUTTINI, R.; MAHMOOD, Z. **Cloud Computing: Concepts, Technology & Architecture**. 1st edition. ed. [S.l.]: Pearson PTG, 2013. 489 p. ISBN 9780133387520.

Exame. **Stakeholders: O que são e quais são os principais tipos.** 2024. Disponível em: <https://exame.com/carreira/guia-de-carreira/stakeholders-o-que-sao-e-quais-sao-os-principais-tipos/>. Acesso em: 16 out. 2024.

FEBRABAN. **Inovações trazem mais competitividade ao sistema financeiro, diz presidente da FEBRABAN.** 2024. Disponível em: <https://portal.febraban.org.br/noticia/4146/pt-br/>. Acesso em: 13 out. 2024.

FREZZATTI, H. M.; SANTOS, J. L. P.; MAKUTA, L. S.; KI, P. W. **Gerenciamento de Recursos Multicloud com Controle de Custos, KPIs e Segurança Cibernética: Trabalho de Conclusão de Curso (Bacharelado em Engenharia da Computação/Mecânica/Mecatrônica)** - Insper, São Paulo. 2023. Disponível em: <https://repositorio.insper.edu.br/handle/11224/6789>. Acesso em: 12 nov. 2024.

Grafana Labs. **Node Graph visualization.** 2024. Disponível em: <https://grafana.com/docs/grafana/latest/panels-visualizations/visualizations/node-graph/>. Acesso em: 24 out. 2024.

IBGE - Agência de Notícias. **Internet já é acessível em 90,0% dos domicílios do país em 2021.** 2022. Disponível em: <https://www.ibge.gov.br/temas/comunicacao/segunda-onda-da-televisao-digital-2022>

//agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/34954-internet-ja-e-acessivel-em-90-0-dos-domicilios-do-pais-em-2021. Acesso em: 13 out. 2024.

IBM. **What is Application Security?** 2024. Disponível em: <https://www.ibm.com/topics/application-security>. Acesso em: 4 nov. 2024.

ISO - International Organization for Standardization. **ISO/IEC 27001:2022**: Information security, cybersecurity and privacy protection - Information security management systems - Requirements. Geneva, 2022. 19 p. Disponível em: <https://www.iso.org/standard/27001>. Acesso em: 12 nov. 2024.

Iugu. **Sistema de Pagamentos Brasileiro: Entenda o que é e como funciona**. 2021. Disponível em: <https://www.iugu.com/blog/sistema-de-pagamentos-brasileiro>. Acesso em: 13 out. 2024.

Katalon. **Bug Life Cycle vs Defect Life Cycle in Software Testing**. 2023. Disponível em: <https://katalon.com/resources-center/blog/bug-defect-life-cycle>. Acesso em: 31 ago. 2024.

MANZOOR, J.; WALEED, A.; JAMALI, A. F.; MASOOD, A. Cybersecurity on a budget: Evaluating security and performance of open-source siem solutions for smes. **PLOS ONE**, v. 19, n. 3, p. 1–24, 2024. Disponível em: <https://doi.org/10.1371/journal.pone.0301183>. Acesso em: 12 nov. 2024.

Market Screener. **An unknown buyer acquired PinPag from StoneCo Ltd**. 2024. Disponível em: <https://www.marketscreener.com/quote/stock/STONECO-LTD-46772609/news/An-unknown-buyer-acquired-PinPag-from-StoneCo-Ltd--46226603/>. Acesso em: 13 out. 2024.

Mergr. **Linux Acquires PinPag**. 2020. Disponível em: <https://mergr.com/transaction/linux-acquires-pinpag>. Acesso em: 13 out. 2024.

Microsoft Azure. **Azure Pricing Calculator**. n.d. Disponível em: <https://azure.microsoft.com/en-us/pricing/calculator/>. Acesso em: 12 nov. 2024.

OpenText. **What is Dynamic Application Security Testing (DAST)?** 2024. Disponível em: <https://www.opentext.com/what-is/dast>. Acesso em: 4 nov. 2024.

OpenText. **What is Static Application Security Testing (SAST)?** 2024. Disponível em: <https://www.opentext.com/what-is/sast>. Acesso em: 4 nov. 2024.

OWASP ModSecurity. **ModSecurity GitHub Repository**. 2024. Disponível em: <https://github.com/owasp-modsecurity/ModSecurity>. Acesso em: 11 nov. 2024.

PagSeguro. **O que é adquirente e subadquirente?** 2022. Disponível em: <https://blog.pagseguro.uol.com.br/o-que-e-adquirente-e-subadquirente/>. Acesso em: 13 out. 2024.

PCI Security Standards Council. **About PCI Security Standards Council (PCI-SSC)**. 2024. Disponível em: https://www.pcisecuritystandards.org/about_us/founding-members/. Acesso em: 16 out. 2024.

PCI Security Standards Council, LLC. **Payment Card Industry Data Security Standard: Requirements and Testing Procedures**. [S.l.], 2024. Disponível em: https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4_0_1.pdf. Acesso em: 16 out. 2024.

PinPag. **PinPag - Soluções de Pagamento**. 2024. Disponível em: <https://pinpag.com.br/>. Acesso em: 13 out. 2024.

PinPag. **Quem Somos - PinPag**. 2024. Disponível em: <https://pinpag.com.br/quem-somos>. Acesso em: 13 out. 2024.

Red Hat. **O que é CI/CD?** 2024. Disponível em: <https://www.redhat.com/pt-br/topics/devops/what-is-ci-cd>. Acesso em: 24 out. 2024.

Revista HSM. **O Brasil na liderança do Open Finance: a revolução nos pagamentos com o Pix por aproximação**. 2024. Disponível em: <https://revistahsm.com.br/o-brasil-na-lideranca-do-open-finance-a-revolucao-nos-pagamentos-com-o-pix-por-aproximacao/>. Acesso em: 13 out. 2024.

RIES, E. **A Startup Enxuta**. [S.l.]: Editora Sextante, 2011. 268 p. ISBN 9788543108629.

SAMPLES, A. **AWS WAF Operations - Kibana Dashboards for AWS WAF (README em Português)**. 2021. Disponível em: https://github.com/aws-samples/aws-waf-ops-dashboards/blob/main/README_pt.md. Acesso em: 9 nov. 2024.

SCHWABER, K.; SUTHERLAND, J. **The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game**. [S.l.]: Creative Commons, 2020. 14 p.

Semgrep. **Quickstart**. 2024. Disponível em: <https://semgrep.dev/docs/getting-started/quickstart>. Acesso em: 5 nov. 2024.

SMYčKA, M. Bakalářská práce, **Evaluation and Application of SAST Tools [online]**. 2024. SUPERVISOR : Lukáš Daubner. Disponível em: <https://is.muni.cz/th/j4bxg/>. Acesso em: 3 dez. 2024.

Statista. **Global Data Creation Volume 2010-2025**. 2021. Disponível em: <https://www.statista.com/statistics/871513/worldwide-data-created/>. Acesso em: 13 out. 2024.

Statista. **Number of Internet Users Worldwide 2005-2023**. 2023. Disponível em: <https://www.statista.com/statistics/273018/number-of-internet-users-worldwide/>. Acesso em: 13 out. 2024.

Valor Econômico. **Itaú migra para a nuvem e acelera digitalização**. 2020. Disponível em: <https://valor.globo.com/financas/noticia/2020/11/30/itau-migra-para-a-nuvem-e-acelera-digitalizacao.ghml>. Acesso em: 13 out. 2024.

VAZãO, A.; SANTOS, L.; PIEDADE, M. B.; RABADãO, C. Siem open source solutions: A comparative study. In: **2019 14th Iberian Conference on Information Systems and Technologies (CISTI)**. [s.n.], 2019. p. 1–5. Disponível em: https://www.researchgate.net/publication/338052058_SIEM_Open_Source_Solutions_A_Comparative_Study. Acesso em: 12 de nov. 2024.

VERGINADIS, Y. A review of monitoring probes for cloud computing continuum. In: BAROLLI, L. (Ed.). **Advanced Information Networking and Applications**. Cham: Springer International Publishing, 2023. p. 631–643. ISBN 978-3-031-28694-0. Disponível em: https://link.springer.com/chapter/10.1007/978-3-031-28694-0_59. Acesso em: 12 nov. 2024.

Volico. **Biggest Challenges of the Data Center Industry**. 2024. Disponível em: <https://www.volico.com/biggest-challenges-of-the-data-center-industry/>. Acesso em: 13 out. 2024.

Wazuh. **Wazuh SIEM Platform**. 2024. Disponível em: <https://wazuh.com/platform/siem/>. Acesso em: 24 out. 2024.

Zendesk. **Qual o tempo de espera ideal para um bom atendimento?** 2023. Disponível em: <https://www.zendesk.com.br/blog/tempo-de-espera-para-atendimento/>. Acesso em: 13 out. 2024.

APÊNDICE A – MODSECURITY INSTALLATION MANUAL

A.1 COMPLETE GUIDE FOR INSTALLING AND VERIFYING MOD- SECURITY 3.0 ON APACHE IN LINUX (CONSIDERATIONS FOR WINDOWS ENVIRONMENT)

A.1.1 Introduction

This guide details the installation and configuration of ModSecurity 3.0 on a Linux server running Apache. It also includes guidelines for verifying if the WAF (Web Application Firewall) is functioning correctly. Additionally, we will address some specific points that may differ if the execution environment is Windows.

A.1.2 Test Machine Information

For reference purposes, the following information describes the environment in which ModSecurity was configured and tested.

- **Machine Name:** [removed]
- **Creation Date and Time:** September 27, 2024, 3:15:05 PM UTC-03:00
- **Region:** us-central1-b
- **Machine Type:** e2-standard-2
- **CPU Platform:** Intel Broadwell
- **Architecture:** x86/64
- **Operating System:** Ubuntu 22.04 (Jammy Jellyfish)
- **Disk Space:** 85 GB (Ubuntu 22.04 image from September 2024)

A.1.3 Prerequisites

A.1.3.0.1 In Linux Environment

- Linux instance with Apache installed.
- Root access or sudo privileges.
- SSH connection to the machine (if it is a remote server).

A.1.3.0.2 In Windows Environment

For those configuring a Windows environment, the process will be similar, but with some differences:

- **Web Server:** In Windows, Apache can be installed, but IIS (Internet Information Services) is the platform's default web server. ModSecurity is natively compatible with Apache, so we recommend installing it on Apache to avoid additional complexity.
- **File System:** In Linux, ModSecurity and Apache configuration files are in directories such as `/etc/apache2` and `/usr/local/modsecurity`. In a Windows environment, these directories may vary and the path will be similar to `C:\Apache24\conf`.
- **Permissions and Log Directories:** In Linux, we verify permissions with commands like `chmod` and `chown`. In Windows, file and directory permissions configuration is done through the system interface or using `icacls` in the Command Prompt.

A.1.4 Installation Step-by-Step

A.1.4.0.1 Step 1: Install Apache

If Apache is not yet installed, use the commands below to install it on Linux. On Windows, it is necessary to download the Apache installer from Apache Lounge.

On Linux (Ubuntu)

```
sudo apt update
sudo apt install -y apache2
```

On Windows

- Download Apache from the Apache Lounge website.
- Extract the ZIP content and follow the instructions to register Apache as a Windows service.

A.1.4.0.2 Step 2: Install Dependencies

On Linux

```
sudo apt update
sudo apt install -y build-essential git libpcre3 libpcre3-dev \
    libxml2 libxml2-dev libyajl-dev libtool autoconf \
    automake pkg-config
```

On Windows Install Microsoft Visual Studio Build Tools to compile ModSecurity, or download the pre-compiled version of ModSecurity for Windows (if available). Installation and configuration may require administrator permissions.

A.1.4.0.3 Step 3: Clone the ModSecurity Repository

```
git clone --depth 1 https://github.com/SpiderLabs/ModSecurity
cd ModSecurity
```

A.1.4.0.4 Step 4: Initialize and Update Submodules

```
git submodule init
git submodule update
```

A.1.4.0.5 Step 5: Compile ModSecurity

On Linux

```
./build.sh
./configure
make
sudo make install
```

On Windows The compilation process on Windows is more complex and may require specific compilation tools.

A.1.4.0.6 Step 6: Install the Apache Connector

1. Exit the ModSecurity directory

```
cd ..
```

2. Clone the Apache connector repository

```
git clone --depth 1 https://github.com/SpiderLabs/ModSecurity-apache.git
cd ModSecurity-apache
```

3. Compile and install the connector

```
./autogen.sh
./configure --with-apxs=/usr/bin/apxs
make
sudo make install
```

A.1.4.0.7 Step 7: Configure ModSecurity

Copy the default ModSecurity configuration file:

```
sudo cp /usr/local/modsecurity/etc/modsecurity.conf-recommended \
/usr/local/modsecurity/modsecurity.conf
```

Edit the configuration file to define the operation mode:

```
sudo nano /usr/local/modsecurity/modsecurity.conf
```

- `SecRuleEngine DetectionOnly # Monitoring mode`
- `SecRuleEngine On # Blocking mode`

A.1.4.0.8 Step 8: Configure Apache to Use ModSecurity

Create a configuration file to load ModSecurity in Apache:

```
sudo nano /etc/apache2/mods-available/security3.conf
```

Add the following content:

```
LoadModule security3_module /usr/lib/apache2/modules/mod_security3.so
<IfModule security3_module>
    Include /usr/local/modsecurity/modsecurity.conf
</IfModule>
```

Enable the module in Apache:

```
sudo a2enmod security3
```

A.1.4.0.9 Step 9: Restart Apache

```
sudo systemctl restart apache2
```

A.1.5 Installation Verification

A.1.5.0.1 1. Confirm that ModSecurity is Loaded

```
apachectl -M | grep security
```

A.1.5.0.2 2. Perform a Test with Malicious Request

```
curl -I "http://IP_da_instancia/?test=<script>alert(1)</script>"
```

A.1.5.0.3 3. Check the Audit Log

```
sudo tail -f /var/log/apache2/modsec_audit.log
```

A.1.6 Final Considerations

This guide covers the installation and verification process of ModSecurity on Linux, with guidelines for adaptations on Windows. In Windows environments, some directories and commands will be different. Check the official ModSecurity documentation for specific details about Apache configuration on Windows.